

SECURE

COLLEGEWALLET

Security-focused campus identity and payment platform



ANDROID • KOTLIN • FLASK • MYSQL • SQLITE • BLUETOOTH RFCOMM

A security-focused prototype that combines digital student identity with token-based online payments and offline Bluetooth transactions, backed by replay protection, encrypted local storage, backend revalidation, and audit logging.

PROJECT HIGHLIGHTS

MOBILE + WEB

Android student wallet with browser-based merchant dashboard

TOKEN SECURITY

HMAC-SHA256 tokens, expiry validation, replay prevention

OFFLINE PAYMENTS

Bluetooth Classic transfer with SQLite queue and secure sync

SECURITY ENGINEERING

Encrypted local storage, balance checks, audit event logging

AYUDH DAHAL

Cybersecurity & Networking

GitHub: github.com/Singo69/CollegeWallet

Abstract

The increasing adoption of digital identity programs and cashless payment solutions has presented educational institutions with opportunities to modernize campus operations. Nonetheless, most colleges continue to rely on individual systems to identify students and to pay them, in addition to experiencing security, convenience, and unreliable internet issues. The project outlines the design and development of a College Digital Credential Wallet, which is a prototype system that integrates both digital student identity and token-based payment into the same system.

The deployed system is an Android application (written in Kotlin) to be used by student users, a Flask server, a web portal on a browser used by merchants, a MySQL database to store records centrally, and an SQLite offline queue to process delayed transactions. The system is compatible with online payments as well as offline payments via Bluetooth Classic and thus transactions can be initiated when there is no connectivity with network and synchronized when connectivity returns.

The system uses password hashing, signature payment tokens, token expiry verification, replay attack, audit logging and encrypted local storage of offline token data to enhance security. An Agile Scrum methodology was used to develop the project that enabled features to be added gradually and polished by repeating tests and reviews.

Unit testing, system testing and security testing were performed to test. The findings demonstrated that the prototype could successfully execute the major functions planned that includes ability to log in, to make an online payment, to make an offline payment, to synchronize transactions, to update the analytics, and to validate the token security. Overall, the project will show a viable and safe prototype of combining digital identity and campus payment service into one college-specific wallet system.

Table of Contents

Acknowledgement.....	Error! Bookmark not defined.
Abstract.....	2
1. Introduction	1
1.1. Problem Scenario	1
1.2. Problem as a Solution.....	2
1.3. Aim and Objectives.....	3
1.3.1. Aim	3
1.3.2. Objectives	3
1.4. Report Structure	4
2. Background and Literature Review	5
2.1. About the End-User	5
2.1.1. Student Users	5
2.1.2. Merchant Users.....	5
2.1.3. Administrative Users	5
2.2. Understanding the project.....	6
2.3. Review of Similar projects.....	7
2.3.1. Transact Campus.....	7
2.3.2. TouchNet OneCard Campus ID	7
2.3.3. Tappit	8
2.3.4. Lipa Payments (Reference for Bluetooth)	9
2.4. Comparison Table	10
2.5. Comparison Analysis	12
3. Development.....	13
3.1. Considered Methodology	13

3.1.1. Agile Methodology.....	13
3.1.2. Waterfall Model	14
3.1.3 Prototype Model	15
3.2. Selected Methodology and Justification.....	16
3.2.1. Agile Methodology - Scrum Framework	16
3.3. Phases of methodology	17
3.3.1. Sprint 1: Preliminary Research and Planning.....	17
3.3.2. Sprint 2: Foundation System Setup.....	17
3.3.3. Sprint 3: Android Application, Web-based merchant and Online payment...	17
3.3.4. Sprint 4: Bluetooth Offline payment development	17
3.3.5. Sprint 5: Testing and Security Adaptations and Improvements.....	17
3.3.6. Sprint 6: Finalization and Report Preparation.....	18
3.4. Survey Analysis	19
3.4.1. Pre-Survey Analysis.....	19
3.4.2. Post-Survey Analysis	20
3.5. Requirements Analysis	21
3.5.1. Hardware Requirements	21
3.5.2. Software Requirements.....	22
3.6. System Design.....	23
3.6.1. WBS Diagram (Work Breakdown Structure)	23
3.6.2. System Architecture Diagram.....	24
3.6.3. UML Diagrams	25
3.6.3.1. Use Case Diagram	25
3.6.3.2. Class Diagram	26
3.6.3.3. Sequence Diagram.....	27

3.6.3.4. Flowchart.....	30
3.6.4. Data Flow Diagram	33
3.6.5. Entity Relationship Diagram (ERD)	34
3.6.6. Wireframes.....	35
3.6.7. UI Design	36
3.7. Implementation	37
3.7.1. Android Application Implementation	37
3.7.2. Backend Implementation.....	42
3.7.3. Merchant Web portal	49
3.7.4. Database and Local storage Implementation	56
3.7.5. Security Implementation.....	60
4. Testing and Analysis	65
4.1. Unit Testing.....	65
Test Plan:.....	65
4.1.1. Test Case 1	67
4.1.2. Test Case 2.....	69
4.1.3. Test Case 3.....	73
4.1.4. Test Case 4.....	76
4.1.5. Test Case 5.....	78
4.1.6. Test Case 6.....	81
4.1.7. Test Case 7	83
4.1.8. Test Case 8.....	85
4.1.9. Test Case 9.....	88
4.1.10. Test Case 10.....	90
4.1.11. Test Case 11.....	92

4.1.12. Test Case 12.....	94
4.1.13. Test Case 13.....	96
4.1.14 Test Case 14.....	98
4.1.15. Test Case 15.....	101
4.2. System Testing.....	104
Test Plan:.....	104
4.2.1. Test Case 1.....	105
4.2.2. Test Case 2.....	110
4.3. Security Testing.....	116
Test Plan:.....	116
4.3.1. Test Case 1.....	117
4.3.2. Test Case 2.....	120
4.3.3. Test Case 3.....	123
4.3.4. Test Case 4.....	126
4.3.5. Test Case 5.....	129
4.3.6. Test Case 6.....	131
4.3.7. Test Case 7.....	133
4.4. Critical Analysis.....	135
4.4.1. Project Deliverables Evaluation.....	135
4.4.2. Limitations and Challenges.....	135
5. Conclusion and Future Works.....	136
5.1. Legal, Social and Ethical Issues.....	136
5.1.1. Legal Issues.....	136
5.1.2. Social Issues.....	136
5.1.3. Ethical Issues.....	137

5.2. Advantages:.....	137
5.3. Future Work	138
5.4. Conclusion	139
8. References	140
9. Appendix:	142
9.1. Technical Project Terms	142
9.1.1. Tokenization.....	142
9.1.2. HMAC-SHA256	142
9.1.3. Encrypted Local Storage	143
9.1.4. Bluetooth Classic RFCOMM	143
9.1.5. Synchronization and Offline Queue.....	143
9.1.6. Audit Logging	144
9.2. Actual System Architecture Overview	145
9.3. Extended Implementation	146
9.3.1. Android Application Details (Continued).....	146
9.3.2. Backend Service Implementation (Continued)	146
9.3.3. Merchant Web Portal Implementation (Continued)	147
9.3.4. Database and Local Storage Implementation (Continued).....	147
9.3.5. Security Mechanisms Implementation (Continued)	148
9.4. Future Works Explained	149
9.4.1. Biometric Authentication Integration.....	149
9.4.2. Real-Time Notification System	149
9.4.3. Institution-Wide Multi-Merchant Expansion	149
9.4.4. Advanced Administrative Dashboard	149
9.4.5. Backup and Recovery Based on Clouds	150

9.4.6. Cross-Platform Application Support	150
9.5. Gantt Chart	151
9.6. Survey:	154
9.6.1. Pre-Survey Form	154
9.6.2. Pre-Survey Form Filled Sample	160
9.6.3. Pre-Survey Response	166
9.6.4. Post-Survey Form	173
9.6.5. Post-Survey Form Filled Sample.....	179
9.6.6. Post-Survey Response.....	185

Table of Figures

Figure 1: Transact Campus (Transact, 2023).....	7
Figure 2: TouchNet (TouchNet, 2023).....	8
Figure 3: Tappit (Tappit, 2021).....	8
Figure 4: LipaPayments (Lipapayments, 2021).....	9
Figure 5 : Agile Methodology (nvisia, 2020)	13
Figure 6: Waterfall Model (Frank, 2025).....	14
Figure 7: Prototype Model (More, 2023).....	15
Figure 8: Agile-Scrum Framework (Truong, 2025)	16
Figure 9 : WBS Diagram	23
Figure 10 : System Architecture	24
Figure 11: Use Case Diagram.....	25
Figure 12: Class Diagram.....	26
Figure 13: Login Sequence	27
Figure 14: Online Payment Sequence.....	28
Figure 15: Offline Tap and Sync Sequence.....	29
Figure 16: Flowchart A (Login Operation)	30
Figure 17: Flowchart B (Online Process).....	31
Figure 18: Flowchart C (Offline Bluetooth Tap)	31
Figure 19: Flowchart D (Sync Process).....	32
Figure 20: Data Flow Diagram (DFD0).....	33
Figure 21: Data Flow Diagram (DFD1).....	33
Figure 22: Entity Relationship Diagram (ERD)	34
Figure 23: Wireframe 1 (Login and Dashboard)	35
Figure 24: Wireframe 2 (Online and Offline Payment Popup)	35
Figure 25: UI Design 1 (Login Page).....	36
Figure 26: UI Design 2 (Dashboard).....	36
Figure 27: Login API call and session creation	38
Figure 28: Login API call and session creation (continued).....	38
Figure 29: Online merchant discovery and automatic selection	39
Figure 30: Online tokenized payment flow	39

Figure 31: Online tokenized payment flow (continued 1).....	40
Figure 32: Online tokenized payment flow (continued 2).....	40
Figure 33: Offline Bluetooth Classic payment flow	41
Figure 34: Offline Bluetooth Classic payment flow (continued)	41
Figure 35: Student Login Endpoint.....	42
Figure 36: Student Login Endpoint (continued)	43
Figure 37: Online token issue endpoint.....	44
Figure 38: Online token issue endpoint (continued)	45
Figure 39: Tokenized online payment consumption	46
Figure 40: Tokenized Online Payment Consumption (continued)	46
Figure 41: Offline token pack succeed	47
Figure 42: Merchant Discovery Endpoint	48
Figure 43: Merchant Login route	49
Figure 44: Merchant Login route (continued)	50
Figure 45: Merchant Dashboard Route	51
Figure 46: Merchant Dashboard Route (continued)	52
Figure 47: Offline pending route	53
Figure 48: Offline Queue sync route.....	54
Figure 49: Offline Queue sync route (continued).....	55
Figure 50: Tables in database.....	56
Figure 51: Users table	57
Figure 52: Transactions table	57
Figure 53: Online payment tokens table.....	57
Figure 54: Offline payment tokens table.....	58
Figure 55: Audit events table.....	58
Figure 56: Merchant presence table.....	58
Figure 57: Offline queue storage process	59
Figure 58: Encrypted offline token storage	60
Figure 59: Online replay / expiry validation	61
Figure 60: Offline sync security validation	62
Figure 61: Offline sync security validation (Token expire and replay)	63

Figure 62: Offline sync security validation (Insufficient balance)	64
Figure 63: Merchant Invalid Credentials	67
Figure 64 Android Invalid Credentials	68
Figure 65: Merchant Valid Credentials	69
Figure 66: Merchant Login Successful	70
Figure 67: Android Valid Credentials	71
Figure 68 Android Login Successful	72
Figure 69: Canteen not accepting online payment	73
Figure 70: Coffee Station not accepting online payment	74
Figure 71: Stationary not accepting online payment	74
Figure 72: Android shows no one is accepting nearby	75
Figure 73: Canteen merchant only accepting online payment	76
Figure 74: Canteen was shown and selected automatically	77
Figure 75: Canteen accepting online payment	78
Figure 76: Coffee Station accepting online payment too	79
Figure 77: Stationary accepting online payment too	79
Figure 78: All 3 merchant being displayed	80
Figure 79: Adding details for online payment	81
Figure 80: Online payment Successful	82
Figure 81: Android Displaying successful online transaction history	83
Figure 82: Merchant web displaying successful online transaction history	84
Figure 83: Canteen not accepting offline payment	85
Figure 84: Coffee Station not accepting offline	86
Figure 85: Stationary not accepting offline payment	86
Figure 86: Android Shows unavailable when no one is accepting nearby	87
Figure 87: Canteen starts accepting offline payment	88
Figure 88: Canteen verified as nearby accepting merchant	89
Figure 89: Adding details for offline payment	90
Figure 90: Offline payment successful	91
Figure 91: Offline payment displayed in transaction history	92
Figure 92: Pending payment in merchant after student transfer	93

Figure 93: Successful sync after syncing the payment.....	94
Figure 94: Successfully displayed in transaction history after syncing	95
Figure 95: Analytics before.....	96
Figure 96: Total 2000 tokens transaction online.....	97
Figure 97: Analytics after.....	97
Figure 98: Analytics before offline payment	98
Figure 99: Offline payments pending.....	99
Figure 100: Successful offline transactions	99
Figure 101: Analytics after offline payments.....	100
Figure 102: Merchant logout Confirmation	101
Figure 103: Merchant Logout Successful	102
Figure 104: Android Logout Confirmation.....	102
Figure 105: Android Logout Successful.....	103
Figure 106: Adding valid student details for login	105
Figure 107: Login Successful	106
Figure 108: No one accepting online payment.....	106
Figure 109: Merchant Online tapping started	107
Figure 110: Canteen gets selected automatically.....	107
Figure 111: Payment Successful.....	108
Figure 112: Student Transaction History	109
Figure 113: Canteen Transaction History.....	109
Figure 114: Adding valid login details.....	111
Figure 115: Login Successful	111
Figure 116: No one nearby accepting offline payment	112
Figure 117: Stationary starts accepting offline payment.....	112
Figure 118: Stationary gets verified and selected automatically.....	113
Figure 119: Payment Successful.....	114
Figure 120: Student Transaction history.....	114
Figure 121: Payment pending in merchant web	115
Figure 122: Transaction History after syncing	115
Figure 123: Successful Payment.....	117

Figure 124: Online Payment log with details	118
Figure 125: Same token used for testing.....	118
Figure 126: Token replay failed since it was already used	119
Figure 127: Successful Offline Payment	120
Figure 128: Offline Payment logs with details	121
Figure 129: Same token used for testing.....	121
Figure 130: Token replay failed since it was already used	122
Figure 131: Successful Online Payment	123
Figure 132: Payment logs with details.....	124
Figure 133: Using same token id for expiration test	124
Figure 134: Failed as token got expired	125
Figure 135: Successful Offline Payment	126
Figure 136: Payment logs with details.....	127
Figure 137: Using same token id for expiration test	127
Figure 138: Offline payment failed since token was expired	128
Figure 139: Android package normal file showing raw data	129
Figure 140: Android package secure offline token file showing encrypted token	130
Figure 141: Initial balance of the user	131
Figure 142: Insufficient balance for online payment.....	132
Figure 143: Initial User balance.....	133
Figure 144: Insufficient balance for offline payment	134
Figure 145: Gantt Chart.....	151
Figure 146: Gantt Chart (List View).....	153
Figure 147: Pre-Survey Form 1	154
Figure 148: Pre-Survey Form 2.....	155
Figure 149: Pre-Survey Form 3.....	156
Figure 150: Pre-Survey Form 4	157
Figure 151: Pre-Survey Form 5.....	158
Figure 152: Pre-Survey Form 6	159
Figure 153: Pre-Survey Form Filled Sample 1	160
Figure 154: Pre-Survey Form Filled Sample 2	161

Figure 155: Pre-Survey Form Filled Sample 3	162
Figure 156: Pre-Survey Form Filled Sample 4	163
Figure 157: Pre-Survey Form Filled Sample 5	164
Figure 158: Pre-Survey Form Filled Sample 6	165
Figure 159: Pre-Survey Response 1	166
Figure 160: Pre-Survey Response 2	166
Figure 161: Pre-Survey Response 3	167
Figure 162: Pre-Survey Response 4	167
Figure 163: Pre-Survey Response 5	168
Figure 164: Pre-Survey Response 6	168
Figure 165: Pre-Survey Response 7	169
Figure 166: Pre-Survey Response 8	169
Figure 167: Pre-Survey Response 9	170
Figure 168: Pre-Survey Response 10	170
Figure 169: Pre-Survey Response 11	171
Figure 170: Pre-Survey Response 12	171
Figure 171: Pre-Survey Response 13	172
Figure 172: Pre-Survey Response 14	172
Figure 173: Post-Survey Form 1	173
Figure 174: Post-Survey Form 2	174
Figure 175: Post-Survey Form 3	175
Figure 176: Post-Survey Form 4	176
Figure 177: Post-Survey Form 5	177
Figure 178: Post-Survey Form 6	178
Figure 179: Post-Survey Form Filled Sample 1	179
Figure 180: Post-Survey Form Filled Sample 2	180
Figure 181: Post-Survey Form Filled Sample 3	181
Figure 182: Post-Survey Form Filled Sample 4	182
Figure 183: Post-Survey Form Filled Sample 5	183
Figure 184: Post-Survey Form Filled Sample 6	184
Figure 185: Post-Survey Response 1	185

Figure 186: Post-Survey Response 2.....	185
Figure 187: Post-Survey Response 3.....	186
Figure 188: Post-Survey Response 4.....	186
Figure 189: Post-Survey Response 5.....	187
Figure 190: Post-Survey Response 6.....	187
Figure 191: Post-Survey Response 7.....	188
Figure 192: Post-Survey Response 8.....	188
Figure 193: Post-Survey Response 9.....	189
Figure 194: Post-Survey Response 10.....	189
Figure 195: Post-Survey Response 11.....	190
Figure 196: Post-Survey Response 12.....	190
Figure 197: Post-Survey Response 13.....	191
Figure 198: Post-Survey Response 14.....	191
Figure 199: Post-Survey Response 15.....	192

Table of Tables

Table 1: Comparison Table	11
Table 2: Unit Testing: Test Plan	66
Table 3: Unit Testing 1	67
Table 4: Unit Testing 2	69
Table 5: Unit Testing 3	73
Table 6: Unit Testing 4	76
Table 7: Unit Testing 5	78
Table 8: Unit Testing 6	81
Table 9: Unit Testing 7	83
Table 10: Unit testing 8	85
Table 11: Unit Testing 9	88
Table 12: Unit Testing 10	90
Table 13: Unit Testing 11	92
Table 14: Unit Testing 12	94
Table 15: Unit Testing 13	96
Table 16: Unit Testing 14	98
Table 17: Unit Testing 15	101
Table 18: System Testing Test Plan.....	104
Table 19: System Testing 1.....	105
Table 20: System Testing 2.....	110
Table 21: Security Testing Test Plan.....	116
Table 22: Security Testing 1.....	117
Table 23: Security Testing 2.....	120
Table 24: Security Testing 3.....	123
Table 25: Security Testing 4.....	126
Table 26: Security Testing 5.....	129
Table 27: Security Testing 6.....	131
Table 28: Security Testing 7.....	133

1. Introduction

Educational institutions have experienced a rapid digital transformation which leads campuses to implement technology-based solutions for handling identity management and financial operations (Zhang, 11). The worldwide digital identity solutions industry will grow to \$49.5 billion by 2026 and 133.98 USD Billion by 2035.(Market Research Future, 2025).

The current campus systems use physical ID cards and QR codes and separate payment applications which create disjointed user experiences and expose multiple security vulnerabilities. The systems operate without proper integration between identity verification and payment processing (IEEE, 2004).

The merging of digital identity management with secure payment processing enables the creation of a single system which boosts security while making operations easier for users. The project develops an integrated digital credential wallet which functions as a campus-specific solution for educational institutions.

1.1. Problem Scenario

Traditional college identification and payment systems face several critical challenges:

- **Security Vulnerabilities:** Physical ID cards face security risks because they can be lost or stolen and duplicated easily whereas QR codes remain vulnerable to screenshot attacks and duplication. Existing digital payment systems function through continuous internet connectivity which generates operational limits for businesses that operate in areas with unreliable network access (Kieseberg, 2014).
- **User Inconvenience:** Students and staff must carry multiple cards and remember various credentials for different campus services. The separate systems for identification and payment and access control create an inefficient user experience.

- **Limited Offline Capability:** Most digital payment systems need constant internet access which makes them useless during network outages and in places with weak network signals like campus basements and remote buildings.

1.2. Problem as a Solution

The College Digital Credential Wallet solves these problems by uniting safe identity management with token-based payment processing systems. The solution will:

- **Implement Strong Encryption:** The solution will use AES-256 encryption to protect data at rest and during transmission and requires transport layer security implementation to protect sensitive information.
- **Tokenization Framework:** The solution will replace actual credentials and payment information with dynamically generated tokens like Apple Pay to stop sensitive information from being exposed during transactions.
- **Offline Transaction Capability:** The system uses Bluetooth Classic RFCOMM - based communication to process transactions without internet access and synchronize data between devices when their connection becomes available.
- **Unified User Experience:** The system provides an application for identification, access control and payment functionality to decrease user mental effort while making college operations more efficient.

1.3. Aim and Objectives

1.3.1. Aim

To design and develop a secure, integrated digital credential wallet for college environment that supports both online and offline token-based payments while demonstrating advanced cybersecurity principles and providing superior user experience compared to existing solutions.

1.3.2. Objectives

- To create a mobile application which provides secure digital credential storage and token-based payment functionality through standard encryption methods.
- To create a tokenization system which produces one-time dynamic tokens for transactions to stop both replay attacks and credential leaks.
- To create an offline transaction system which operates through Bluetooth Classic RFCOMM technology while maintaining secure operations without internet access.
- To build a secure backend system which contains authentication and authorization features together with audit logging capabilities.
- To design and implement a modular system architecture that allows for future integration with additional campus services and systems, providing scalability and extensibility for evolving institutional needs.

1.4. Report Structure

This project report is structured across several chapters to clearly and logically present background study, development process, testing, and overall evaluation.

- **Chapter 1 - Introduction** describes the current situation and the problem that is being solved. After that, it describes how the proposed system serves as a solution, followed by the project aim and objectives.
- **Chapter 2 - Background and Literature Review** gives information about the end, user / client and describes the project domain. It also examines the current similar projects and presents a comparison table with the analysis to give reasons for the proposed solution.
- **Chapter 3 - Development** discusses the methodologies used in the project along with justifications for their usage. The various phases of development, survey, requirement, design and implementation of the system are presented here.
- **Chapter 4 - Testing and Analysis** covers the testing strategy employed along with the results of unit testing and system testing of the developed system. It further deals with the critical analysis of the deliverables, security measures, challenges faced and limitations encountered during the testing process.
- **Chapter 5 – Conclusion** summarises the overall progress , future works and restates the importance of the project outcomes.

Finally, References list all sources used, and the Appendix contains supporting materials as required.

2. Background and Literature Review

2.1. About the End-User

2.1.1. Student Users

The primary end users of the proposed system are college students who require a fast and secure means of accessing their digital identity card and performing token, based payments within the campus environment. Generally, students use physical ID cards and cash, based payments, which are susceptible to being lost, copied, or becoming inconvenient during rush hours. A digital wallet eliminates the necessity of carrying physical documents while also enhancing convenience and enabling tracking.

2.1.2. Merchant Users

The secondary end users are college vendors/merchants (cafeteria, stationery, events desk, etc.) who need a straightforward method of accepting student payments and confirming transactions. Additionally, merchants require dependable operations even when the internet is not available because campus networks may be unstable, or certain areas may have weak connectivity.

2.1.3. Administrative Users

The administrative users are college administrators, who are responsible for managing student accounts, vendors, and monitoring system activity through reports and audit logs. Admin, level access is essential for maintaining system integrity, dealing with account issues, and ensuring that the system is in line with institutional policies.

2.2. Understanding the project

This project proposes a **College Credential Digital Wallet System** that combines two key functions:

1. Digital Credential (Identity Card)

Students are allowed access to the mobile application for an authentic and aesthetic display of the digital ID card, basic profile details, and wallet information (token balance and history). The main point is to lessen the usage of physical ID cards and offer safe credential access through the mobile application.

2. Token-Based Payments (Online + Offline)

The system supports two payment modes:

- **Online tap / online payment:** When internet is available, the app sends a payment request to the backend service. The backend verifies the user and balance, executes the transaction, records it in the database, and returns a confirmation.
- **Offline tap / offline payment:** When internet is not available, the student selects offline tap. The app interacts with a Bluetooth connected merchant device. The Bluetooth connected device checks student identity proof (signature/credential proof) and payment amount rules locally and returns approve/decline. Approved offline transactions are recorded locally in an offline queue and later synchronized with the backend when the internet becomes available.

Overall, the project focuses on security and reliability by combining **secure authentication, transaction logging**, and an **offline-first payment flow** supported by later synchronization.

2.3. Review of Similar projects

To understand the design and implementation of a College Credential Digital Wallet, it was necessary to examine several existing platforms and related solutions.

2.3.1. Transact Campus

Transact Campus delivers a campus, centric ecosystem that encompasses integrated payments, campus ID, and campus commerce services. As a complete platform, it is built to link students, campus services, and merchants together in one managed system. Besides that, Transact facilitates the use of an integrated payment method with the addition of a campus ID feature in the same ecosystem. (Transact, 2023)

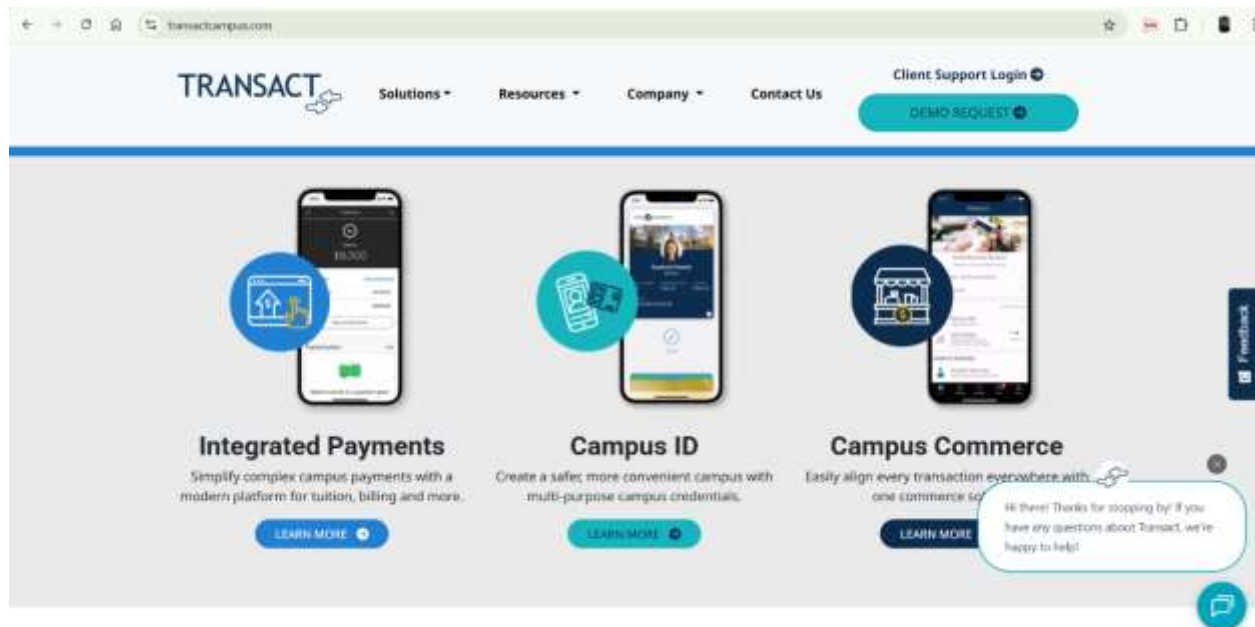


Figure 1: Transact Campus (Transact, 2023)

2.3.2. TouchNet OneCard Campus ID

TouchNet OneCard Campus ID is a campus identity and credential solution designed for universities. The system emphasizes the delivery of a digital campus ID experience, which entails support for mobile credentials via its OneCard Mobile ID product. (TouchNet, 2023)



Figure 2: TouchNet (TouchNet, 2023)

2.3.3. Tappit

Tappit is a cashless payments platform that supports tap-based payments and is designed for high, volume environments such as events. Tappit emphasizes cashless payment capability and mentions that systems can work even without WiFi, making it a relevant point when offline transaction handling is being discussed. (Tappit, 2021)

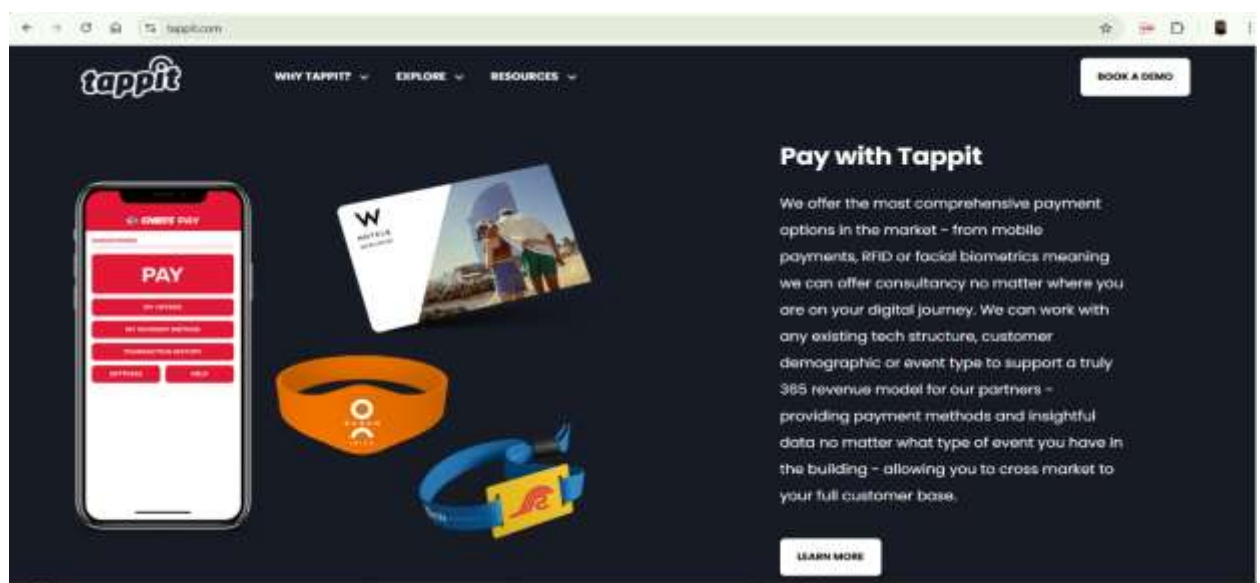


Figure 3: Tappit (Tappit, 2021)

2.3.4. Lipa Payments (Reference for Bluetooth)

Lipa Payments is a payment software solution that comes with merchant and consumer, facing apps. The company provides technology to businesses (e. g., banks and fintech) to allow contactless transactions using regular smartphones, instead of a dedicated hardware. LipaPayments talks about a Bluetooth, based payment capability (Lipa Pay) and refers Bluetooth as the technology that is used to enable offline mobile payments. (Lipapayments, 2021)



Figure 4: LipaPayments (Lipapayments, 2021)

2.4. Comparison Table

Features	Transact Campus	TouchNet OneCard	Tappit	Proposed College Credential Digital Wallet
Users	Students, staff, campus services, merchants	Students, staff, campus services, merchants	Event organizers, staff, attendees	Student users, college admin, vendor merchant, Bluetooth merchant device
Targeted Market	Universities and large campuses	Universities and large campuses	Events, festivals, venues	College campus environment (prototype for student credential + payments)
Credential Type	Mobile credential and campus ID card ecosystem	Campus ID and OneCard ecosystem	RFID wristbands / event access IDs	Digital ID card inside Android app plus wallet tokens
Payments Mode	Online connected payments and campus commerce	Online connected payments and campus commerce	Primarily on-site cashless payments	Online tap payment plus offline Bluetooth based tap payment with later sync
Hardware Dependency	Relies on campus infrastructure and supported readers	Relies on campus infrastructure and supported readers	RFID wristbands and event readers	Bluetooth based merchant device for offline, internet and server for online

Scope	Broad campus ecosystem: ID, commerce, services	Broad campus ecosystem: ID, access, payments	Event-only payment ecosystem	Focused academic prototype: ID + token wallet + online/offline payment flows
Cost / Deployment	High, requires institutional rollout	High, requires institutional rollout	Commercial event deployment	Low-cost prototype suitable for college FYP demonstration

Table 1: Comparison Table

2.5. Comparison Analysis

Based on the comparison table, the reviewed systems show that cashless payment and digital credential platforms already exist in different domains. Transact Campus and TouchNet OneCard are two examples of full campus ecosystems that integrate various services such as student ID, access, dining, and payment services.

Tappit concentrates largely on event, based payments through RFID wristbands and dedicated event devices. The system is great in high, volume event environments. However, it is not designed for academic identity credentialing and student verification workflows that a college system requires. Consequently, while it serves the purpose of understanding cashless transaction models, it falls short in meeting the combined need of digital student credential plus campus wallet functionality.

In comparison to those systems, the College Credential Digital Wallet is planned as a simple and efficient tool that merges two major ideas in one system: a student digital ID card and a token, based payment wallet.

The comparison, in general, reflects that present systems are either too extensive and costly (enterprise campus platforms) or too limited and domain, specific (event payment solutions). The proposed initiative is at a middle, ground by concentrating on a college, level scope with core features, and at the same time, showing sophisticated ideas such as offline transaction approval and subsequent synchronization, thereby enhancing the project's technical contribution.

3. Development

3.1. Considered Methodology

3.1.1. Agile Methodology

Agile approach focuses on the iterative development, collaboration with customers, and responsiveness to change via short developmental cycles known as sprints. It is a fitting strategy especially in those projects that have changing requirements and those that require periodic stakeholder feedback (IEEE, 2025).



Figure 5 : Agile Methodology (nvisia, 2020)

3.1.2. Waterfall Model

The waterfall model is linear and sequential where one stage has to be finished before the other commences. Although this gives it structure, it does not have much flexibility on how much it can change once it has begun developing (Geeksforgeeks, 2023).

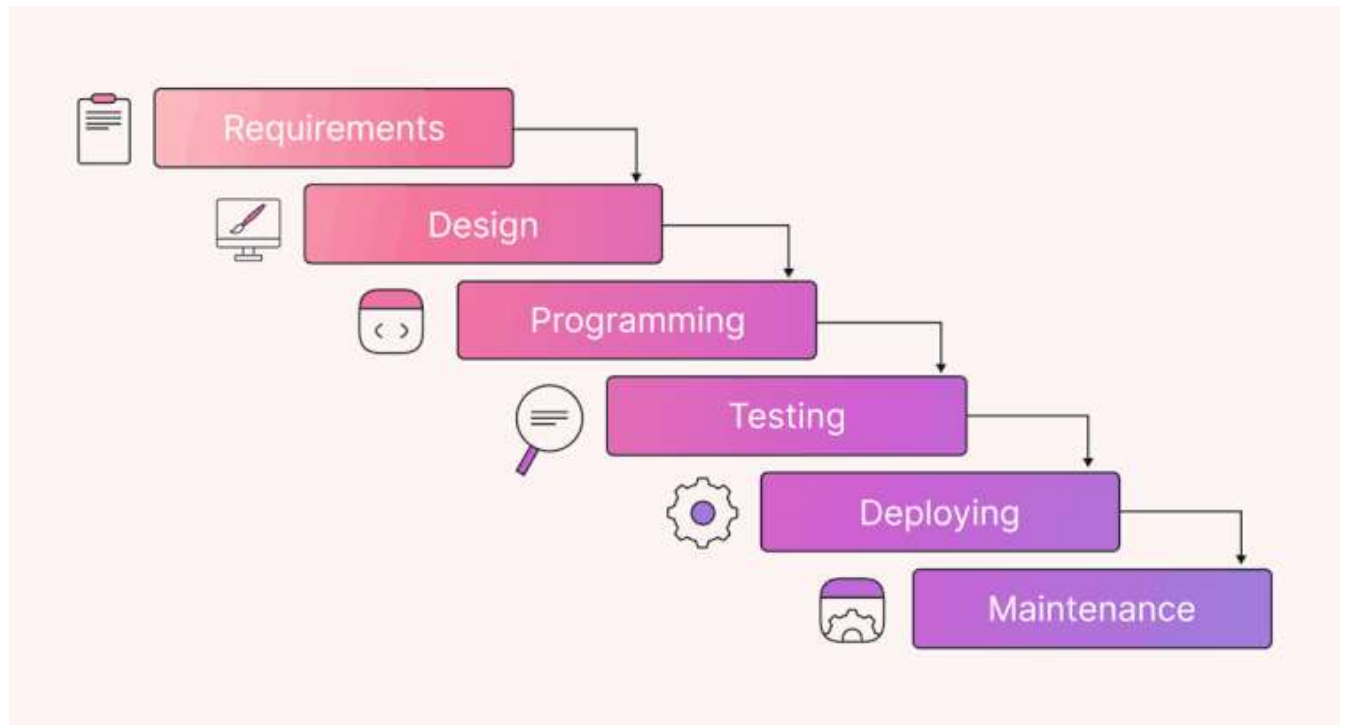


Figure 6: Waterfall Model (Frank, 2025)

3.1.3 Prototype Model

Prototype model is the model that emphasizes the development of a working prototype that is quickly built to allow users to give feedback to the final product development. The strategy is useful in cases where the requirements are not clear and user feedback is required (Relia, 2023).

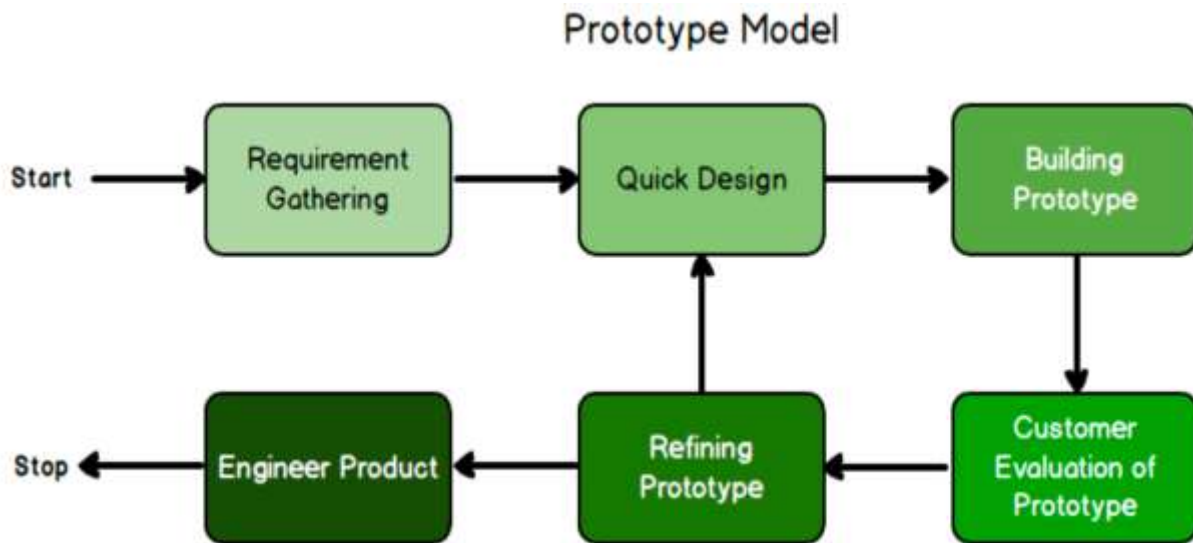


Figure 7: Prototype Model (More, 2023)

3.2. Selected Methodology and Justification

3.2.1. Agile Methodology - Scrum Framework

Scrum has been chosen as the framework of the project because of its approach towards complex software development, which is structured and flexible at the same time. The primary advantages of Scrum for this project include its emphasis on **transparency** through visible artifacts like product backlogs and sprint reviews, which ensures all stakeholders maintain clear understanding of project progress. Scrum has three pillars that have a good fit to the project requirements:

- **Transparency:** All project artifacts, such as the product backlog, sprint backlog, and increments will be made available with transparency so that there is a clear understanding of the project progress.
- **Inspection:** Periodic sprint reviews and retrospectives will provide the ability to constantly evaluate the progress and foresee possible problems at the initial stages of the development process.
- **Adaptation:** Scrum is iterative and can be made to consider the feedback and modify the requirements as the project progresses, which is crucial in the security-oriented nature of this project (Drumond, 2018).

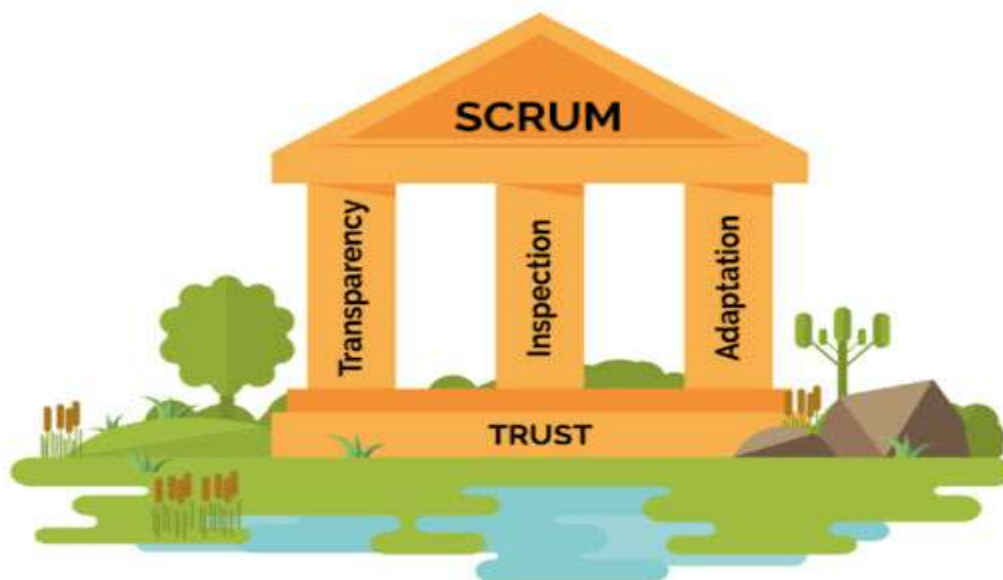


Figure 8: Agile-Scrum Framework (Truong, 2025)

3.3. Phases of methodology

The project was developed using Agile Scrum methodology where the development was broken down into short and manageable sprints. This method was appropriate since the system evolved in the process of implementation particularly in the aspects of offline payment flow, security, testing, and the refinement of the interface. Rather than adhering to a set of steps, Scrum enabled the project to be enhanced in small steps, with each sprint having a deliverable that informed the next step.

3.3.1. Sprint 1: Preliminary Research and Planning

This sprint was dedicated to background research, identification of requirements, and setting out of the project goal and scope. The mobile application, backend, merchant interface, database and payment workflow were also planned initially.

3.3.2. Sprint 2: Foundation System Setup

This step was dedicated to setting up the Android project, Flask backend, MySQL database and the simple API framework. This sprint was characterized by the creation of login flow, database connection, and project structure.

3.3.3. Sprint 3: Android Application, Web-based merchant and Online payment

The Android app and web that was developed on both the student and merchant side during this sprint had key screens, like login, dashboard, wallet, and transaction history. The online payment workflow was also implemented and connected with the backend.

3.3.4. Sprint 4: Bluetooth Offline payment development

This sprint was about the offline payment feature via Bluetooth communication. Local queuing, offline token processing and subsequent synchronization were invented and improved in this stage.

3.3.5. Sprint 5: Testing and Security Adaptations and Improvements

This sprint enhanced security features like token validation, replay rejection, expiry checking, password hashing and encrypted storage. This was also done in unit testing, system testing and security testing.

3.3.6. Sprint 6: Finalization and Report Preparation

The last sprint was devoted to bug fixing, refining the UI, gathering of screenshots, updating reports, and final assessment. It made sure that the report and system implemented were similar.

For Gantt Chart, please refer to [Appendix 9.5](#).

3.4. Survey Analysis

3.4.1. Pre-Survey Analysis

The pre-survey was done prior to the development of the system to find out the issues with current college IDs and payment systems. A total of 14 responses were collected. The results showed that 92.9% of respondents still used a physical ID card for college identification, mainly for identity verification and attendance. No one had made card payments at the canteens indicating that the current ID system was not integrated to make payments.

In campus payments, 66.7% primarily involved the usage of QR-based payment systems, the rest utilized cash or a combination of both. Furthermore, 64.3% reported they often had payment issues due to bad or no internet connection and 35.7% reported they occasionally did. This strongly pointed to the necessity of the offline support of transactions.

The survey revealed that there was displeasure with existing systems. When choosing issues like the loss of cards, inconvenience, and internet addiction, half of the respondents answered all the above. Moreover, 71.4% rated offline payment capability as very important, and 100% wanted to use one mobile application instead of physical ID cards and use different payment systems.

In general, the pre-survey strongly supported the idea of creating a secure digital wallet that combines student identification, online payment, and offline Bluetooth-based payment capabilities.

For pre-survey results, please refer to [Appendix 9.6.1](#).

3.4.2. Post-Survey Analysis

The post-survey was done when the users had tried the developed prototype to gauge usability, performance, security and acceptance. There were 20 responses gathered. The outcomes were very favorable. The system was considered easy to use by most users: 85% of users rated the ease of logging in as very easy, 65% that the interface was very clear, and 70% of users that displaying the digital student ID was very easy.

The payment features were also positively received. In the case of online payment, 65 percent were very satisfied and 35% satisfied. In case of offline Bluetooth payment, 55% considered it very effective and 45% considered it to be effective. Likewise, three-fourths of the offline users described offline transactions as very helpful upon reconnection.

Performance and trust of the system were rated highly as well. Half of the respondents indicated that payment operations were very fast and the other half indicated that payment operations are fast. Concerning security, 70% were very secure when using the system. Moreover, 85% found the combination of digital ID and payment features in one application very useful.

Overall, post-survey findings indicate that the created prototype was able to handle the problems detected during the pre-survey and had a high level of practical value, usability, and user acceptance.

For post-survey results, please refer to [Appendix 9.6.2](#).

3.5. Requirements Analysis

Requirement analysis is a procedure of recognizing and capturing the resources required to design, develop, test and evaluate a software system successfully. In this project, the requirement analysis was conducted to identify the hardware and software requirements needed in the implementation of the College Digital Credential Wallet system in a realistic and cost-effective academic setting. The requirements were chosen on the basis of feasibility, availability and appropriateness to show online and offline payment processes since the project is a prototype designed on a final year project.

The Android application development, backend API development, database management, offline transaction management, offline communication through Bluetooth, offline testing, and report preparation were the resources needed in the project. The following hardware and software requirements were used throughout the project development process.

3.5.1. Hardware Requirements

The hardware specifications of this project were made realistic and minimal in that the system would be created and tested within the existing academic facility.

- **Development Laptop / PC:** A laptop or personal computer that is powerful enough to execute Android Studio, Visual Studio Code, Python backend applications, MySQL, and testing software.
- **Android Smartphone:** This is necessary to install and test the student wallet application in an actual mobile setting.
- **Storage Capacity:** The Android project files, the backend source code, database files, screenshots, diagrams, report documents, and other data needed to be stored locally.

An expensive hardware configuration was not needed since the project was developed as a prototype system and not as a large-scale production deployment.

3.5.2. Software Requirements

The software requirements of this project contain the tools, frameworks, programming languages, libraries and platforms on which the system is built and tested.

- **Android Studio:** It is an application that is used to develop the Android student wallet application.
- **Kotlin and Jetpack Compose:** Kotlin and Jetpack Compose are used to implement the application interface and logic of Android.
- **Visual Studio Code:** Python scripting, project file management and backend development.
- **Python:** Python is used in developing the Flask backend and the offline Bluetooth receiver/synchronization modules.
- **Flask:** A web framework to create REST API endpoints and web functionality on the merchant side.
- **MySQL:** This is the main database to store user, wallet, merchant, token, audit, and transaction data.
- **SQLite:** local storage of queued offline transactions before being synchronized.
- **Retrofit:** An Android application that is used to connect with Flask backend.
- **Bluetooth Classic / RFCOMM support libraries:** The libraries are employed to realize and test the offline payment communication flow.
- **GitHub:** Version control and project backup where necessary.
- **Microsoft Word:** Report writing and formatting.
- **Draw.io / diagram tools:** This is used to create system diagrams, UML diagrams, flowcharts, and design artefacts.

The reason why these software tools were chosen was because they were practical, available and capable of implementing the project within the parameters of an academic prototype.

3.6. System Design

3.6.1. WBS Diagram (Work Breakdown Structure)

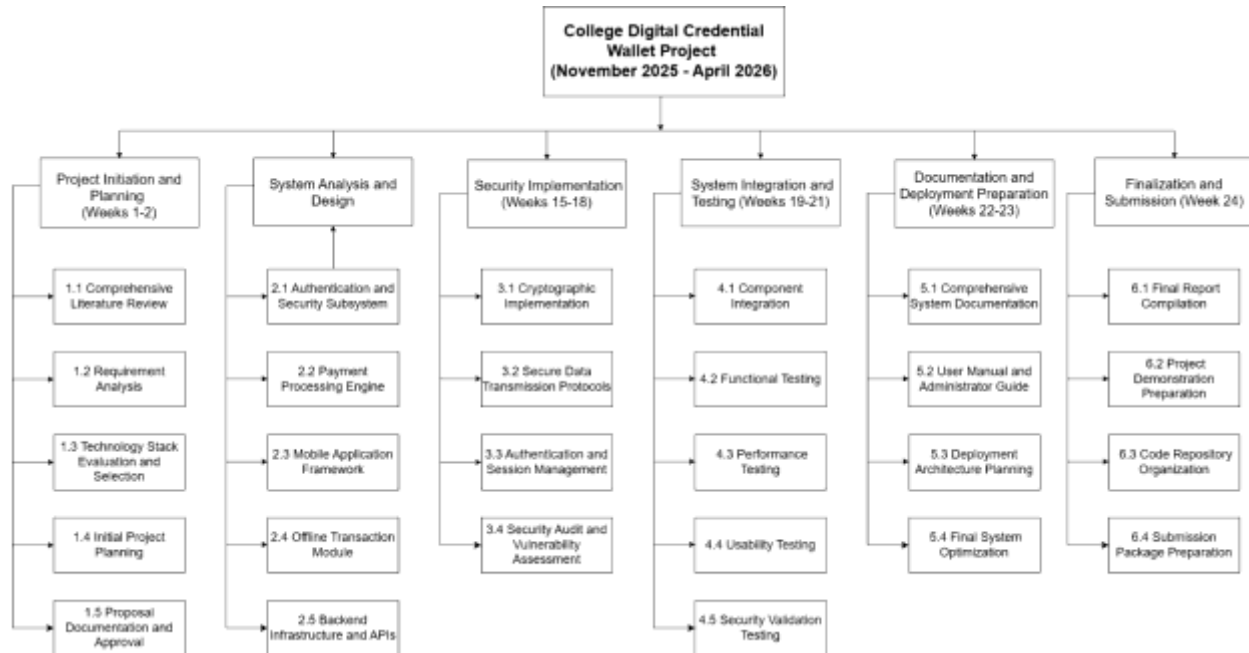


Figure 9 : WBS Diagram

3.6.2. System Architecture Diagram

The system uses a client–server architecture. The Android app works with a Flask REST API that handles and returns data from a MySQL database. Network requests for online payments are routed through the bank/payment gateway. Offline payment functionality is carried out by the app through a Bluetooth tap with the merchant device, and transactions are saved in a local offline queue which is later synced to the backend when the internet connection is available.

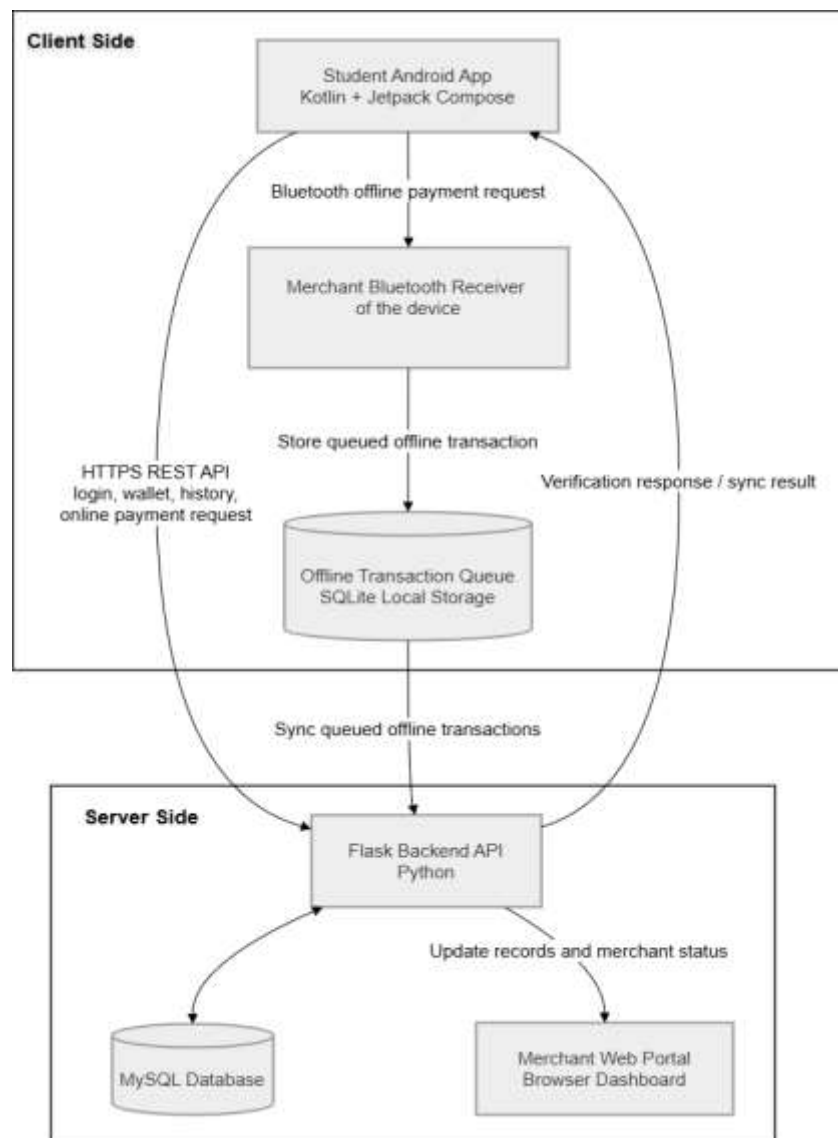


Figure 10 : System Architecture

A more detailed explanation of the actual implemented architecture is provided in [Appendix 9.2](#).

3.6.3. UML Diagrams

UML diagrams are used to visualise the system from different perspectives. In this report, the UML diagrams included are the **Use Case Diagram**, **Class Diagram**, and **Sequence Diagram**, which together describe the system's functionalities, structure, and interaction flow.

3.6.3.1. Use Case Diagram

A Use Case Diagram shows the main actors of a system and the key functions they can perform. In this project, the diagram shows how Student User, Vendor Merchant, College Admin, Bluetooth Merchant Device, and Bank/Payment Gateway interact with the College Credential Digital Wallet System.

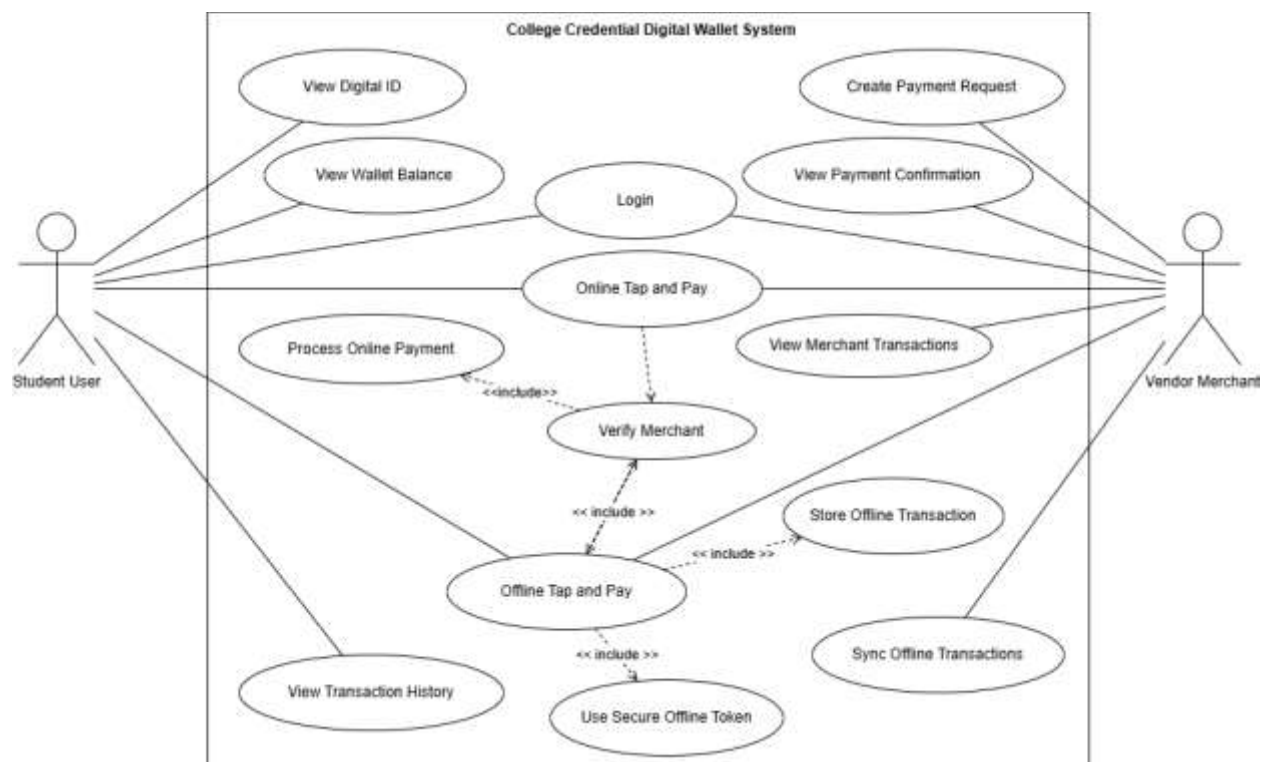


Figure 11: Use Case Diagram

3.6.3.2. Class Diagram

Class Diagram shows the system's structure by presenting the main classes, their attributes and methods, and the relationships between them. In this project, the diagram illustrates how the Android app screens communicate with the backend using Retrofit, how offline transactions are temporarily stored using an OfflineQueueRepository, and how the Flask backend uses DAO classes to read/write user, wallet, and transaction data in the database.

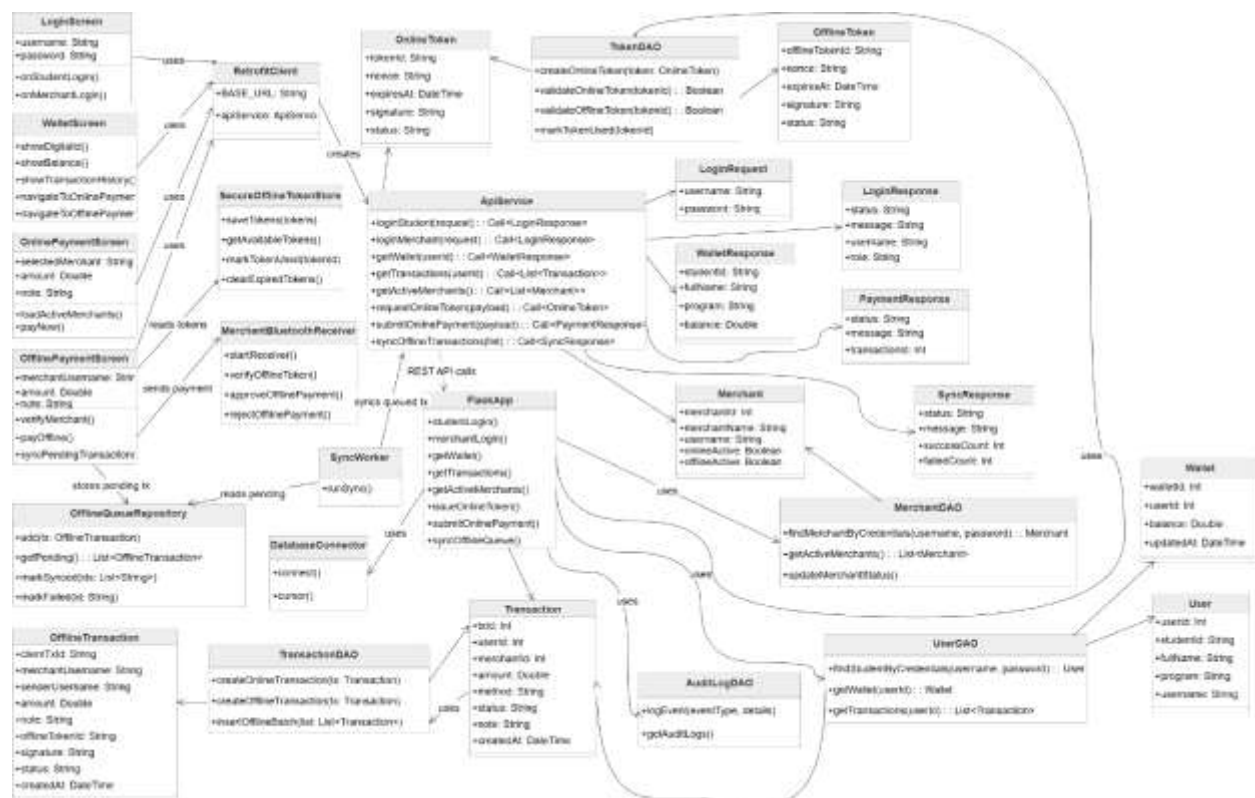


Figure 12: Class Diagram

3.6.3.3. Sequence Diagram

A Sequence Diagram shows the time-ordered messages exchanged between actors/components to complete a specific scenario.

- **Login Sequence:**

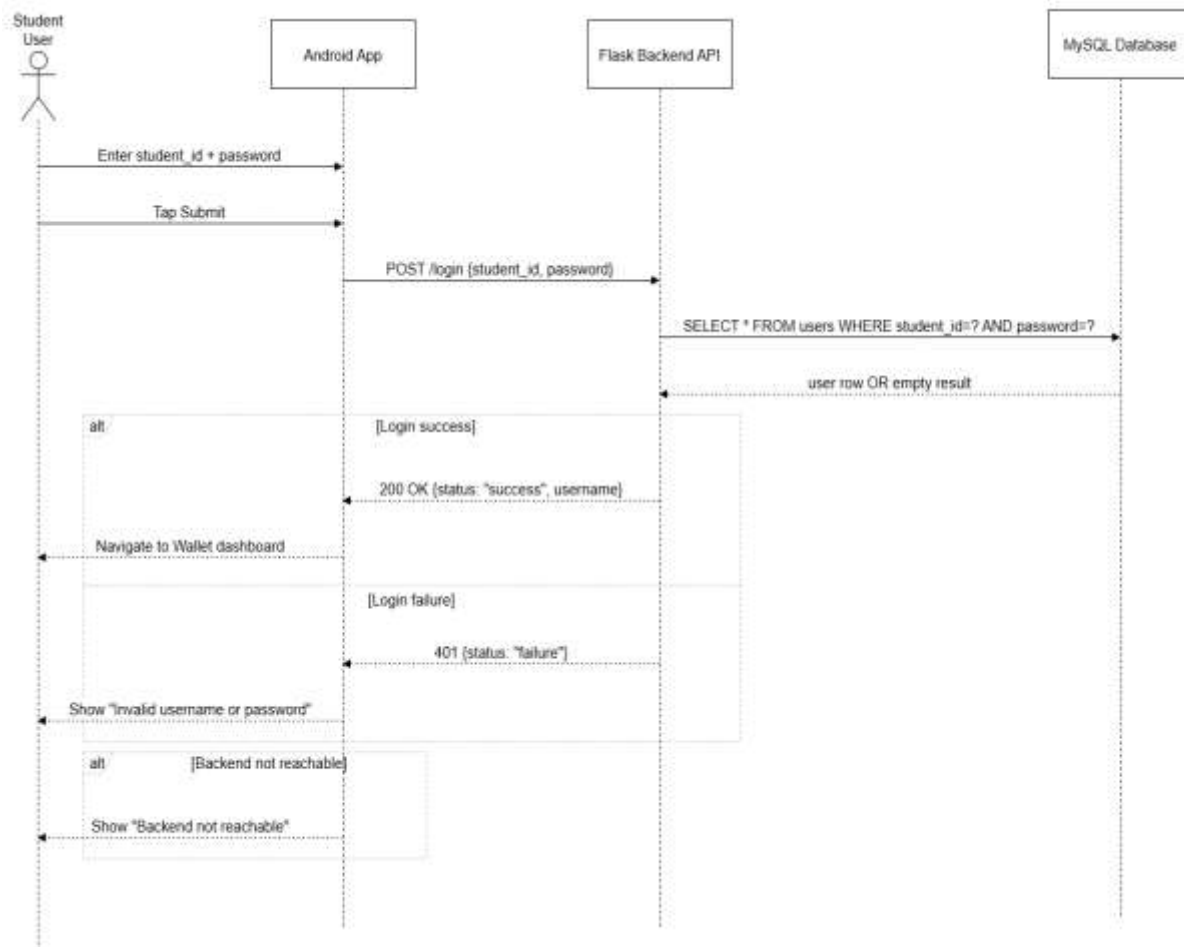


Figure 13: Login Sequence

- **Online Payment Sequence**

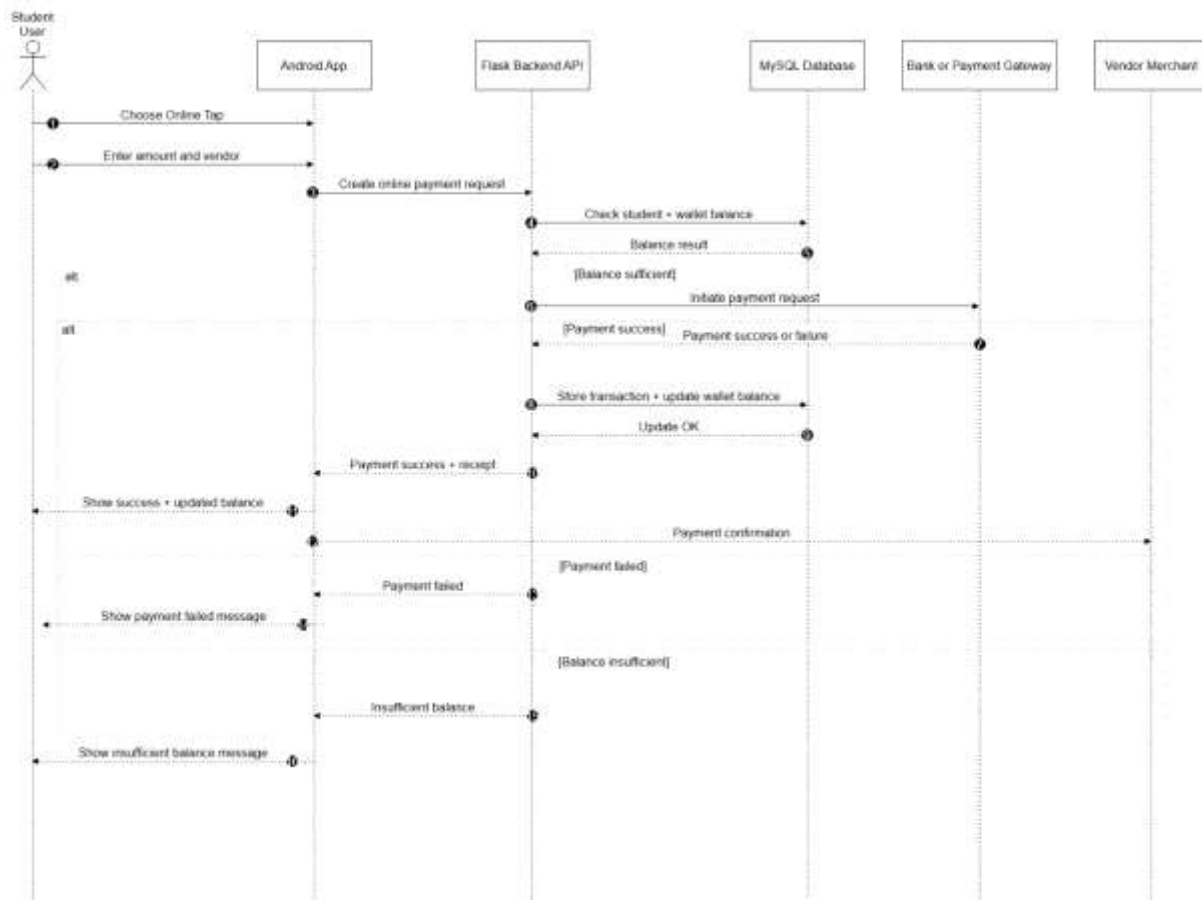


Figure 14: Online Payment Sequence

- Offline Tap and Sync Sequence

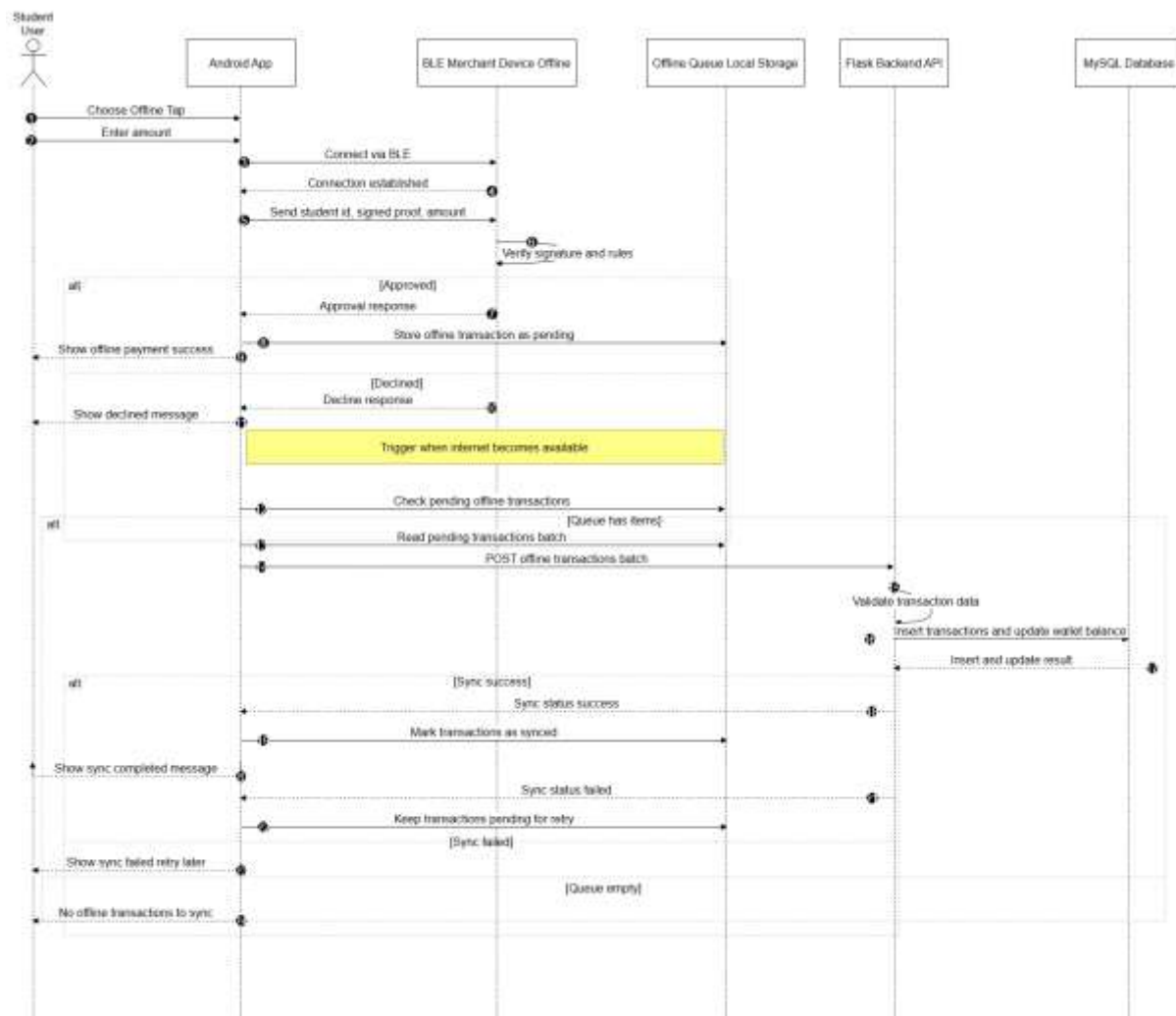


Figure 15: Offline Tap and Sync Sequence

3.6.3.4. Flowchart

A flowchart is used to represent the step-by-step logic of a process using actions and decision points.

- **Flowchart A:** In this project, Flowchart A shows the login and option selection.

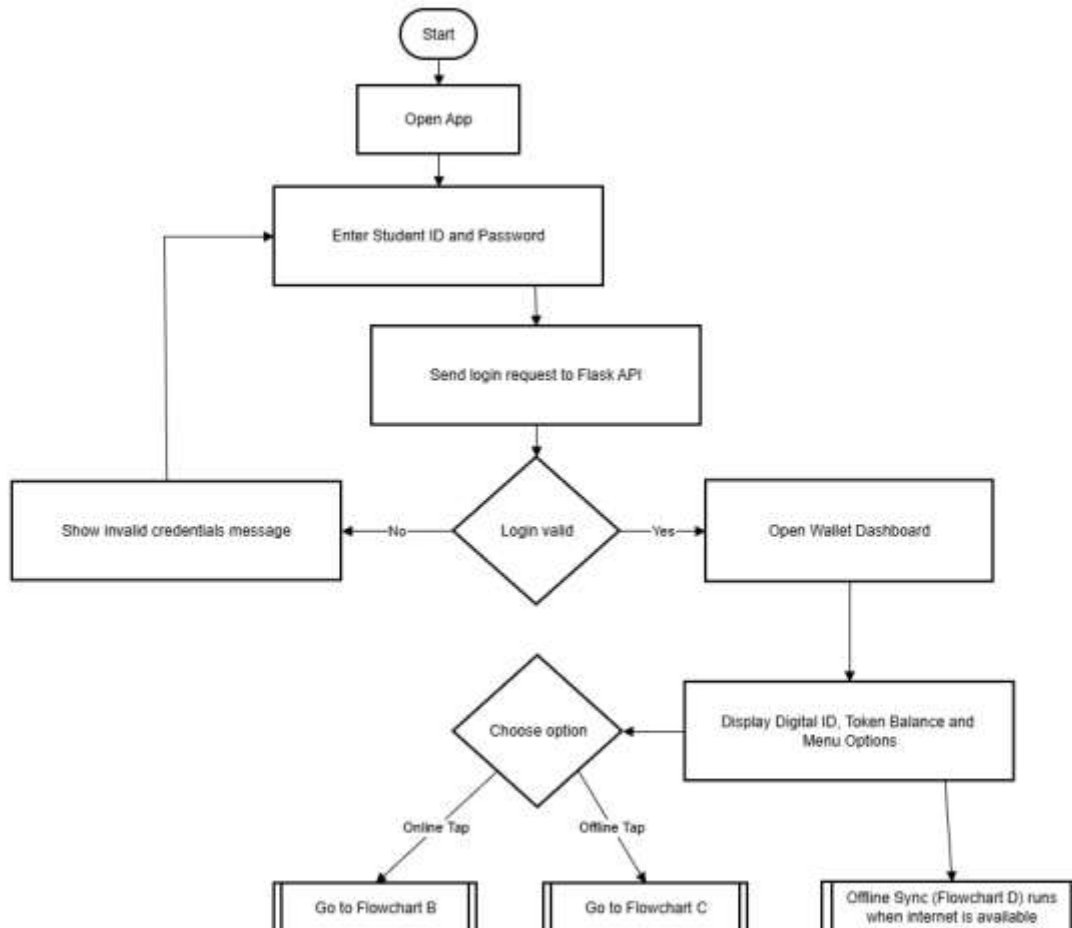


Figure 16: Flowchart A (Login Operation)

- **Flowchart B:** It shows the online payment process.

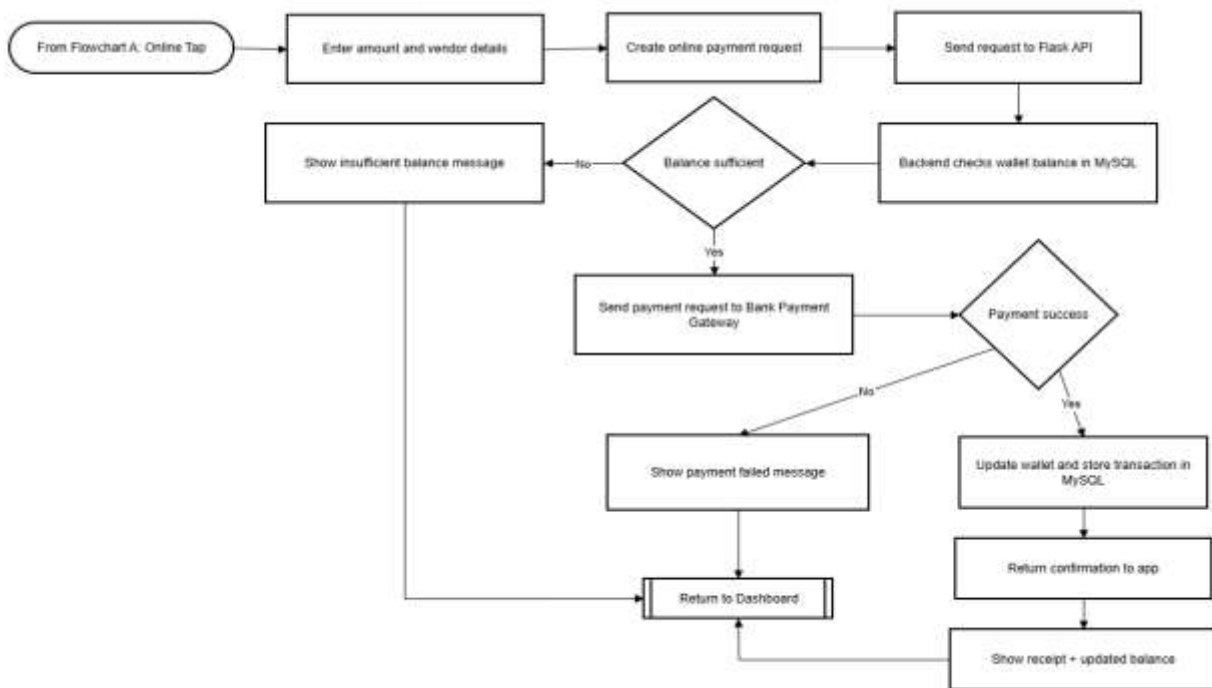


Figure 17: Flowchart B (Online Process)

- **Flowchart C:** It shows the offline Bluetooth tap and pay process

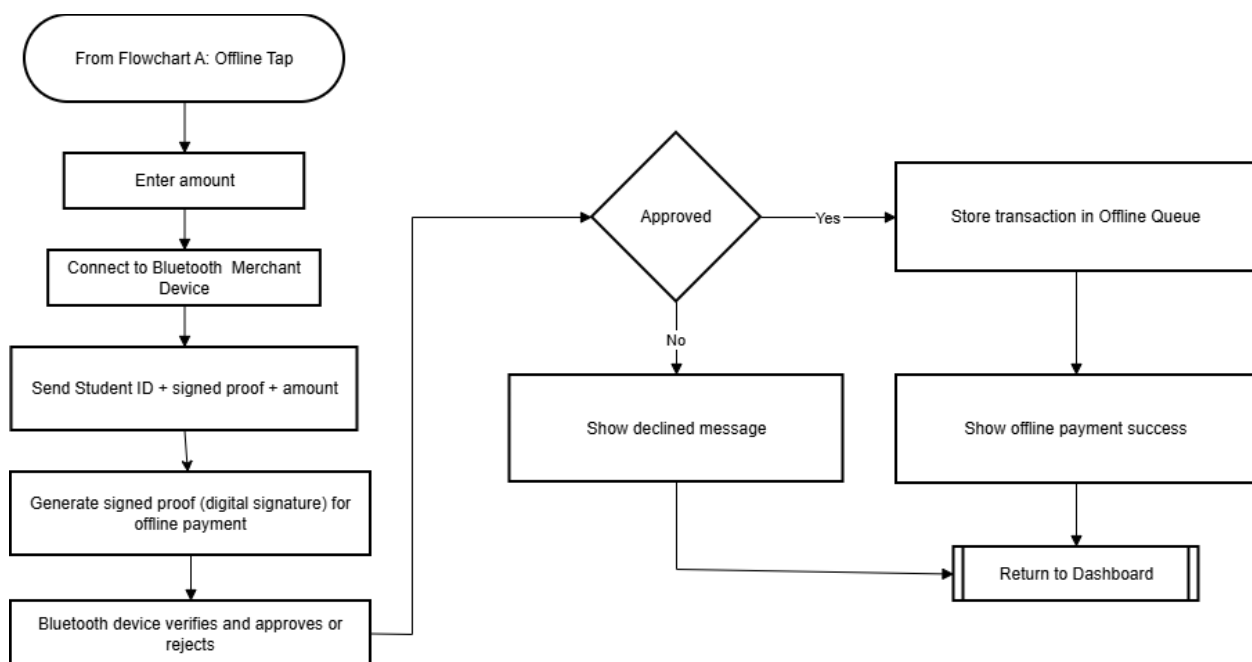


Figure 18: Flowchart C (Offline Bluetooth Tap)

- **Flowchart D:** It shows the offline sync process when internet becomes available.

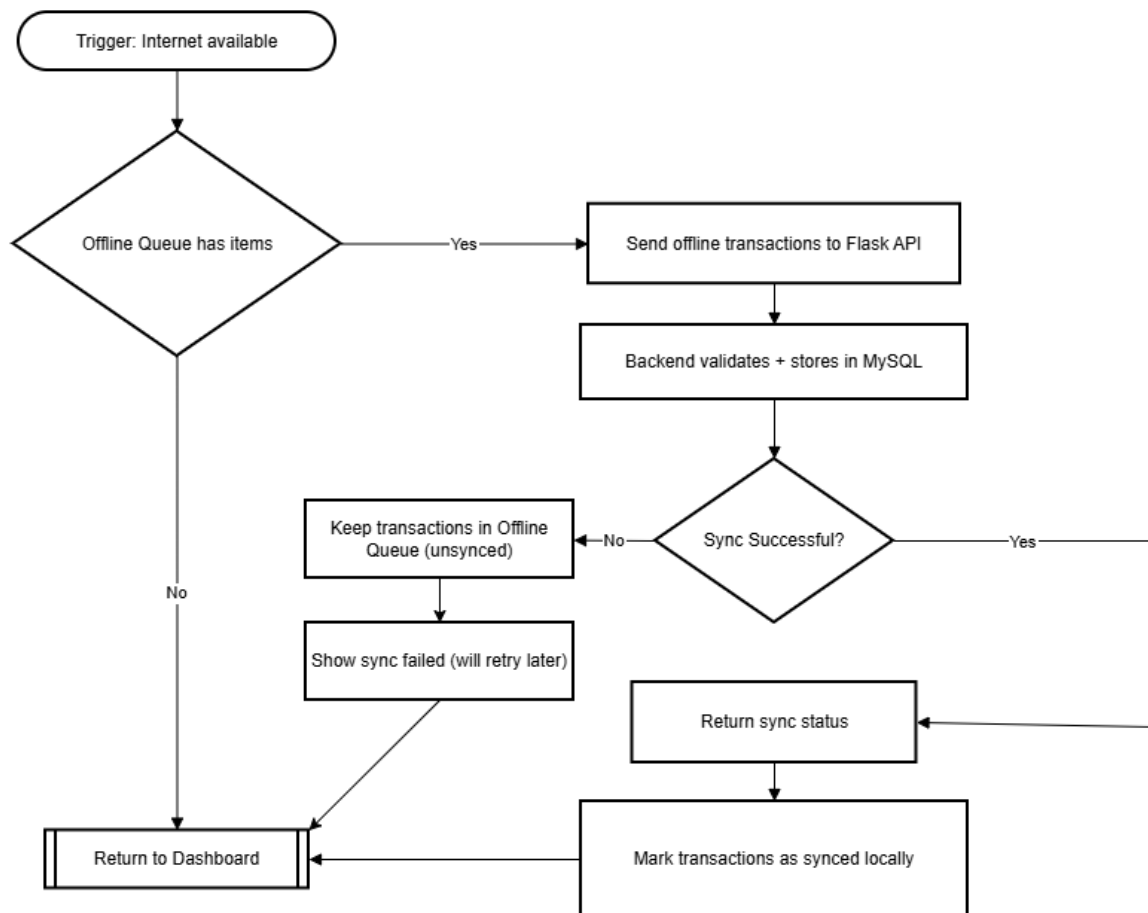


Figure 19: Flowchart D (Sync Process)

3.6.4. Data Flow Diagram

A Data Flow Diagram (DFD) shows the flow of data between the system and the external entities. Here, the DFD of the College Credential Digital Wallet System is describing data exchange for login, wallet info, online payments, offline payments, and syncing offline transactions between the system and the Student User, Vendor Merchant, College Admin, Bank/Payment Gateway, and Bluetooth Merchant Device.

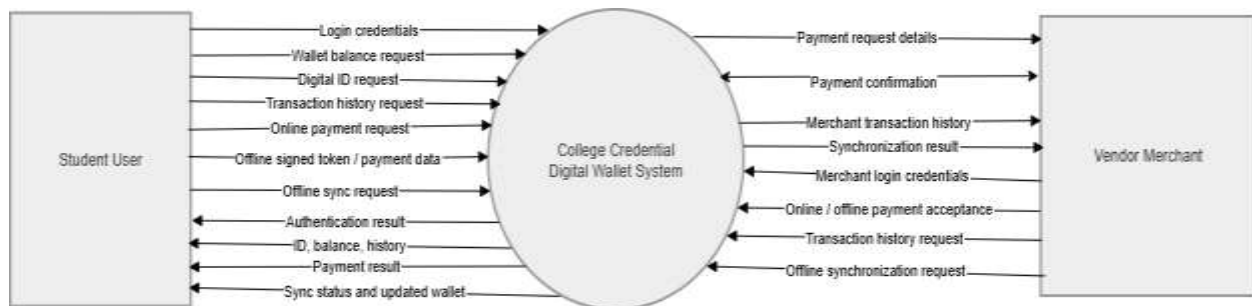


Figure 20: Data Flow Diagram (DFD0)

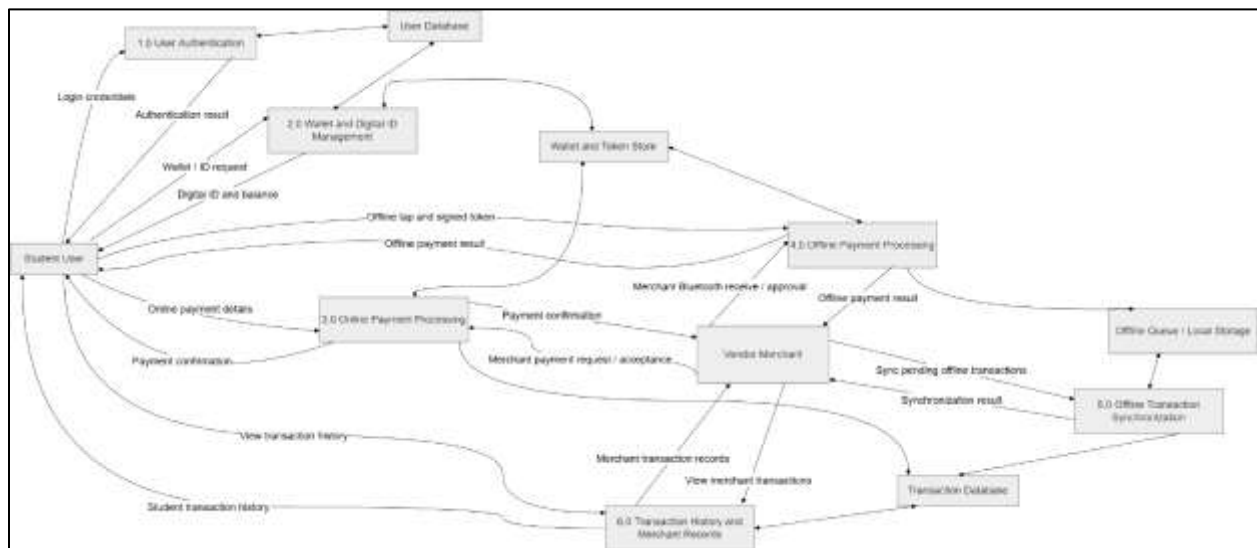


Figure 21: Data Flow Diagram (DFD1)

3.6.5. Entity Relationship Diagram (ERD)

Entity Relationship Diagram (ERD) is a graphical representation of the database design, including tables, keys, and relationships. In this case, the ERD is showing the connection of the USERS, WALLETS, TRANSACTIONS, OFFLINE_QUEUE, VENDORS, and ADMINS entities through PK/FK to keep records of user accounts, wallet balances, vendor data, payment records, and offline transactions that will be synced with the main transaction history later.

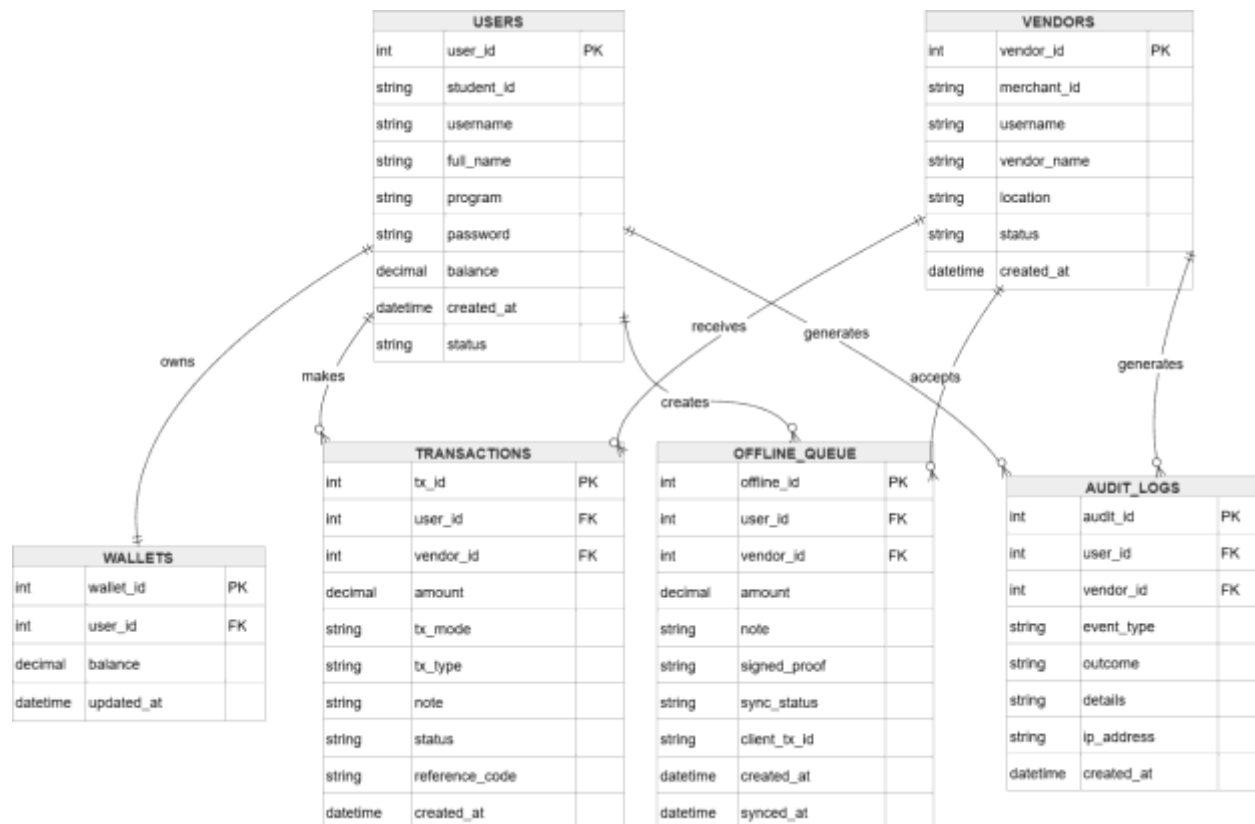


Figure 22: Entity Relationship Diagram (ERD)

3.6.6. Wireframes

A wireframe is a low-fidelity layout that shows what goes where on the screen (structure + navigation) without focusing on final colors/design. Here, the Login screen captures the basic authentication flow (logo + username/password + submit). The Dashboard screen shows the main student info (Digital ID, token balance, history) and clearly gives the next actions via Online Tap / Offline Tap, plus the popups show what fields + status messages the user will see during each payment mode.

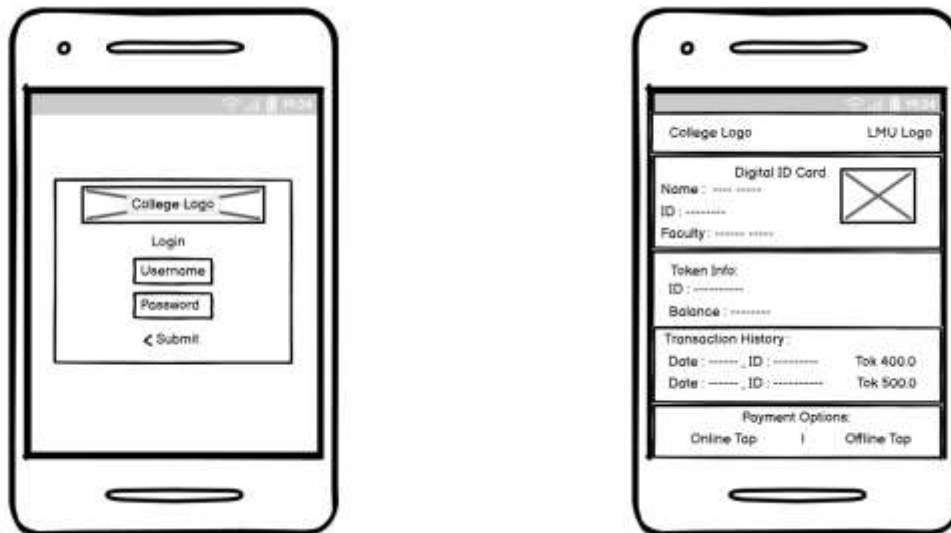


Figure 23: Wireframe 1 (Login and Dashboard)

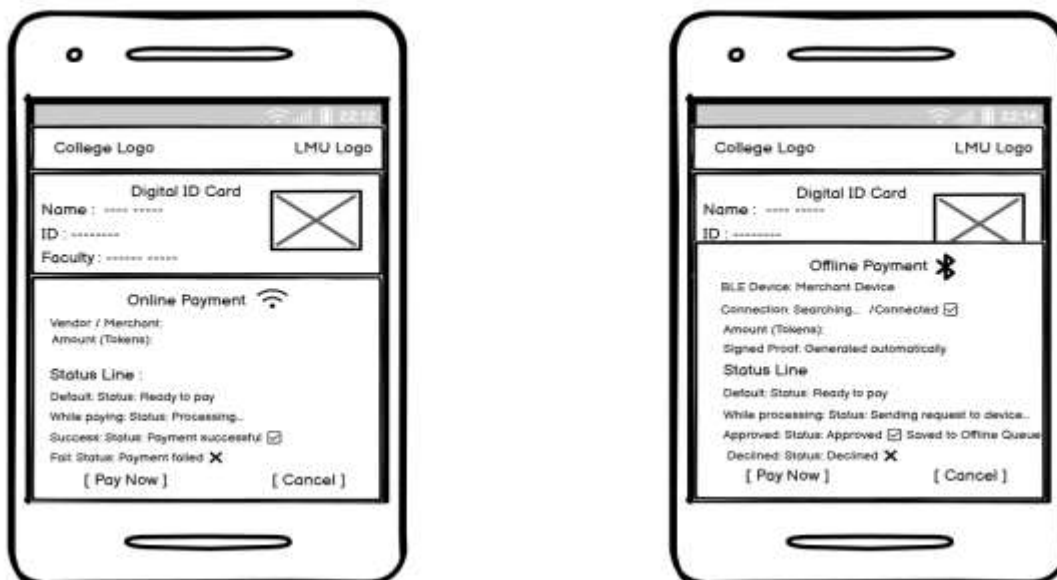


Figure 24: Wireframe 2 (Online and Offline Payment Popup)

3.6.7. UI Design

A UI design is essentially the first detailed visual representation of the wireframe that illustrates the brand, the spacing, the typography, and the colours of the final product. In this case, the UI design uses the project branding and demonstrates a refined Login screen and Wallet/Dashboard screen with the digital ID card, the balance, the transaction list, and an intuitive navigation to online/offline payment. This is a way of confirming that the interface remains consistent, readable, and user, friendly even without its complete realization.



Figure 25: UI Design 1 (Login Page)



Figure 26: UI Design 2 (Dashboard)

3.7. Implementation

This chapter explains how the CollegeWallet system is implemented in practice. The developed system operated as a hybrid system comprising an Android student application, a Flask-based backend server, a browser-based merchant portal, a central MySQL database and a local SQLite queue to store offline transactions. The implementation was aimed at facilitating the online and offline payment processes and maintaining the system as simple as possible to test and demonstrate within a final year project environment. The interface-level evidence is displayed in the chapter of testing, whereas the chosen code-level evidence is shown with the help of significant screenshots and other materials in the appendix. The implementation was based on practical functionality, web-side and back-end security checks, and testability.

3.7.1. Android Application Implementation

Android application is developed in Kotlin and Jetpack Compose. It serves as the primary interface for the student user, and it offers the fundamental functionality of the system i.e. login, access to dashboard, display of wallet balance, online payment, offline payment, transaction history and logout. The user initially authenticates with valid credentials, and the application gets the user profile and balance on the backend. This enables the wallet interface to show live information straight out of the server, and it will be able to make payment decisions based on the current account state. The Android implementation was simplified and practicalized with a flow such that the stages could be individually tested during the development and once again in the testing chapter. The UI design was minimized to be clean and task oriented to minimize user confusion in payment actions.

Further Android implementation details are provided in [Appendix 9.3.1](#).

Important Screenshots:

```

fun LoginScreen(navController: NavController) {
    } {
        } {
            onClick = {
                val request = LoginRequest(username = u, password = p)
                RetrofitClient.api.login(request).enqueue(callback = object : Callback<LoginResponse> {
                    override fun onResponse(
                        call: Call<LoginResponse>,
                        response: Response<LoginResponse>
                    ) {
                        loading = false
                        val body = response.body()

                        if (response.isSuccessful && body?.status == "success") {
                            // Save session (keep field names consistent with backend response)
                            UserSession.username = body.username
                            UserSession.studentId = body.studentId
                            UserSession.program = body.program
                            UserSession.balance = body.balance
                            UserSession.fullName = body.fullName

                            Toast.makeText(context, "Login successful 🟢", Toast.LENGTH_SHORT).show()

                            // Navigate cleanly (prevent a weird back behavior)
                            navController.navigate(route = "wallet") {
                                replace(route = "login") { includeIndex = true }
                            }
                        } else {
                            Toast.makeText(
                                context,
                                "Invalid username or password",
                                Toast.LENGTH_LONG
                            ).show()
                        }
                    }
                })
            }
        }
    }
}

```

Figure 27: Login API call and session creation

```

fun LoginScreen(navController: NavController) {
    } {
        } {
            onClick = {
                class {
                    Toast.makeText(
                        context,
                        "Invalid username or password",
                        Toast.LENGTH_LONG
                    ).show()
                }

                override fun onFailure(call: Call<LoginResponse>, t: Throwable) {
                    loading = false
                    val msg = t.message ?: t.javaClass.simpleName
                    Toast.makeText(context, "Login error: $msg", Toast.LENGTH_LONG).show()
                    android.util.Log.e(tag, "LOGIN_API", msg = "Login failed", e = t)
                }
            }
        }
    }

    shape = RoundedCornerShape(size = 10.dp),
    colors = ButtonDefaults.buttonColors(
        backgroundColor = Color(0xFFD0D0D0),
        textColor = Color.Black
    ),
    modifier = Modifier
        .width(width = 180.dp)
        .height(height = 48.dp)
}

```

Figure 28: Login API call and session creation (continued)

Here, the Android LoginScreen() calls the backend /login API and processes the response inside onResponse(). If authentication is successful, the returned user data is stored locally and the user is redirected to the wallet interface.

```

fun OnlinePaymentScreen() {
    fun fetchMerchants() {
        RetrofitClient.api.getMerchants(networkId).enqueue(callback = object : Callback<MerchantListResponse> {
            override fun onResponse(
                call: Call<MerchantListResponse>,
                response: Response<MerchantListResponse>
            ) {
                discoveryLoading = false
                val body = response.body()

                if (response.isSuccessful && body?.status == "success") {
                    merchants = body.merchants
                    applySelectionRules(!!body.merchants)

                    discoveryMessage = if (merchants.isEmpty()) {
                        "No merchants found on the given Wi-Fi (networkId=$networkId)."
                    } else null
                } else {
                    merchants = emptyList()
                    selectedMerchant = ""
                    discoveryMessage = body?.message ?: "Failed to load merchants [HTTP ${response.code()}]"
                }
            }
        })

        override fun onFailure(call: Call<MerchantListResponse>, t: Throwable) {
            discoveryLoading = false
            merchants = emptyList()
            selectedMerchant = ""
            discoveryMessage = "Backend not reachable. Check Flask IP + Wi-Fi."
        }
    }
}

```

Figure 29: Online merchant discovery and automatic selection

```

fun OnlinePaymentScreen() {
    ...
    Column(modifier = Modifier.padding(16.dp)) {
        ...
        paying = true

        val issueReq = OnlineTokenIssueRequest(
            sender_username = sender,
            merchant_username = selectedMerchant,
            amount = amt,
            note = note.ifBlank { null }
        )

        RetrofitClient.api.issueOnlineToken(issueReq)
            .enqueue(callback = object : Callback<OnlineTokenIssueResponse> {
                override fun onResponse(
                    call: Call<OnlineTokenIssueResponse>,
                    response: Response<OnlineTokenIssueResponse>
                ) {
                    val issueBody = response.body()

                    if (!response.isSuccessful && issueBody?.status == "success") {
                        paying = false
                        Toast.makeText(
                            context,
                            issueBody?.message ?: "Token issue failed",
                            duration = Toast.LENGTH_SHORT
                        ).show()
                        return
                    }
                }
            })
    }
}

```

Figure 30: Online tokenized payment flow

Here, the Android `OnlinePaymentScreen()` uses `fetchMerchants()` to call the `/merchants` endpoint and retrieve active merchants. The helper function `applySelectionRules()` then automatically selects the merchant when only one valid merchant is available.

```

fun OnlinePaymentScreen() {
    Column(modifier = Modifier.padding(8.dp)) {
        onClick = {
            val tokenId = issueBody.token_id
            val nonce = issueBody.nonce
            val expiresAt = issueBody.expires_at
            val signature = issueBody.signature

            if (tokenId.isNullOrEmpty() || nonce.isNullOrEmpty() || expiresAt.isNullOrEmpty() || signature.isNullOrEmpty()) {
                paying = false
                Toast.makeText(context, text = "Invalid token response", duration = Toast.LENGTH_SHORT).show()
                return
            }

            val consumeReq = OnlineTokenConsumeRequest(
                token_id = tokenId,
                nonce = nonce,
                expires_at = expiresAt,
                signature = signature
            )

            RetrofitClient.api.payOnlineTokenized(consumeReq)
                .enqueue(
                    call = Call<OnlinePayResponse>(),
                    callback = Callback<OnlinePayResponse> {
                        override fun onResponse(
                            call: Call<OnlinePayResponse>,
                            response: Response<OnlinePayResponse>
                        ) {
                            paying = false
                            val body = response.body()
                        }
                    }
                )
        }
    }
}

```

Figure 31: Online tokenized payment flow (continued 1)

```

fun OnlinePaymentScreen() {
    Column(modifier = Modifier.padding(8.dp)) {
        onClick = {
            if (response.isSuccessful && body?.status == "success") {
                UserSession.balance = body?.sender_new_balance
                Toast.makeText(context, text = "Payment successful", duration = Toast.LENGTH_SHORT).show()
                onPaymentSuccess()
            } else {
                Toast.makeText(
                    context,
                    text = body?.message ?: "Payment failed",
                    duration = Toast.LENGTH_SHORT
                ).show()
            }
        }

        override fun onFailure(call: Call<OnlinePayResponse>, t: Throwable) {
            paying = false
            Toast.makeText(context, text = "Backend not reachable", duration = Toast.LENGTH_SHORT).show()
        }

        override fun onFailure(call: Call<OnlineTokenIssueResponse>, t: Throwable) {
            paying = false
            Toast.makeText(context, text = "Backend not reachable", duration = Toast.LENGTH_SHORT).show()
        }
    }
}

```

Figure 32: Online tokenized payment flow (continued 2)

Here, the online payment flow first calls `issueOnlineToken()` to request a short-lived signed token and then calls `payOnlineTokenized()` to complete the transaction. This two-step process is used to ensure that payment is performed through validated tokenized data rather than plain request submission.

```

fun OfflinePaymentScreen() {
    fun sendOfflinePayment() {
        val currentBalance = UserSession.balance ?: 0.0
        if (amt > currentBalance) {
            Toast.makeText(context, msg = "Insufficient Balance", duration = Toast.LENGTH_SHORT).show()
            return
        }

        val nextToken: OfflineTokenItem? = OfflineTokenStore.peekNextToken(context, username.trim())
        if (nextToken == null) {
            successText = null
            statusText = "No offline tokens available, go online once and tap Refresh Tokens."
            return
        }

        if (amt > nextToken.max_amount) {
            successText = null
            statusText = "Amount is higher than this offline token limit. Refresh tokens while online."
            return
        }

        if (BluetoothAdapter == null) {
            statusText = "Bluetooth not supported on this phone."
            successText = null
            return
        }

        if (!BluetoothAdapter.isEnabled) {
            statusText = "Bluetooth is OFF. Please turn Bluetooth ON first."
            successText = null
            return
        }
    }
}

```

Figure 33: Offline Bluetooth Classic payment flow

```

fun OfflinePaymentScreen() {
    fun sendOfflinePayment() {
        CoroutineScope(context = Dispatchers.IO).launch {
            val buffer = ByteArray(1024)
            val count = Input.read(E, buffer)
            val responseText = if (count > 0) {
                String(bytes = buffer, offset = 0, length = count, Charset.forName("UTF-8"))
            } else {
                ""
            }

            val success = responseText.contains("pk": true) || responseText.contains("pk": true)

            withContext(context = Dispatchers.Main) {
                paying = false

                if (success) {
                    OfflineTokenStore.consumeToken(context, username.trim(), tokenid = nextToken.token_id)
                    refreshOfflineTokenCount()

                    amountText = ""
                    noteText = ""
                    statusText = null
                    successText = null

                    Toast.makeText(context, msg = "Payment successful", duration = Toast.LENGTH_SHORT).show()
                    onQueued()
                } else {
                    successText = null
                    statusText = "Offline payment failed. Please try again."
                }
            }
        }
    }
}

```

Figure 34: Offline Bluetooth Classic payment flow (continued)

Here, the offline payment screen uses `verifyMerchant()` to confirm the merchant device and `sendOfflinePayment()` to send the payment through Bluetooth Classic. During this flow, `OfflineTokenStore.peekNextToken()` is used to read the next available offline token and `OfflineTokenStore.consumeToken()` is called after successful transfer.

3.7.2. Backend Implementation

The system was designed with Flask in Python as the backend, which acts as the main controller of the system. It manages authentication, online payments, offline token generation, transaction processing, merchant operations, audit logging, and debug/testing routes. The backend is linked to MySQL via connection pool that enables the Android app and merchant portal to make repeated requests that are managed in an organized manner. It also has session-based merchant access to web routes and utility functions to manage token data, audit records and transaction validation. One of the primary implementations of the project is the support of the online and offline payment models in the backend. The application startup also generates supporting tables like audit logs and token-storage details when it is necessary.

Further backend implementation details are provided in [Appendix 9.3.2](#)

Important Screenshots:

```
def login():
    if not username or not password:
        log_audit_event(
            event_type="student_login",
            outcome="FAILED",
            actor_username=username,
            details={"reason": "username and password required", "ip": request.remote_addr}
        )
        return jsonify({"status": "failure", "message": "username and password required"}), 400

    conn = get_conn()
    cur = conn.cursor(dictionary=True)

    try:
        cur.execute("SELECT * FROM users WHERE username=%s", (username,))
        user = cur.fetchone()

        if not user:
            log_audit_event(
                event_type="student_login",
                outcome="FAILED",
                actor_username=username,
                details={"reason": "Invalid username or password", "ip": request.remote_addr}
            )
            return jsonify({"status": "failure", "message": "Invalid username or password"}), 401

        if not verify_password_and_upgrade(user, password):
            log_audit_event(
                event_type="student_login",
                outcome="FAILED",
                actor_username=username,
                details={"reason": "Invalid username or password", "ip": request.remote_addr}
            )
            return jsonify({"status": "failure", "message": "Invalid username or password"}), 401
```

Figure 35: Student Login Endpoint

```

def login():
    if not verify_password_and_upgrade(user, password):
        log_audit_event(
            event_type="student_login",
            outcome="FAILED",
            actor_username=username,
            details={"reason": "Invalid username or password", "ip": request.remote_addr}
        )
        return jsonify({"status": "failure", "message": "Invalid username or password"}), 401

    log_audit_event(
        event_type="student_login",
        outcome="SUCCESS",
        actor_username=username,
        details={"ip": request.remote_addr}
    )

    return jsonify({
        "status": "success",
        "message": "Login successful",
        "username": user.get("username"),
        "full_name": user.get("full_name"),
        "studentid": user.get("student_id"),
        "program": user.get("program"),
        "balance": float(user.get("balance") or 0.0),
    }), 200

except Exception as e:
    print("ERROR in /login:", repr(e))
    log_audit_event(
        event_type="student_login",
        outcome="FAILED",
        actor_username=username,
        details={"reason": f"Server error: {str(e)}", "ip": request.remote_addr}
    )
    return jsonify({"status": "failure", "message": f"Server error: {str(e)}"}), 500

```

Figure 36: Student Login Endpoint (continued)

Here, the Flask backend `login()` function handles the `/login` route and verifies the credentials. Password validation is performed through `verify_password_and_upgrade()`, after which the authenticated user details are returned to the Android application.

```

@app.route("/token/issue/online", methods=["POST"])
def issue_online_token():
    issue_online_token_body()

    data = request.json or {}
    sender_username = (data.get("sender_username") or "").strip()
    merchant_username = (data.get("merchant_username") or data.get("receiver_username") or "").strip()
    amount = data.get("amount")
    ip = data.get("ip")

    if not sender_username or not merchant_username:
        log_mail_event(
            event_type="online_token_issue",
            outcome="FAILURE",
            sender_username=sender_username,
            receiver_username=merchant_username,
            details="reason: 'sender_username and merchant_username required', 'ip': request.remote_addr"
        )
        return jsonify({"status": "Failure", "message": "sender_username and merchant_username required"}), 400

    if sender_username == merchant_username:
        log_mail_event(
            event_type="online_token_issue",
            outcome="FAILURE",
            sender_username=sender_username,
            receiver_username=merchant_username,
            details="reason: 'sender and merchant cannot be the same', 'ip': request.remote_addr"
        )
        return jsonify({"status": "Failure", "message": "sender and merchant cannot be the same"}), 400

    try:
        amount = float(amount)
        amount = (type(amount), ValueError)
        log_mail_event(
            event_type="online_token_issue",
            outcome="FAILURE",
            sender_username=sender_username,
            receiver_username=merchant_username,
            details="reason: 'amount must be a number', 'ip': request.remote_addr"
        )
        return jsonify({"status": "Failure", "message": "amount must be a number"}), 400

    if amount <= 0:
        log_mail_event(
            event_type="online_token_issue",
            outcome="FAILURE",
            sender_username=sender_username,
            receiver_username=merchant_username,
            details="reason: 'amount must be > 0', 'ip': request.remote_addr"
        )
        return jsonify({"status": "Failure", "message": "amount must be > 0"}), 400

    conn = get_conn()
    cur = conn.cursor(dictionary=True)

```

Figure 37: Online token issue endpoint

```

def issue_online_token():
    log_audit_event(
        event_type="online_token_issue",
        outcome="SUCCESS",
        sender_username=sender_username,
        receiver_username=merchant_username,
        merchant_username=merchant_username,
        amount=amount,
        client_tx_id=token_id,
        details={
            "expires_at": expires_at_str,
            "ttl_seconds": ONLINE_TOKEN_TTL_SECONDS,
            "ip": request.remote_addr
        }
    )

    return jsonify({
        "status": "success",
        "message": "Online payment token issued",
        "token_id": token_id,
        "nonce": nonce,
        "expires_at": expires_at_str,
        "signature": signature
    }), 200

except Exception as e:
    print("ERROR in /token/issue/online:", repr(e))
    try:
        conn.rollback()
    except Exception:
        pass

    log_audit_event(
        event_type="online_token_issue",
        outcome="FAILED",
        sender_username=sender_username,
        receiver_username=merchant_username,
        amount=amount if isinstance(amount, (int, float)) else None,
        details={"reason": f"Server error: {str(e)}", "ip": request.remote_addr}
    )

    return jsonify({"status": "failure", "message": f"Server error: {str(e)}"}), 500

finally:
    cur.close()
    conn.close()

```

Figure 38: Online token issue endpoint (continued)

Here, the backend `issue_online_token()` function handles the `/token/issue/online` route and generates a signed online payment token. The token signature is created using `build_online_token_signature()`, and the issued token is stored with expiry and usage status for later validation.

```

def pay_online_tokenized():
    if status == "USED":
        conn.rollback()
        log_audit_event(
            event_type="online_token_consume",
            outcome="FAILED",
            sender_username=sender_username,
            receiver_username=merchant_username,
            merchant_username=merchant_username,
            amount=amount,
            client_tx_id=token_id,
            details={"reason": "Token already used", "transaction_id": token.get("transaction_id"), "ip": request.remote_addr}
        )
        return jsonify({"status": "failure", "message": "Token already used"}), 409

    if status == "EXPIRED":
        conn.rollback()
        log_audit_event(
            event_type="online_token_consume",
            outcome="FAILED",
            sender_username=sender_username,
            receiver_username=merchant_username,
            merchant_username=merchant_username,
            amount=amount,
            client_tx_id=token_id,
            details={"reason": "Token already expired", "ip": request.remote_addr}
        )
        return jsonify({"status": "failure", "message": "Token expired"}), 400

```

Figure 39: Tokenized online payment consumption

```

def pay_online_tokenized():
    if status == "USED":
        conn.rollback()
        log_audit_event(
            event_type="online_token_consume",
            outcome="FAILED",
            sender_username=sender_username,
            receiver_username=merchant_username,
            merchant_username=merchant_username,
            amount=amount,
            client_tx_id=token_id,
            details={"reason": "Token already used", "transaction_id": token.get("transaction_id"), "ip": request.remote_addr}
        )
        return jsonify({"status": "failure", "message": "Token already used"}), 409

    if status == "EXPIRED":
        conn.rollback()
        log_audit_event(
            event_type="online_token_consume",
            outcome="FAILED",
            sender_username=sender_username,
            receiver_username=merchant_username,
            merchant_username=merchant_username,
            amount=amount,
            client_tx_id=token_id,
            details={"reason": "Token already expired", "ip": request.remote_addr}
        )
        return jsonify({"status": "failure", "message": "Token expired"}), 400

```

Figure 40: Tokenized Online Payment Consumption (continued)

Here, the backend `pay_online_tokenized()` function is called to consume the previously issued online token and complete the payment. In this step, the backend verifies token status, checks replay and expiry conditions, updates balances, and records the final transaction in the database.

```
def issue_offline_token_pack():
    log_audit_event(
        event_type="offline_token_pack_issue",
        outcome="SUCCESS",
        actor_username=sender_username,
        sender_username=sender_username,
        details={
            "pack_size": pack_size,
            "ttl_hours": OFFLINE_TOKEN_TTL_HOURS,
            "max_amount_per_token": round(sender_balance, 2),
            "ip": request.remote_addr
        }
    )

    return jsonify({
        "status": "success",
        "message": "Offline token pack issued",
        "sender_username": sender_username,
        "pack_size": pack_size,
        "ttl_hours": OFFLINE_TOKEN_TTL_HOURS,
        "tokens": tokens
    }), 200

except Exception as e:
    print("ERROR in /token/issue/offline-pack:", repr(e))
    try:
        conn.rollback()
    except Exception:
        pass

    log_audit_event(
        event_type="offline_token_pack_issue",
        outcome="FAILED",
        actor_username=sender_username,
        sender_username=sender_username,
        details={"reason": f"Server error: {str(e)}", "ip": request.remote_addr}
    )
```

Figure 41: Offline token pack succeed

Here, the backend `issue_offline_token_pack()` function generates a pack of offline tokens for disconnected payment use. The token signature is created using `build_offline_token_signature()`, and the token pack is returned to the Android application for later offline payment processing.

```

@app.route("/merchants", methods=["GET"])
def get_merchants():
    network_id = request.args.get("networkId")
    if not network_id:
        return jsonify({"status": "failure", "message": "networkid query param required"}), 400

    conn = get_conn()
    cur = conn.cursor(dictionary=True)

    try:
        cur.execute(
            """
            SELECT u.username
            FROM users u
            JOIN merchant_presence p ON p.merchant_username = u.username
            WHERE u.role='merchant'
            AND p.network_id=Is
            AND p.last_seen >> (NOW() - INTERVAL {MERCHANT_TTL_SECONDS} SECOND)
            ORDER BY u.username
            """
            , (network_id,)
        )
        rows = cur.fetchall() or []
        merchants = [r["username"] for r in rows]

        return jsonify({
            "status": "success",
            "networkId": network_id,
            "ttlSeconds": MERCHANT_TTL_SECONDS,
            "merchants": merchants
        }), 200
    except Exception as e:
        print("ERROR in /merchants:", repr(e))
        return jsonify({"status": "failure", "message": f"server error: {str(e)}"}), 500

```

Figure 42: Merchant Discovery Endpoint

Here, the backend `get_merchants()` function handles the `/merchants` route and returns currently active merchants. This allows the Android online payment screen to display nearby available merchants before the student initiates payment.

3.7.3. Merchant Web portal

The merchant side was developed as a web portal that was a browser based on the same Flask backend. This portal will enable merchants to log in, accept or cancel payments, see the history of transactions, consider outstanding offline transactions, perform synchronization, and look at analytics. The merchant portal is significant as it offers operational control without the need to have a second mobile application. This simplified development also enabled the project to be more easily demonstrated in a final year project setting, as many actions could be initiated directly on the backend or browser side. The access of merchant web is secured using the session-based login checks and merchant specific actions are limited within the logged-in merchant account.

Further merchant-side implementation details are provided in [Appendix 9.3.3](#).

Important Screenshots:

```
@app.route("/merchant/merchantlogin", methods=["GET", "POST"])
def merchant_login():
    ensure_password_hash_column()
    backfill_password_hashes()

    if request.method == "GET":
        return render_template_string(MERCHANT_LOGIN_HTML, error=None)

    username = (request.form.get("username") or "").strip()
    password = request.form.get("password") or ""

    if not username or not password:
        log_audit_event(
            event_type="merchant_login",
            outcome="FAILED",
            actor_username=username,
            merchant_username=username,
            details=["reason": "Username and password required", "ip": request.remote_addr]
        )
        return render_template_string(MERCHANT_LOGIN_HTML, error="Username and password required")

    conn = get_conn()
    cur = conn.cursor(dictionary=True)
    try:
        cur.execute(
            "SELECT id, username, role, password, password_hash FROM users WHERE username=%s",
            (username,)
        )
        u = cur.fetchone()
```

Figure 43: Merchant Login route

```
def merchant_login():
    if not verify_password_and_upgrade(u, password):
        log_audit_event(
            event_type="merchant_login",
            outcome="FAILED",
            actor_username=username,
            merchant_username=username,
            details={"reason": "Invalid credentials", "ip": request.remote_addr}
        )
        return render_template_string(MERCHANT_LOGIN_HTML, error="Invalid credentials")

    if u.get("role") != "merchant":
        log_audit_event(
            event_type="merchant_login",
            outcome="FAILED",
            actor_username=username,
            merchant_username=username,
            details={"reason": "This account is not a merchant", "ip": request.remote_addr}
        )
        return render_template_string(MERCHANT_LOGIN_HTML, error="This account is not a merchant")

    session["merchant_username"] = u["username"]

    log_audit_event(
        event_type="merchant_login",
        outcome="SUCCESS",
        actor_username=u["username"],
        merchant_username=u["username"],
        details={"ip": request.remote_addr}
    )

    return redirect(f"/merchant/{u['username']}?toast=login_success")

finally:
    cur.close()
    conn.close()
```

Figure 44: Merchant Login route (continued)

Here, the merchant web portal uses the `merchant_login()` function to validate merchant credentials and create the web session. After successful login, the merchant is redirected to the protected merchant dashboard.

```

def merchant_dashboard(merchant_username):
    logged_in = session.get("merchant_username")
    if not logged_in:
        return redirect("/merchant/merchantlogin")

    if logged_in.lower() != merchant_username.lower():
        return redirect(f"/merchant/{logged_in}")

    conn = get_conn()
    cur = conn.cursor(dictionary=True)

    try:
        cur.execute(
            "SELECT id, username, full_name, student_id, balance, role FROM users WHERE username=%s",
            (logged_in,)
        )
        merchant = cur.fetchone()
        if not merchant or merchant.get("role") != "merchant":
            session.pop("merchant_username", None)
            return redirect("/merchant/merchantlogin")

        merchant_id = merchant["id"]
        merchant["balance"] = float(merchant.get("balance") or 0.0)

        remote_ip = request.remote_addr or ""
        network_id = ip_to_network_id(remote_ip)

        cur.execute("SELECT last_seen FROM merchant_presence WHERE merchant_username=%s", (logged_in,))
        pres = cur.fetchone()
        is_active = pres is not None

        cur.execute(
            """
            SELECT
                t.id,
                t.amount,
                t.method,
                t.status,
                t.note,
                t.created_at,
                u1.username AS sender_username,
                u2.username AS receiver_username
            FROM transactions t
            JOIN users u1 ON u1.id = t.sender_user_id
            JOIN users u2 ON u2.id = t.receiver_user_id
            WHERE t.sender_user_id=%s OR t.receiver_user_id=%s
            ORDER BY t.created_at DESC
            """,
            (merchant_id, merchant_id)
        )
        txs = cur.fetchall() or []
        for t in txs:
            t["amount"] = float(t["amount"])

        cur.execute(

```

Figure 45: Merchant Dashboard Route

```

cur.execute(
    """
    SELECT
        COALESCE(SUM(t.amount), 0) AS total_30,
        COALESCE(AVG(t.amount), 0) AS avg_tx,
        COALESCE(COUNT(*), 0) AS cnt_tx
    FROM transactions t
    WHERE t.receiver_user_id=%s
        AND t.status='SUCCESS'
        AND t.created_at >= (NOW() - INTERVAL 30 DAY)
    """,
    (merchant_id,)
)
m = cur.fetchone() or {}
total_30 = float(m.get("total_30") or 0.0)
avg_tx = float(m.get("avg_tx") or 0.0)
cnt_tx = float(m.get("cnt_tx") or 0.0)
daily_avg = cnt_tx / 30.0

metrics = {
    "total_30": total_30,
    "avg_tx": avg_tx,
    "daily_avg": daily_avg
}

return render_template_string(
    MERCHANT_DASHBOARD_HTML,
    merchant=merchant,
    network_id=network_id,
    is_active=is_active,
    txs=txs,
    metrics=metrics
)

finally:
    cur.close()
    conn.close()

```

Figure 46: Merchant Dashboard Route (continued)

Here, `merchant_dashboard(merchant_username)` loads the merchant dashboard, which retrieves merchant-specific payment records and summary data. This route is used to display transaction information, control merchant operations, and support analytics on the web portal.

```

@app.route("/offline/queue/pending", methods=["GET"])
@merchant_login_required
def offline_queue_pending():
    merchant_username = request.args.get("merchant_username")
    if not merchant_username:
        return jsonify({"status": "failure", "message": "merchant_username required"}), 400

    if (session.get("merchant_username") or "").lower() != merchant_username.lower():
        return jsonify({"status": "failure", "message": "Unauthorized merchant queue access"}), 403

    offline_init_db()
    conn = offline_db()
    cur = conn.cursor()
    cur.execute("""
        SELECT client_tx_id, sender_username, amount, note, status, last_error, created_at
        FROM offline_payments
        WHERE merchant_username=? AND status='PENDING'
        ORDER BY id ASC
        """, (merchant_username,))
    rows = [dict(r) for r in cur.fetchall()]
    conn.close()

    return jsonify({"status": "success", "merchant_username": merchant_username, "pending": rows}), 200

```

Figure 47: Offline pending route

Here, the merchant-side pending queue is retrieved using the `offline_queue_pending()` route. This is used to display offline transactions that were received earlier but have not yet been synchronized with the backend.

```

@app.route("/offline/queue/sync", methods=["POST"])
@merchant_login_required
def offline_queue_sync():
    data = request.json or {}
    merchant_username = data.get("merchant_username")
    if not merchant_username:
        return jsonify({"status": "Failure", "message": "merchant_username required"}), 400

    if (session.get("merchant_username") or "").lower() != merchant_username.lower():
        return jsonify({"status": "Failure", "message": "Unauthorized merchant queue sync"}), 403

    offline_init_db()

    conn = offline_db()
    cur = conn.cursor()
    cur.execute("""
        SELECT client_tx_id, sender_username, amount, note,
               offline_token_id, offline_token_nonce, offline_token_expires_at, offline_token_signature
        FROM offline_payments
        WHERE merchant_username=? AND status='PENDING'
        ORDER BY id ASC
    """, (merchant_username,))
    pending = [dict(r) for r in cur.fetchall()]
    conn.close()

    if not pending:
        log_audit_event(
            event_type="offline_queue_sync",
            outcome="SUCCESS",
            actor_username=merchant_username,
            merchant_username=merchant_username,
            details={"message": "No pending offline payments", "ip": request.remote_addr}
        )
        return jsonify({"status": "success", "message": "No pending offline payments", "results": []}), 200

    sync_payload, sync_code = process_offline_sync(merchant_username, pending)

    if sync_code != 200:
        log_audit_event(
            event_type="offline_queue_sync",
            outcome="FAILED",
            actor_username=merchant_username,
            merchant_username=merchant_username,
            details={"message": sync_payload.get("message"), "ip": request.remote_addr}
        )
        return jsonify(sync_payload), sync_code

    results = sync_payload.get("results", [])

    conn2 = offline_db()
    cur2 = conn2.cursor()

```

Figure 48: Offline Queue sync route

```

def offline_queue_sync():
    for r in results:
        txid = r.get("client_tx_id")
        st = r.get("status")
        msg = r.get("message")

        if st in ("SUCCESS", "DUPLICATE"):
            cur2.execute(
                "UPDATE offline_payments SET status='SYNCED', last_error=NULL WHERE client_tx_id=?",
                (txid,)
            )
        else:
            cur2.execute(
                "UPDATE offline_payments SET status='FAILED', last_error=? WHERE client_tx_id=?",
                (msg or "FAILED", txid)
            )

    conn2.commit()
    conn2.close()

    success_count = len([r for r in results if r.get("status") in ("SUCCESS", "DUPLICATE")])
    failed_count = len([r for r in results if r.get("status") == "FAILED"])

    log_audit_event(
        event_type="offline_queue_sync",
        outcome="SUCCESS" if failed_count == 0 else "PARTIAL",
        actor_username=merchant_username,
        merchant_username=merchant_username,
        details={
            "success_count": success_count,
            "failed_count": failed_count,
            "results_count": len(results),
            "ip": request.remote_addr
        }
    )

    return jsonify({
        "status": "success",
        "merchant_username": merchant_username,
        "results": results
    }), 200

```

Figure 49: Offline Queue sync route (continued)

Here, the offline synchronization process is handled through `offline_queue_sync()` and the helper function `process_offline_sync(merchant_username, payments)`. This function validates pending offline transactions and moves valid records into the main transaction history after successful synchronization.

3.7.4. Database and Local storage Implementation

The project has two storage layers: MySQL to store central system data and SQLite to store merchant-side offline queuing. MySQL contains a table of users, balances, transactions, audit events, and online and offline token records of payment. This central database assists real time online operations and historical reporting via merchant portal. The system is appropriate to a campus wallet system since it isolates the account information, payment information and token information and yet can be connected during validation and reporting. SQLite on the other hand is utilized in the offline merchant queue so that offline transactions can still be recorded even in cases where the server is not available to synchronize the transactions. This is one of the key technical design decisions of the final implementation that divides central and local storage.

Further database and local storage details are provided in [Appendix 9.3.4](#)

Important Screenshots:

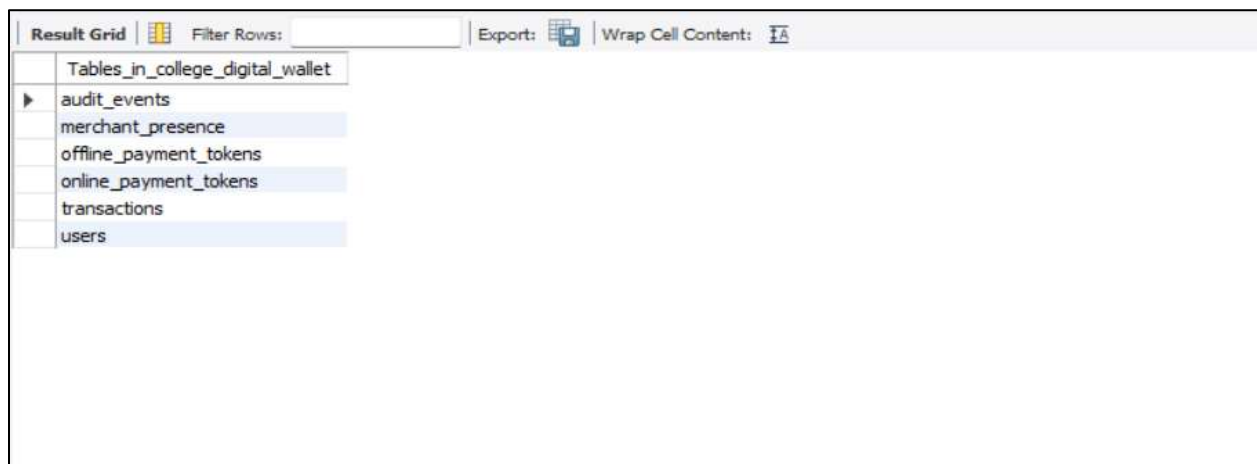


Figure 50: Tables in database

Result Grid										
Filter Rows:					Edit:		Export/Import:		Wrap Cell Content:	
	id	username	password	role	full_name	student_id	program	balance	created_at	password_hash
▶	1	Ayudh	ayudh123	student	Ayudh Dahal	NP01NTA4230180	85c (Hons) Networking & IT Security	471468.00	2026-02-18 02:07:07	script:32768:8:1\$QF9QjP
	2	Subhash	subhash123	student	Subhash Budha Magar	NP01NTA4230181	85c (Hons) Networking & IT Security	499700.00	2026-02-18 02:07:07	script:32768:8:1\$QWcd87
	3	Chetan	chetan123	student	Chetan Oli	NP01NTA4230182	85c (Hons) Networking & IT Security	5000.00	2026-02-18 02:07:07	script:32768:8:1\$plwst20
	4	Ayush	ayush123	student	Aayush Bista	NP01NTA4230183	85c (Hons) Networking & IT Security	5000.00	2026-02-18 02:07:07	script:32768:8:1\$uxAw00W
	5	Canteen	canteen123	merchant	College Canteen	M-0001	College Canteen	27212.00	2026-02-18 02:07:07	script:32768:8:1\$7tdt4uH
	10	coffee_station	coffee123	merchant	Coffee Station	M-0002	Coffee Station	20130.00	2026-03-10 15:04:07	script:32768:8:1\$DduA6pH
	11	stationary	stationary123	merchant	Stationary	M-0003	Stationary	20480.00	2026-03-10 15:04:07	script:32768:8:1\$lp7H3Cv
	13	Pawan	pawan123	student	Pawan Regami Magar	NP01NTA4230184	85c (Hons) Networking & IT Security	5000.00	2026-04-08 21:02:19	script:32768:8:1\$J7NKS9K

Figure 51: Users table

Result Grid									
Filter Rows:									
Edit:									
Export/Import:									
Wrap Cell Content:									
	id	sender_user_id	receiver_user_id	amount	method	status	note	created_at	client_tx_id
▶	1	1	5	60.00	ONLINE	SUCCESS	coffee	2026-02-18 02:20:53	NULL
	2	1	5	80.00	ONLINE	SUCCESS	drink	2026-02-18 03:03:49	NULL
	3	2	5	100.00	ONLINE	SUCCESS	coffee	2026-02-22 13:48:25	NULL
	4	1	5	100.00	ONLINE	SUCCESS	food	2026-02-22 14:04:43	NULL
	5	1	5	20.00	ONLINE	SUCCESS	food	2026-02-22 14:05:39	NULL
	6	1	5	21.00	ONLINE	SUCCESS	food	2026-03-08 13:53:54	NULL
	7	1	5	20.00	ONLINE	SUCCESS	food	2026-03-08 14:16:21	NULL
	8	1	5	20.00	ONLINE	SUCCESS	food	2026-03-09 21:35:37	NULL
	9	1	5	20.00	ONLINE	SUCCESS	coffee	2026-03-09 23:41:43	NULL
	10	1	5	20.00	ONLINE	SUCCESS	hello	2026-03-10 12:06:29	NULL
	11	1	5	20.00	ONLINE	SUCCESS	hello	2026-03-10 12:08:14	NULL
	12	1	5	19.09	ONLINE	SUCCESS	food	2026-03-10 12:25:44	NULL
	13	1	5	5.00	ONLINE	SUCCESS	food	2026-03-10 12:30:38	NULL
	14	1	5	6.00	ONLINE	SUCCESS	ice	2026-03-10 12:31:07	NULL
	15	1	5	10.00	ONLINE	SUCCESS	ice	2026-03-10 13:41:05	NULL

Figure 52: Transactions table

Result Grid										
Filter Rows:										
Edit:										
Export/Import:										
Wrap Cell Content:										
	id	token_id	sender_username	merchant_username	amount	note	nonce	expires_at	status	signature
▶	1	3d5c799bc24499bab2fe7096393fc3e	Ayudh	Canteen	30.00	Burger	9da9589e5134e08907526f8e8888e7	2026-04-02 17:16:07	USED	cd556239994fab99dcf11fec2ae085
	2	75aef4e31031a4e5a03d8c84b9f29c75d	Ayudh	Canteen	100.00	Pizza	24bc79eae0f4e482675d4484e0f99160	2026-04-02 18:36:39	USED	22d4108e4e3db2b7b3a948cfe888
	3	75d2168d25441386d0e096fcd0b0f	Ayudh	Canteen	100.00	Burger	f10d86040ed43b791c2cecd3e7ee76	2026-04-03 07:03:54	USED	1094b1a3897ed44c58a5dd45d3a25
	4	1159f3d90c5f62318523edf4c3b3e2	Ayudh	Canteen	25.00		94ae5f8d35d4488bfc2b18e8524b3d	2026-04-05 07:57:28	USED	02d2440ee3c184457d4d3299d300
	5	aa36639791b407ca55f17374ee4ef	Ayudh	Canteen	10.00	online expired token test	c53b7b955864b5fa709df878c130b	2026-04-08 13:56:17	EXPIRED	8e178a6711488888bade48a8268e
	6	8933216b6d24f24a550c2b4b7b3d14	Ayudh	Canteen	60.00		27a5db73179049005aadb3025092528	2026-04-08 15:28:16	USED	809a828d0088f8bae7a703842001a
	7	2e8c28ec542b4c12b8be2d712edff7ae	Ayudh	Canteen	100.00	Khaja	a2ba8e123c7344f6d1587ef969e9a37	2026-04-11 10:38:30	USED	af6c7b11c7959d3138b3b832e96713f
	8	9e6d299624b405a35f2812333aee9f	Ayudh	Canteen	20.00	Ju	cdceea5a26a542b48815f4d488a37140	2026-04-11 11:17:52	USED	3668866d432a4c3c86a21b08a0227
	9	8de6907237c1433ca4dbdc179e13df74	Ayudh	Canteen	100.00	Khaja	0a0210295647a8d2d765258b1625d	2026-04-11 11:25:17	USED	46c237ee1106a3a920efbbe1c3c
	10	6e16084e74c249388b6e12f3a088268c	Ayudh	Canteen	40.00	Heflo	f2b5c11f62f47b3986337f417b426a9	2026-04-11 11:31:10	USED	184b9f79328e6c14e05e49061a499
	11	889e79d3c24e58a1d77154a37baf21e	Ayudh	Canteen	20.00		9eb9c1ca0f94805a1181674ef934289	2026-04-11 11:36:38	USED	478c1a40c4c6d430458e74350999
	12	4cf214601c0840ee968811460c588415	Ayudh	Canteen	20.00		5912ba1942be425fa3d9c3445828766	2026-04-11 12:55:55	USED	3b09e632a29f58c9f80e9a2ed7b40
	13	23ee8b392624c1395e48ec60331d56	Ayudh	Canteen	100.00	Khaja	1dcead88832a4b779094c3214e42430	2026-04-11 13:02:38	USED	f9449e9a05c3d59223ee1a6dfe115
	14	8729e5db44839ea380e4c08302	Ayudh	Canteen	20.00		30f9d06148b4e488343380c0e0238c	2026-04-11 13:03:53	USED	647b38af7b3f8e5248f554ef25403

Figure 53: Online payment tokens table

Result Grid										Filter Rows:	Edit:	Export/Import:	Wrap Cell Content:
#	id	token_id	sender_username	nonce	max_amount	expires_at	status	signature	used_merchant_username				
1	1	3c732e230a9418ab9a9b13340b62d1	Ayudh	2a83e6d7091e414e5050d0bee91722b	478233.00	2026-04-06 11:17:38	USED	0b63bee88d7c03e50d3b17fe0e7154618a70c1...	Canteen				
2	2	526e2b779d341718f4b71uch77f78d1	Ayudh	d6037f0d46da41139f014b1674b15059	478233.00	2026-04-06 11:17:38	USED	129d028ffcd7f1ead8bba3c097e9a0a210524...	Canteen				
3	3	3d55721b86d447f8a4769657a71ac3f	Ayudh	28d7d2dc8c2849e6a8e4007269a775a8	478233.00	2026-04-06 11:17:38	USED	7ac778819b7bc603b8195ab55c7954f7e40e45...					
4	4	7323e545674e475d89e58dddb9fcd3	Ayudh	9996c12840e946f89f80abf58018e27	478233.00	2026-04-06 11:17:38	USED	a0f72741c04e3bcd1258306a4e9fc12f5511a20...					
5	5	8b4d3f5e6b84a05a239ddcd6512a	Ayudh	996c3d1b25d427bd765c58139d73ef1	478233.00	2026-04-06 11:17:38	USED	95b1d43d0d147f89bd1aac57aea0c28225d7a02...					
6	6	13066c796d8943be8213d222d6c09f2	Ayudh	d589e107b5b74b46a1b484170b4be918	478233.00	2026-04-06 11:18:12	USED	1a91d41b095d056e644733cd79d7e26cd2...					
7	7	2bec94e77404b1388a732ff13c696a	Ayudh	f9edcfba7544239a2ae711b63540d30	478233.00	2026-04-06 11:18:12	USED	c57a76c44736f2b4030222c898dcf4aeb38...					
8	8	4ba1acc0f50b4f08af5c5e6bd95b3f	Ayudh	2b2d3ef9ba514c5ea53a3883867754b0	478233.00	2026-04-06 11:18:12	USED	fa9ff2e723d88b6b4ef59285c83fe42f2afbb223...					
9	9	77e39938e15944c3b3b5e6b963c451	Ayudh	412d14f470f044d2af5c57288a9371e5	478233.00	2026-04-06 11:18:12	USED	a2f6d1e521ef9b0d0c94e446a93b07907a17...					
10	10	c3421d97876142d4e52d79f898904bf	Ayudh	cd53b6ae2ae5448f9d30c82d7057b1d4	478233.00	2026-04-06 11:18:12	USED	751e72c193a16d53ddfa3c37ae796dacf4c019f...					
11	11	840a58d837547988d9e4ef0af2dea4d	Ayudh	4a6d1b6f6ea223a7e3f13c928733d1	478213.00	2026-04-07 09:37:08	USED	72d957d384ee059f6bee1231233873186b5645...					
12	12	a53079e57ae546fca50f8bee3c817f9	Ayudh	cd8711c5eff2b46159e2f4e059f38f54	478213.00	2026-04-07 09:37:08	USED	6ee60f9ef5d63e662cd08ba17a5627f6d20617...					
13	13	41704a4e6d4407c993d999a9a2b53a	Ayudh	526b67ba04e4f27b3baeeea15d3be2c9	478213.00	2026-04-07 09:37:08	USED	82592a370d118c72b72996925904321842a63b...					
14	14	59844e88e0b4d11bd750c73dc12b6	Ayudh	758a0b359d014ff2add8600957b4effe	478213.00	2026-04-07 09:37:08	USED	60e914479cfdb7977f7f8979384f722254e58d...					

Figure 54: Offline payment tokens table

Result Grid												Filter Rows	Edit	Export/Import	Wrap Cell Content
#	id	event_type	outcome	actor_username	merchant_username	sender_username	receiver_username	amount	client_id	details_json	created_at				
1	1	merchant_checkin	SUCCESS	Canteen	Canteen					{network_id: "192.168.101", "ip": "192.168.101.16", "p": "192.168.101.16"}...	2026-04-02 17:41:10				
2	2	merchant_checkout	SUCCESS	Canteen	Canteen					{ip: "192.168.101.16", "p": "192.168.101.16"}...	2026-04-02 17:41:12				
3	3	student_login	SUCCESS	ayudh						{ip: "192.168.101.7", "p": "192.168.101.7"}...	2026-04-02 20:30:43				
4	4	merchant_checkin	SUCCESS	Canteen	Canteen					{network_id: "192.168.101", "ip": "192.168.101.16", "p": "192.168.101.16"}...	2026-04-02 20:31:18				
5	5	online_payment	REQUESTED			Ayudh	Canteen	20.00		{note: "Food", "ip": "192.168.101.7", "p": "192.168.101.7"}...	2026-04-02 20:31:43				
6	6	online_payment	SUCCESS			Ayudh	Canteen	20.00		{transaction_id: 55, "note": "Food", "ip": "192.168.101.7", "p": "192.168.101.7"}...	2026-04-02 20:31:43				
7	7	merchant_checkin	SUCCESS	Canteen	Canteen					{network_id: "192.168.101", "ip": "192.168.101.16", "p": "192.168.101.16"}...	2026-04-02 20:31:50				
8	8	merchant_checkout	SUCCESS	Canteen	Canteen					{ip: "192.168.101.16", "p": "192.168.101.16"}...	2026-04-02 20:32:00				
9	9	merchant_login	SUCCESS	Canteen	Canteen					{ip: "192.168.101.11", "p": "192.168.101.11"}...	2026-04-02 22:30:21				
10	10	merchant_checkin	SUCCESS	Canteen	Canteen					{network_id: "192.168.101", "ip": "192.168.101.16", "p": "192.168.101.16"}...	2026-04-02 22:32:50				
11	11	merchant_checkout	SUCCESS	Canteen	Canteen					{ip: "192.168.101.11", "p": "192.168.101.11"}...	2026-04-02 22:32:51				
12	12	offline_accepting...	SUCCESS	Canteen	Canteen					{ip: "192.168.101.11", "p": "192.168.101.11"}...	2026-04-02 22:32:54				
13	13	offline_accepting...	FAILED	Canteen	Canteen					{message: "Receiver exited immediately. Check the connection."}	2026-04-02 22:32:54				
14	14	offline_accepting...	SUCCESS	Canteen	Canteen					{ip: "192.168.101.11", "p": "192.168.101.11"}...	2026-04-02 22:32:56				
15	15	offline_accepting...	SUCCESS	Canteen	Canteen					{receiver_id: 853, "ip": "192.168.101.11", "p": "192.168.101.11"}...	2026-04-02 22:33:00				

Figure 55: Audit events table

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	merchant_username	network_id	last_seen
▶	Canteen	192.168.101	2026-04-20 00:00:04
	stationary	192.168.101	2026-04-20 00:00:07
	coffee_station	192.168.101	2026-04-20 00:00:09
*	NULL	NULL	NULL

Figure 56: Merchant presence table

```

def sync_pending(base_url, merchant_username):
    init_db()

    pending = list_pending(merchant_username)
    if not pending:
        print("No pending offline payments to sync.")
        return []

    payload = {
        "merchant_username": merchant_username,
        "payments": [
            {
                "client_tx_id": p["client_tx_id"],
                "sender_username": p["sender_username"],
                "amount": p["amount"],
                "note": p["note"]
            }
            for p in pending
        ]
    }

    url = base_url.rstrip("/") + "/offline/sync"
    print("Syncing to:", url)

    r = requests.post(url, json=payload, timeout=20)
    r.raise_for_status()
    data = r.json()

    results = data.get("results", [])

    # Update local statuses
    conn = db_conn()
    cur = conn.cursor()

    for res in results:
        txid = res.get("client_tx_id")
        st = res.get("status")
        msg = res.get("message")

        if st in ("SUCCESS", "DUPLICATE"):
            cur.execute("""
                UPDATE offline_payments
                SET status='SYNCED', last_error=NULL
                WHERE client_tx_id=?
            """, (txid,))
        else:
            cur.execute("""
                UPDATE offline_payments
                SET status='FAILED', last_error=?
                WHERE client_tx_id=?
            """, (msg or "FAILED", txid))

    conn.commit()
    conn.close()

```

Figure 57: Offline queue storage process

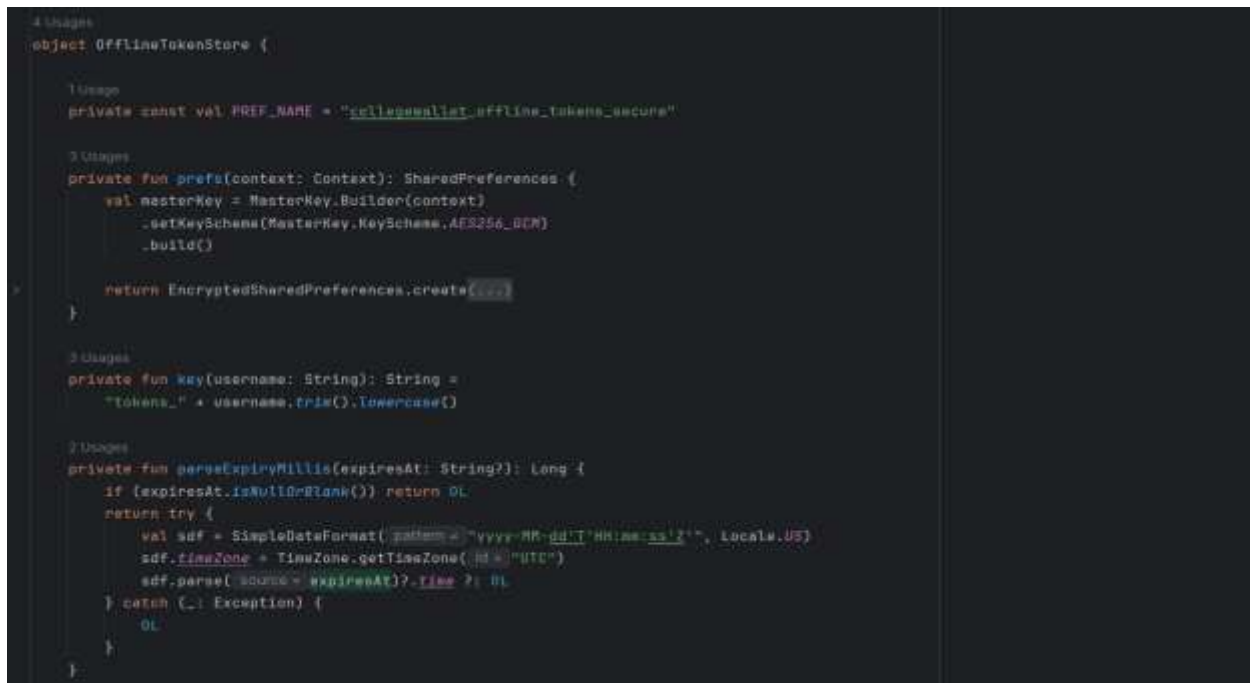
Here, offline transaction data is stored locally in SQLite through `offline_queue.init_db()` and `offline_queue.add_payment()`. This queue is later accessed by `list_pending()` and `sync_pending()` so that delayed offline transactions can be synchronized safely.

3.7.5. Security Implementation

Security was applied primarily using passwords, tokenization, and token expiry, replay protection, audit logging, and secure local storage. The backend also has password-hash support as well as verification logic as opposed to plain credentials. To ensure security in payment, the system does not expose raw reusable payment approval data, but issues signed tokens to ensure their use both online and offline. Online tokens have a short intentional lifetime, whereas offline tokens have a longer lifetime to allow delayed use during network-unavailable conditions. The two types of tokens have identifying values of token IDs, nonces, expiry, and signatures. This allows the system to discard used or out of date tokens when verifying them behind the scenes. Other key events that are recorded using audit logging include the attempts of logins, the token operations, payment requests, and failures, and it is also useful in debugging as well as in providing evidence of reports.

Further security implementation details are provided in [Appendix 9.3.5](#).

Important Screenshots:



```

4 Usage
object OfflineTokenStore {

    1 Usage
    private const val PREF_NAME = "offlinewallet_offline_tokens_secure"

    2 Usage
    private fun prefs(context: Context): SharedPreferences {
        val masterKey = MasterKey.Builder(context)
            .setKeyScheme(MasterKey.KeyScheme.AES256_GCM)
            .build()

        return EncryptedSharedPreferences.create(...)
    }

    3 Usage
    private fun key(username: String): String =
        "tokens_" + username.trim().lowercase()

    2 Usage
    private fun parseExpiryMillis(expiresAt: String?): Long {
        if (expiresAt.isNullOrEmpty()) return 0L
        return try {
            val sdf = SimpleDateFormat(pattern = "yyyy-MM-dd'T'HH:mm:ss'Z'", Locale.US)
            sdf.timeZone = TimeZone.getTimeZone(RD = "UTC")
            sdf.parse(sdf.parse(expiresAt)?.time ?: 0L)
        } catch (e: Exception) {
            0L
        }
    }
}

```

Figure 58: Encrypted offline token storage

Here, security for offline token storage is implemented by OfflineTokenStore using EncryptedSharedPreferences. This ensures that offline token data is saved in encrypted form rather than plain readable local storage.

```
def pay_online_tokenized():
    if status == "USED":
        conn.rollback()
        log_audit_event(
            event_type="online_token_consume",
            outcome="FAILED",
            sender_username=sender_username,
            receiver_username=merchant_username,
            merchant_username=merchant_username,
            amount=amount,
            client_tx_id=token_id,
            details={"reason": "Token already used", "transaction_id": token.get("transaction_id"), "ip": request.remote_addr}
        )
        return jsonify({"status": "failure", "message": "Token already used"}), 400

    if status == "EXPIRED":
        conn.rollback()
        log_audit_event(
            event_type="online_token_consume",
            outcome="FAILED",
            sender_username=sender_username,
            receiver_username=merchant_username,
            merchant_username=merchant_username,
            amount=amount,
            client_tx_id=token_id,
            details={"reason": "Token already expired", "ip": request.remote_addr}
        )
        return jsonify({"status": "failure", "message": "Token expired"}), 400

    if status == "REJECTED":
        conn.rollback()
        log_audit_event(
            event_type="online_token_consume",
            outcome="FAILED",
            sender_username=sender_username,
            receiver_username=merchant_username,
            merchant_username=merchant_username,
            amount=amount,
            client_tx_id=token_id,
            details={"reason": "Token already rejected", "ip": request.remote_addr}
        )
        return jsonify({"status": "failure", "message": "Token rejected"}), 400
```

Figure 59: Online replay / expiry validation

Here, online token security is enforced inside pay_online_tokenized(), where replay and expiry checks are performed before payment is accepted. This prevents reused or expired online tokens from being processed again.

```

def process_offline_sync(merchant_username, payments):
    if offline_token_id:
        cur.execute(
            """
            SELECT token_id, sender_username, nonce, max_amount, expires_at, status, signature, client_tx_id
            FROM offline_payment_tokens
            WHERE token_id=?
            FOR UPDATE
            """
            , (offline_token_id,)
        )
        token_row = cur.fetchone()

        if not token_row:
            conn.rollback()

            log_audit_event(
                event_type="offline_token_sync_rejected",
                outcome="FAILED",
                merchant_username=merchant_username,
                sender_username=sender_username,
                amount=amount,
                client_tx_id=client_tx_id,
                details={"reason": "Offline token not found", "offline_token_id": offline_token_id}
            )

            results.append({
                "client_tx_id": client_tx_id,
                "status": "FAILED",
                "message": "Offline token not found"
            })
            continue

        token_status = (token_row.get("status") or "").upper()
        if token_status != "ISSUED":
            conn.rollback()

            msg = "Offline token already used" if token_status == "USED" else f"Offline token status is {token_status}"

            log_audit_event(
                event_type="offline_token_sync_rejected",
                outcome="FAILED",
                merchant_username=merchant_username,
                sender_username=sender_username,
                amount=amount,
                client_tx_id=client_tx_id,
                details={"reason": msg, "offline_token_id": offline_token_id}
            )

            results.append({
                "client_tx_id": client_tx_id,
                "status": "FAILED",
                "message": msg
            })

```

Figure 60: Offline sync security validation

Here, offline synchronization security is handled by `process_offline_sync()`, which revalidates queued offline transactions before inserting them into the main database. This ensures that offline transactions still pass backend verification even though they were first received locally.

```

def process_offline_sync(merchant_username, payments):
    log_audit_event(
        event_type="offline_token_sync_expired",
        outcome="FAILED",
        merchant_username=merchant_username,
        sender_username=sender_username,
        amount=amount,
        client_tx_id=client_tx_id,
        details={"reason": "Offline token expired", "offline_token_id": offline_token_id}
    )

    results.append({
        "client_tx_id": client_tx_id,
        "status": "FAILED",
        "message": "Offline token expired"
    })
    continue

    expected_signature = build_offline_token_signature(
        sender_username=db_sender,
        token_id=offline_token_id,
        nonce=db_nonce,
        expires_at_str=db_expires_at_str,
        max_amount=max_amount
    )

    if (
        offline_token_nonce != db_nonce or
        offline_token_expires_at != db_expires_at_str or
        offline_token_signature != db_signature or
        offline_token_signature != expected_signature
    ):
        cur.execute(
            """
            UPDATE offline_payment_tokens
            SET status='REJECTED', reject_reason=ks, client_tx_id=ks
            WHERE token_id=ks
            """
            ("Offline token signature mismatch", client_tx_id, offline_token_id)
        )
        conn.commit()

    log_audit_event(
        event_type="offline_token_sync_signature_failed",
        outcome="FAILED",
        merchant_username=merchant_username,
        sender_username=sender_username,
        amount=amount,
        client_tx_id=client_tx_id,
        details={"reason": "Offline token signature mismatch", "offline_token_id": offline_token_id}
    )

```

Figure 61: Offline sync security validation (Token expire and replay)

Here, additional offline replay and expiry protection is enforced during synchronization using the stored offline token metadata. This prevents already-used or expired offline tokens from being accepted again.

```
if offline_token_id:
    log_audit_event(
        event_type="offline_token_sync_rejected",
        outcome="FAILED",
        merchant_username=merchant_username,
        sender_username=sender_username,
        amount=amount,
        client_tx_id=client_tx_id,
        details={"reason": "Insufficient balance", "offline_token_id": offline_token_id}
    )

    log_audit_event(
        event_type="offline_sync_item",
        outcome="FAILED",
        merchant_username=merchant_username,
        sender_username=sender_username,
        amount=amount,
        client_tx_id=client_tx_id,
        details={"message": "Insufficient balance"}
    )

    results.append({"client_tx_id": client_tx_id, "status": "FAILED", "message": "Insufficient balance"})
    continue
```

Figure 62: Offline sync security validation (Insufficient balance)

Here, transaction safety is further maintained through balance validation before payment completion. This prevents transactions that exceed the available student wallet balance from being processed either in the application flow or during backend validation.

4. Testing and Analysis

4.1. Unit Testing

Unit testing in this project was carried out by testing each core feature individually in a focused manner. Each test case was created to ensure that a given functionality of the College Credential Digital Wallet system would perform well in a regular or rather extraordinary condition. The unit testing involved application-side login and payment features, merchant-side controls and transaction processing, and security-related tests, including replay prevention and token expiry tests.

The implementation of the tested features is described in [Section 3.7](#).

Test Plan:

Test Case	Objective
1	To test whether the student and merchant login screens correctly reject invalid credentials.
2	To test whether the student and merchant can log in successfully using valid credentials.
3	To test whether the online payment screen correctly shows that no merchant is accepting payments when no merchant is active.
4	To test whether the online payment screen automatically selects the merchant when only one merchant is accepting payments.
5	To test whether the online payment screen correctly displays multiple merchant options when more than one merchant is accepting payments.
6	To test whether an online payment can be completed successfully with a valid merchant selection and valid payment details.
7	To test whether a successful online payment is correctly reflected in both the student transaction history and the merchant transaction history.

8	To test whether the offline payment screen correctly shows that no merchant is accepting offline payments when no merchant is active.
9	To test whether the offline payment screen correctly verifies the merchant when a merchant is available for offline payment.
10	To test whether an offline payment can be completed successfully with valid payment details.
11	To test whether a successful offline payment is correctly reflected in the student transaction history and in the merchant pending queue.
12	To test whether offline queued transactions can be synchronized successfully and then displayed in the merchant transaction history.
13	To test whether the analytics dashboard updates correctly after multiple successful online payments.
14	To test whether the analytics dashboard updates correctly after multiple successful offline payments.
15	To test whether the logout function works correctly by showing a confirmation prompt and logging the student and merchant out successfully.

Table 2: Unit Testing: Test Plan

4.1.1. Test Case 1

Unit Testing	Objectives
Objective	To test whether the student and merchant login screens correctly reject invalid credentials.
Action	Invalid login credentials were entered into both the student login screen and the merchant login screen, and the login forms were submitted.
Expected Result	The system should reject the login attempts and display an invalid credentials message.
Actual Result	Both the student login screen and the merchant login screen rejected the invalid login attempts and displayed an invalid credentials message.
Conclusion	Test successful.

Table 3: Unit Testing 1

Evidence:

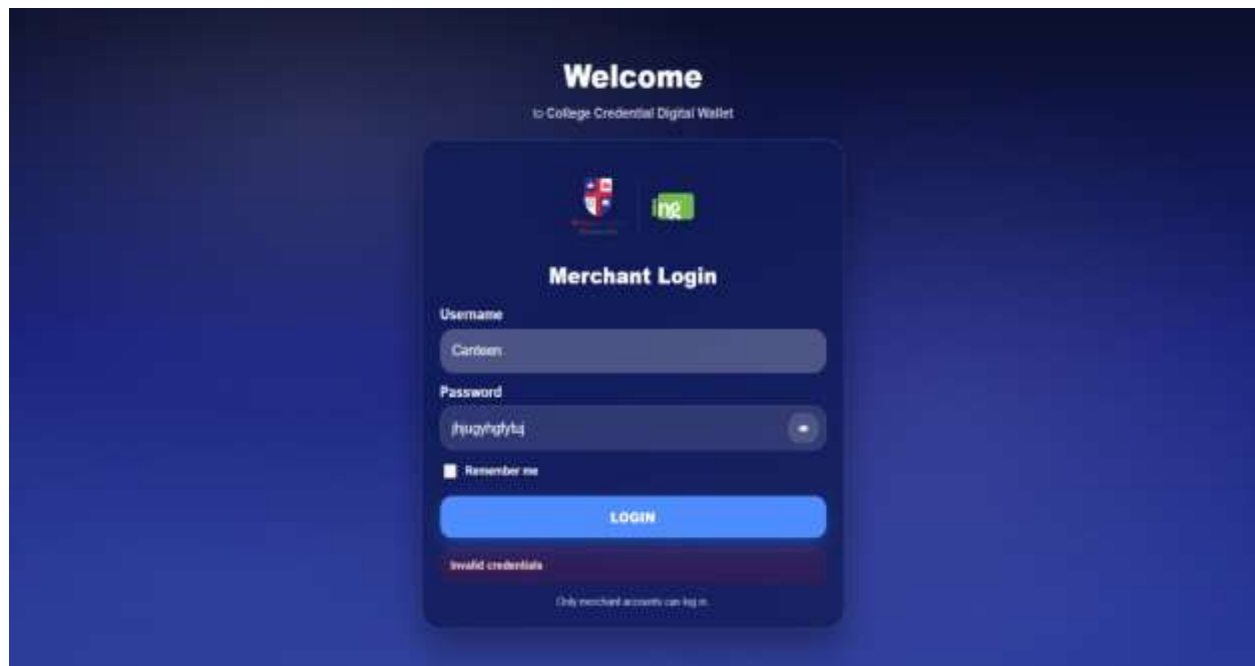


Figure 63: Merchant Invalid Credentials

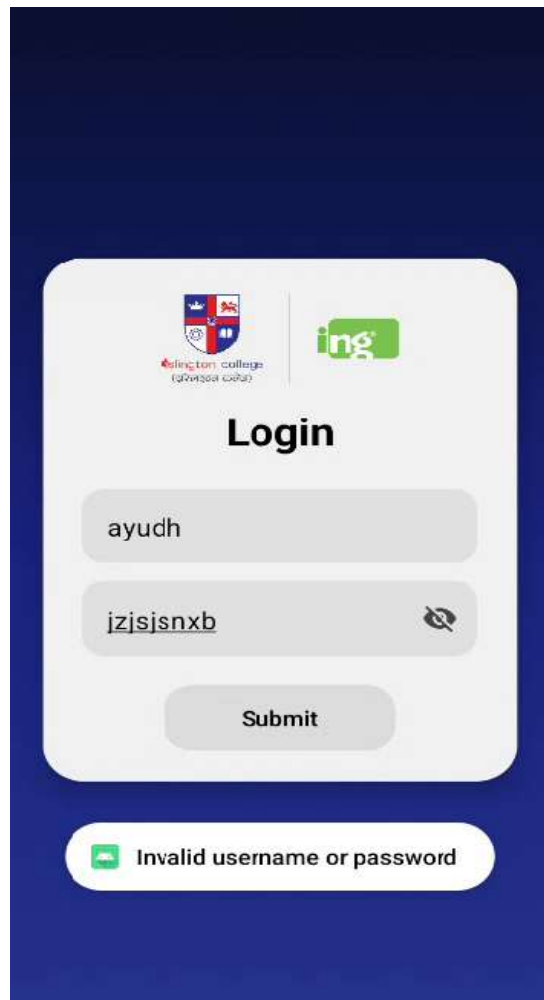


Figure 64 Android Invalid Credentials

4.1.2. Test Case 2

Unit Testing	Objectives
Objective	To test whether the student and merchant can log in successfully using valid credentials.
Action	Valid login credentials were entered into both the student login screen and the merchant login screen, and the login forms were submitted.
Expected Result	The system should allow both users to log in successfully and display their respective dashboards.
Actual Result	Both the student and the merchant were able to log in successfully using valid credentials, and their respective dashboards were displayed.
Conclusion	Test successful.

Table 4: Unit Testing 2

Evidence:

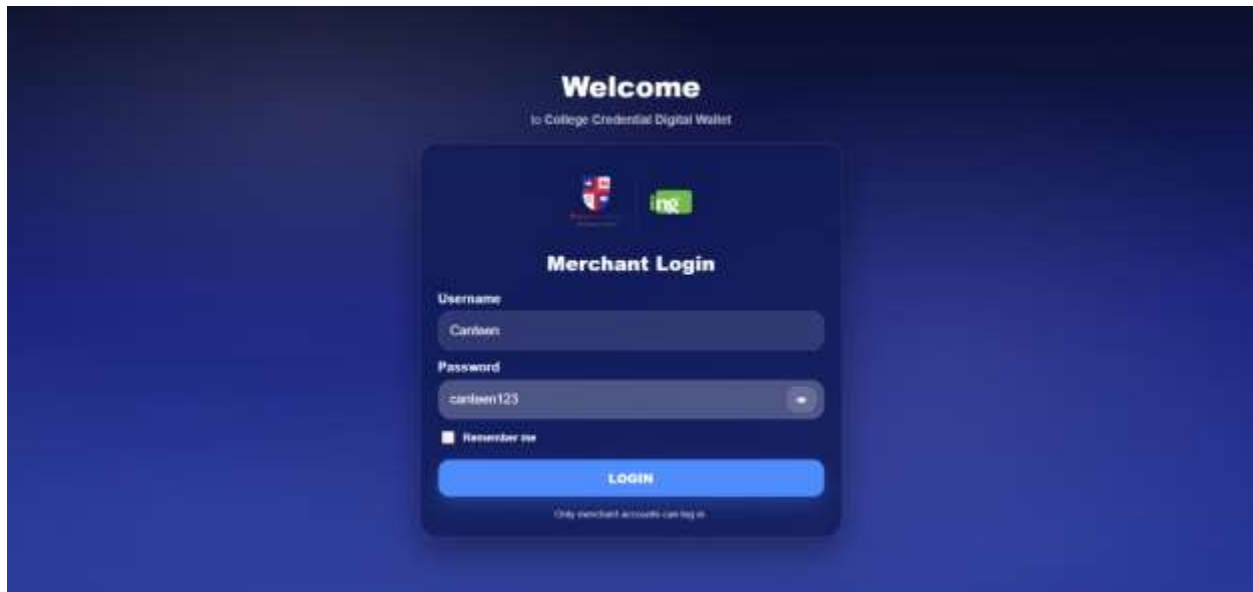


Figure 65: Merchant Valid Credentials

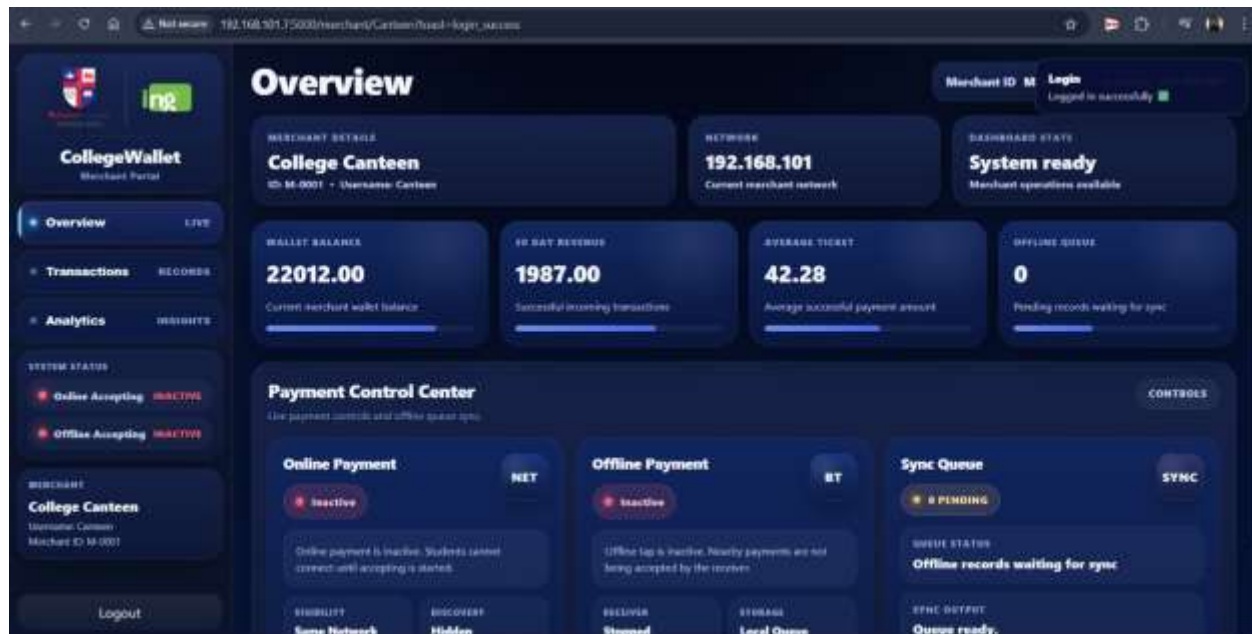


Figure 66: Merchant Login Successful



Figure 67: Android Valid Credentials



Figure 68 Android Login Successful

4.1.3. Test Case 3

Unit Testing	Objectives
Objective	To test whether the online payment screen correctly shows that no merchant is accepting payments when no merchant is active.
Action	The online payment screen was opened while no merchant was accepting online payments.
Expected Result	The system should indicate that no merchant is currently available for online payment.
Actual Result	The online payment screen showed that no merchant was accepting payments at that time.
Conclusion	Test successful.

Table 5: Unit Testing 3

Evidence:

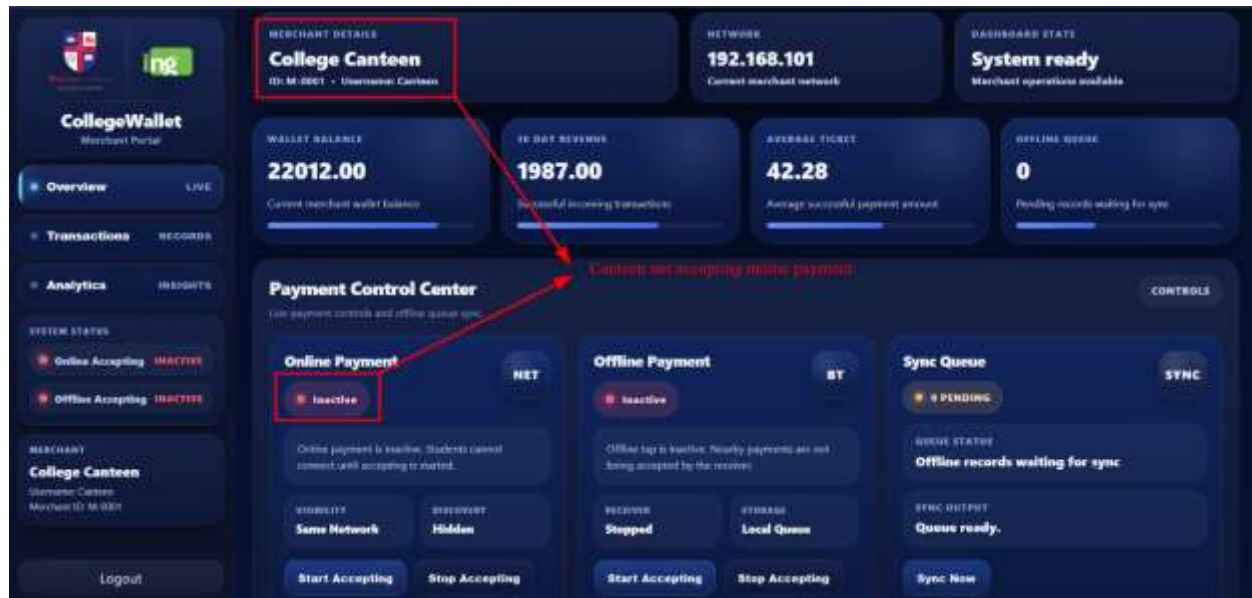


Figure 69: Canteen not accepting online payment

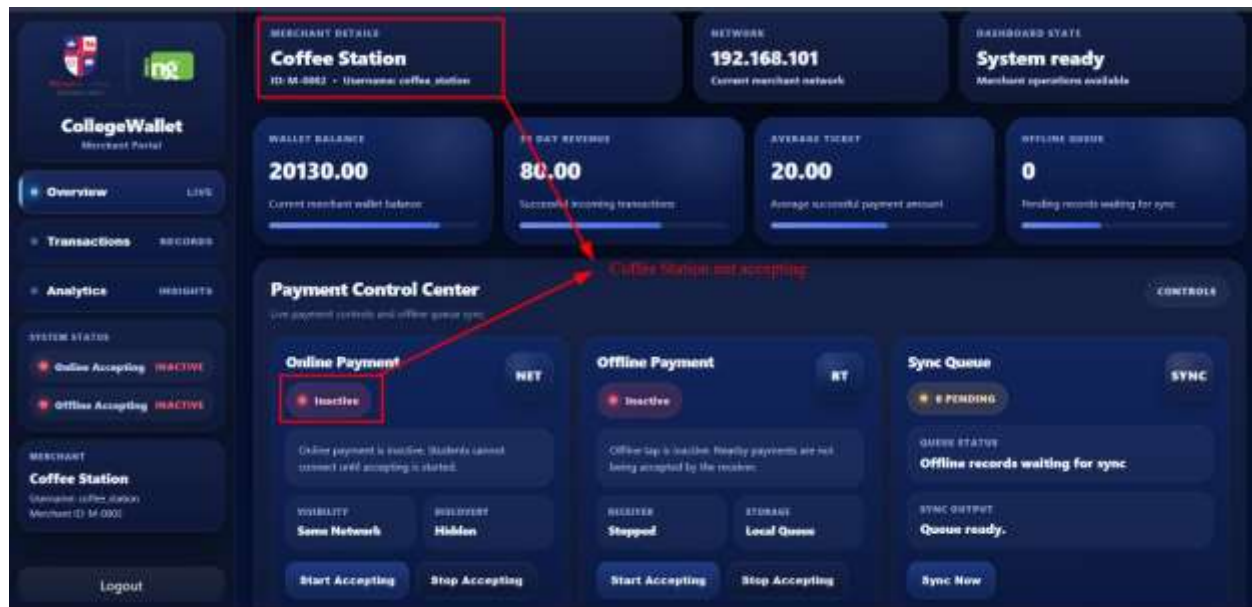


Figure 70: Coffee Station not accepting online payment

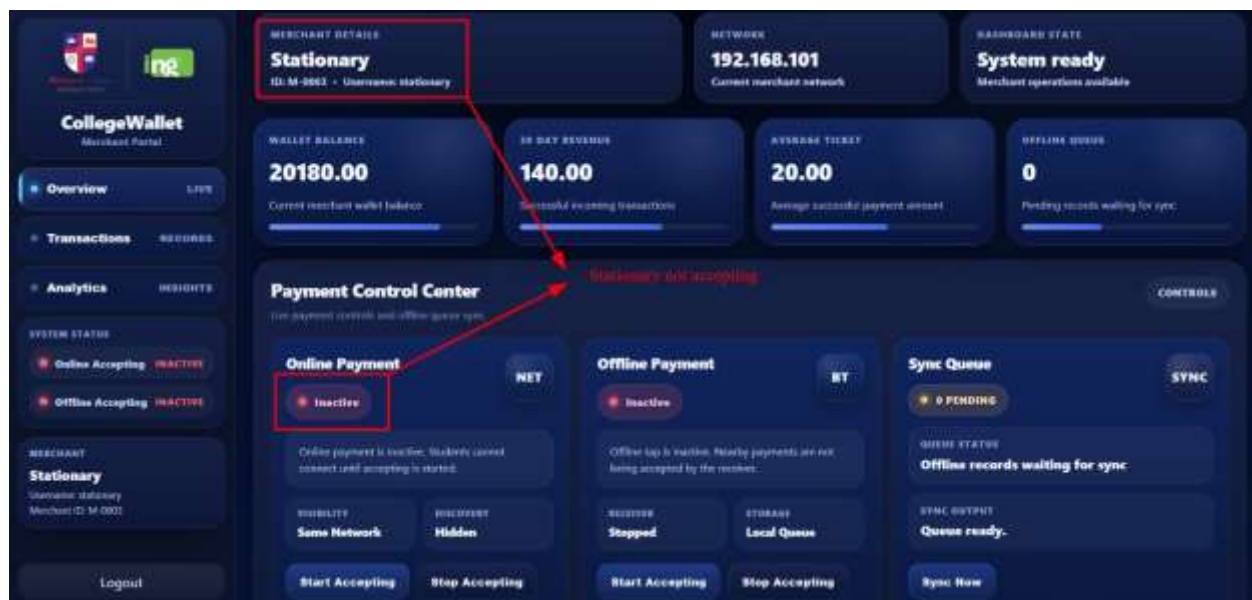


Figure 71: Stationary not accepting online payment

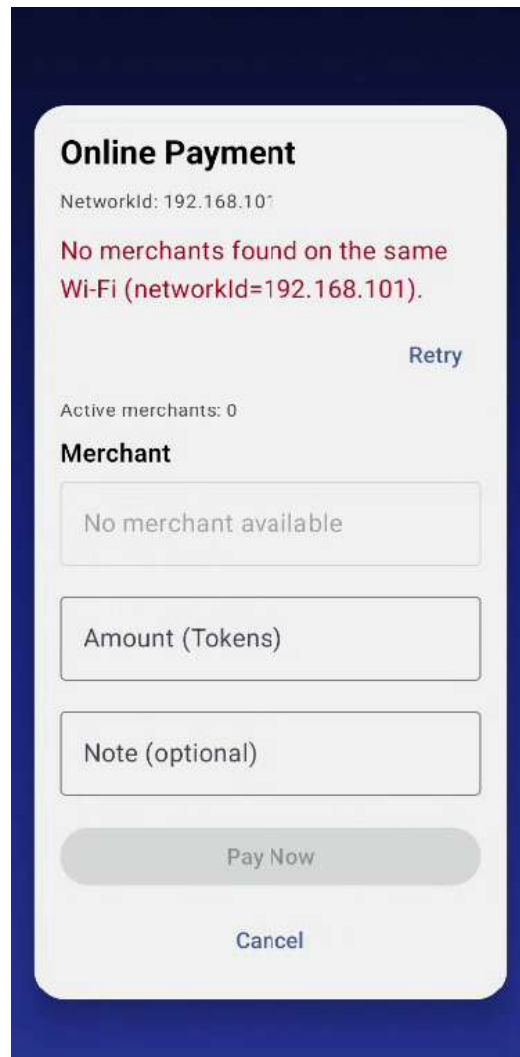


Figure 72: Android shows no one is accepting nearby

4.1.4. Test Case 4

Unit Testing	Objectives
Objective	To test whether the online payment screen automatically selects the merchant when only one merchant is accepting payments.
Action	The online payment screen was opened while only one merchant was accepting online payments.
Expected Result	The system should automatically detect and select the available merchant for payment.
Actual Result	The online payment screen automatically detected and selected the only available merchant for payment.
Conclusion	Test successful.

Table 6: Unit Testing 4

Evidence:

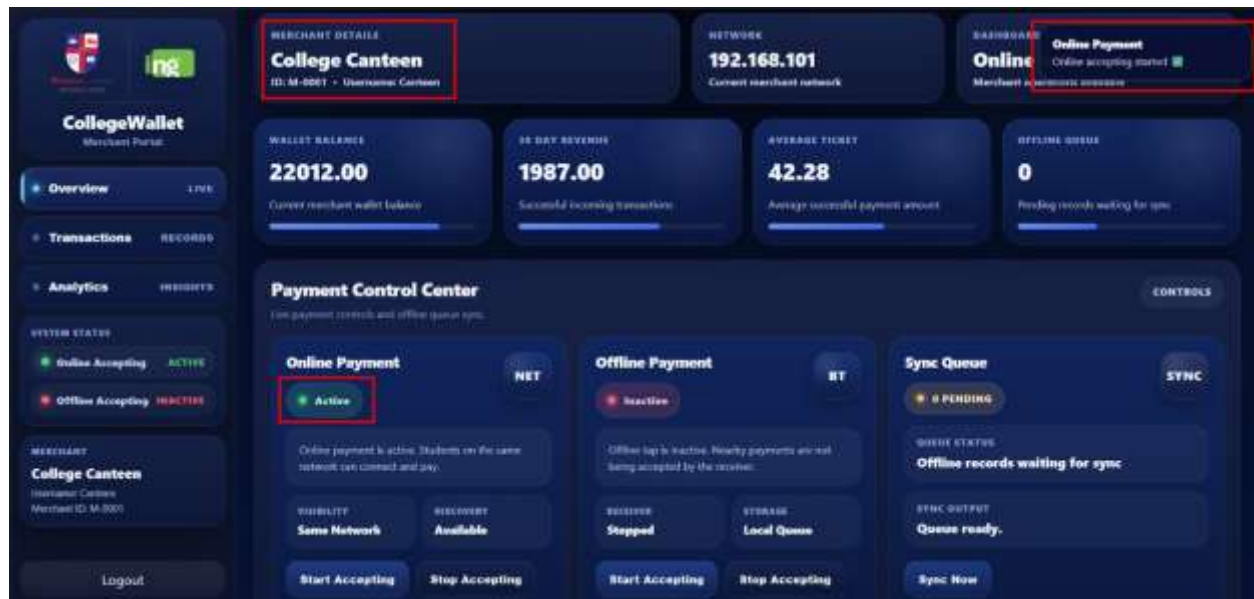


Figure 73: Canteen merchant only accepting online payment

Online Payment

NetworkId: 192.168.101

[Retry](#)

Active merchants: 1

Merchant

Canteen

Auto-selected: Canteen

Amount (Tokens)

Note (optional)

Pay Now

[Cancel](#)

Figure 74: Canteen was shown and selected automatically

Here, rest two merchants i.e. Coffee Station and Stationary were not accepting online payments. So, when only one nearby merchant canteen was accepting it gets selected automatically.

4.1.5. Test Case 5

Unit Testing	Objectives
Objective	To test whether the online payment screen correctly displays multiple merchant options when more than one merchant is accepting payments.
Action	The online payment screen was opened while multiple merchants were accepting online payments at the same time.
Expected Result	The system should display all available merchant options so that the user can select one for payment.
Actual Result	The online payment screen displayed multiple merchant options correctly, allowing the user to choose the desired merchant for payment.
Conclusion	Test successful.

Table 7: Unit Testing 5

Evidence:

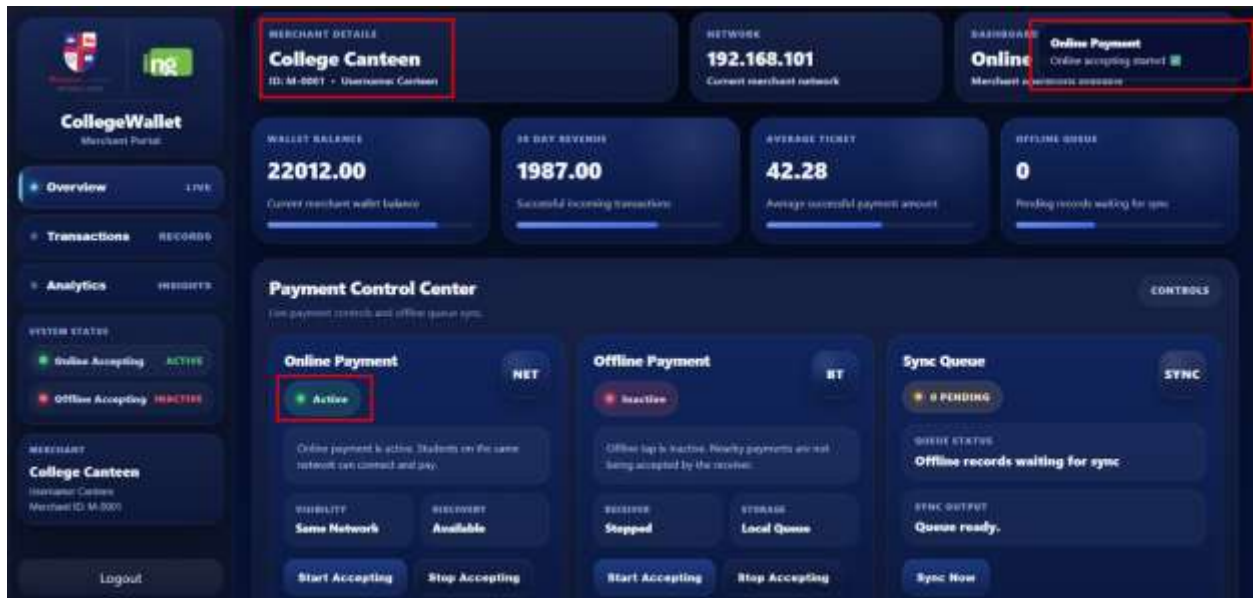


Figure 75: Canteen accepting online payment

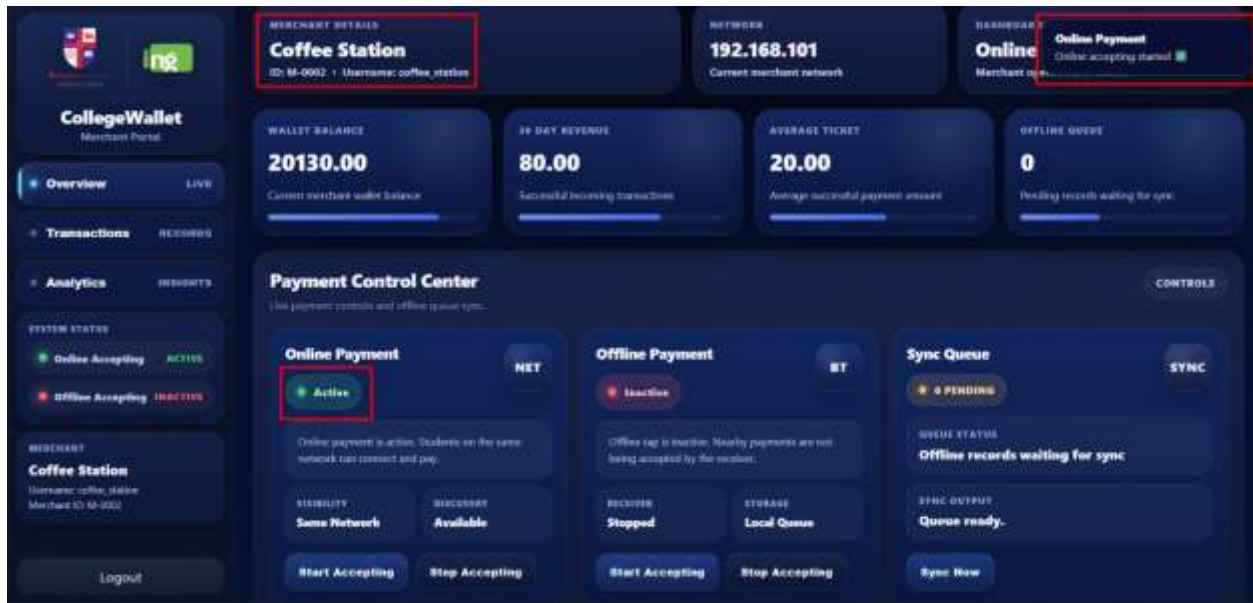


Figure 76: Coffee Station accepting online payment too

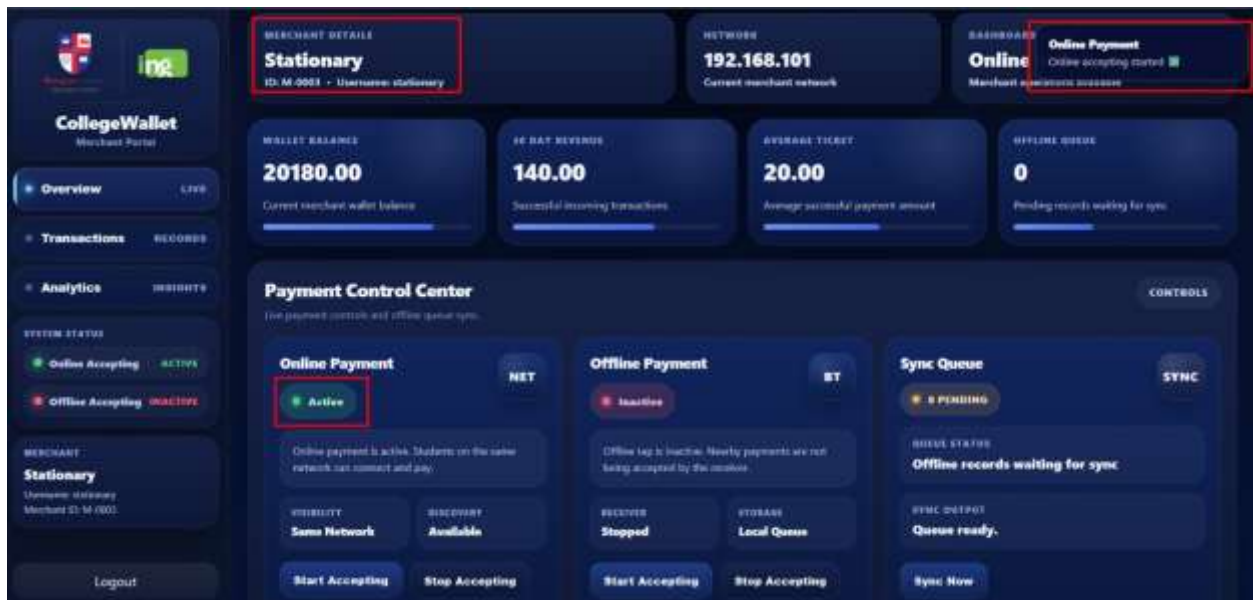


Figure 77: Stationary accepting online payment too

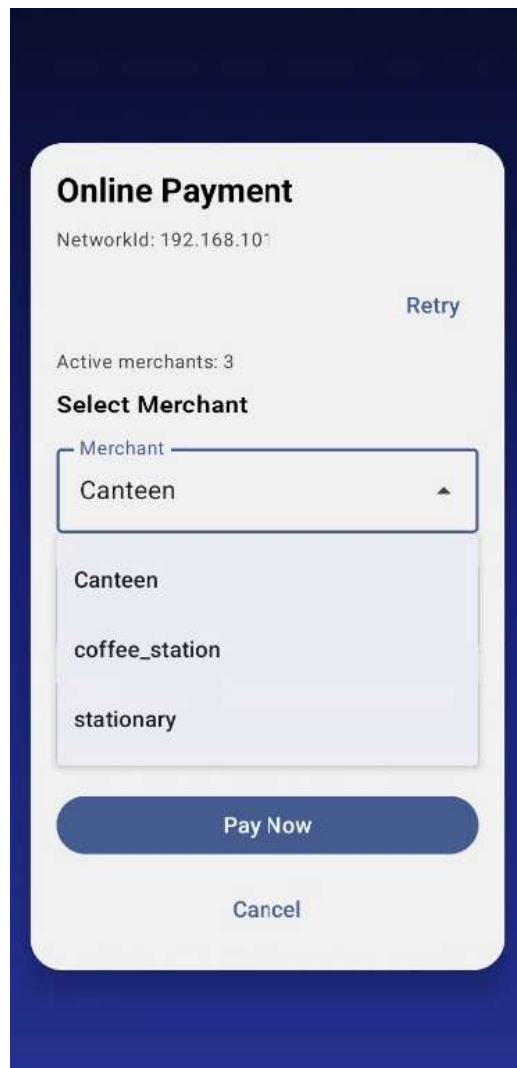


Figure 78: All 3 merchant being displayed

Here, when all 3 merchants start accepting together, all 3 nearby merchants are shown in drop-down from where student can select the preferred merchant and pay.

4.1.6. Test Case 6

Unit Testing	Objectives
Objective	To test whether an online payment can be completed successfully with a valid merchant selection and valid payment details.
Action	A valid merchant was selected on the online payment screen, valid payment details were entered, and the payment was submitted.
Expected Result	The system should process the payment successfully and display a successful transaction message.
Actual Result	The online payment was completed successfully, and the system displayed a successful payment confirmation message.
Conclusion	Test successful.

Table 8: Unit Testing 6

Evidence:



The screenshot shows a mobile application interface for online payment. The screen has a dark blue background. At the top, there is a white card with the title "Online Payment" and a network ID "NetworkId: 192.168.10". Below this, there is a "Retry" button. The section "Active merchants: 3" is followed by a "Select Merchant" section. This section contains a "Merchant" dropdown menu with "Canteen" selected, an "Amount (Tokens)" input field with "100" entered, and a "Note (optional)" input field with "Khaja" entered. At the bottom of the card, there are two buttons: "Pay Now" and "Cancel".

Figure 79: Adding details for online payment



Figure 80: Online payment Successful

4.1.7. Test Case 7

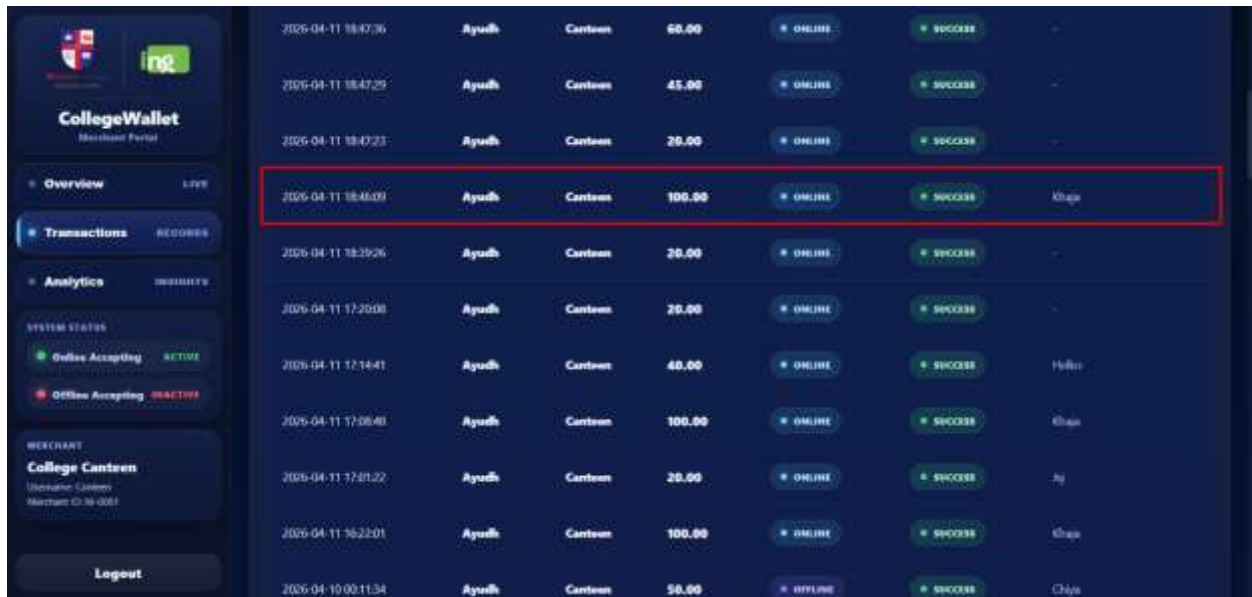
Unit Testing	Objectives
Objective	To test whether a successful online payment is correctly reflected in both the student transaction history and the merchant transaction history.
Action	A successful online payment was completed from the student application, and then the transaction history was checked on both the student side and the merchant side.
Expected Result	The successful online payment should be recorded and displayed in both the student transaction history and the merchant transaction history.
Actual Result	The successful online payment was displayed correctly in both the student transaction history and the merchant transaction history.
Conclusion	Test successful.

Table 9: Unit Testing 7

Evidence:



Figure 81: Android Displaying successful online transaction history



2025-04-11 18:47:36	Ayudh	Canteen	60.00	ONLINE	SUCCESS	-
2025-04-11 18:47:29	Ayudh	Canteen	45.00	ONLINE	SUCCESS	-
2025-04-11 18:47:23	Ayudh	Canteen	20.00	ONLINE	SUCCESS	-
2025-04-11 18:46:09	Ayudh	Canteen	100.00	ONLINE	SUCCESS	Krupa
2025-04-11 18:39:26	Ayudh	Canteen	20.00	ONLINE	SUCCESS	-
2025-04-11 17:29:08	Ayudh	Canteen	20.00	ONLINE	SUCCESS	-
2025-04-11 17:14:41	Ayudh	Canteen	40.00	ONLINE	SUCCESS	Hellu
2025-04-11 17:08:40	Ayudh	Canteen	100.00	ONLINE	SUCCESS	Krupa
2025-04-11 17:01:22	Ayudh	Canteen	20.00	ONLINE	SUCCESS	Jy
2025-04-11 16:22:01	Ayudh	Canteen	100.00	ONLINE	SUCCESS	Krupa
2025-04-10 00:11:34	Ayudh	Canteen	50.00	OFFLINE	SUCCESS	Olga

Figure 82: Merchant web displaying successful online transaction history

4.1.8. Test Case 8

Unit Testing	Objectives
Objective	To test whether the offline payment screen correctly shows that no merchant is accepting offline payments when no merchant is active.
Action	The offline payment screen was opened while no merchant was accepting offline payments.
Expected Result	The system should indicate that no merchant is currently available for offline payment.
Actual Result	The offline payment screen showed that no merchant was accepting offline payments at that time.
Conclusion	Test successful.

Table 10: Unit testing 8

Evidence:

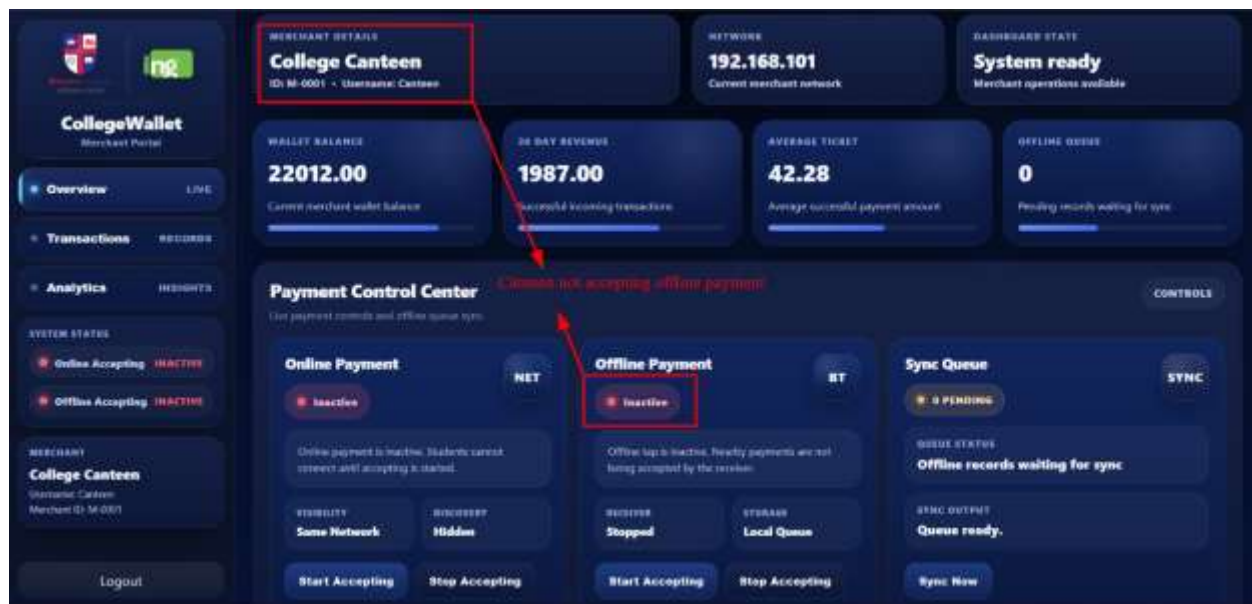


Figure 83: Canteen not accepting offline payment

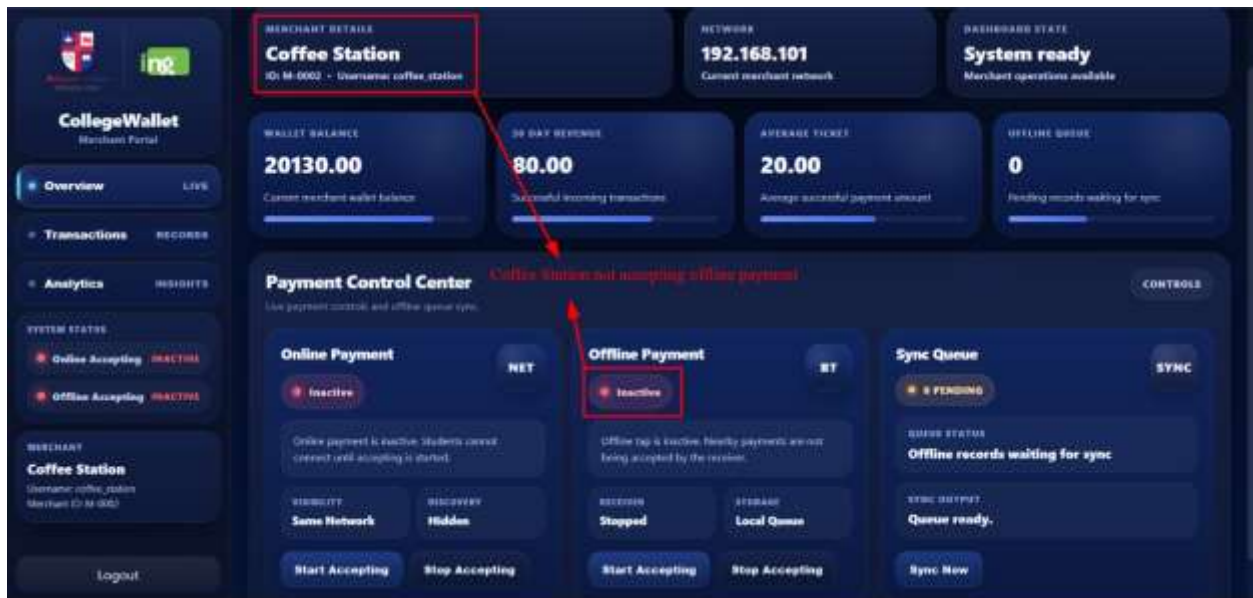


Figure 84: Coffee Station not accepting offline

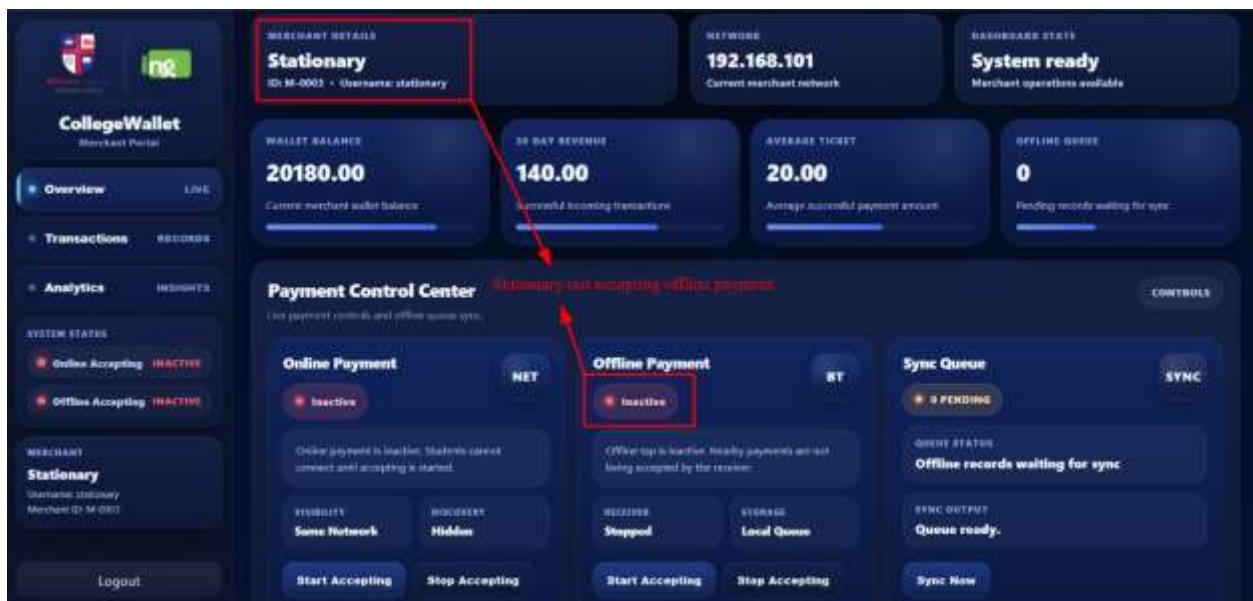


Figure 85: Stationary not accepting offline payment

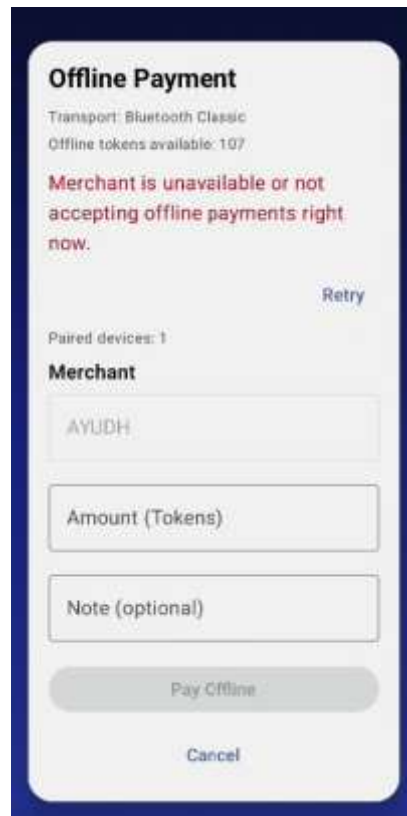


Figure 86: Android Shows unavailable when no one is accepting nearby

4.1.9. Test Case 9

Unit Testing	Objectives
Objective	To test whether the offline payment screen correctly verifies the merchant when a merchant is available for offline payment.
Action	The offline payment screen was opened while one merchant was available for offline payment.
Expected Result	The system should detect and verify the available merchant correctly for offline payment.
Actual Result	The offline payment screen correctly detected and verified the available merchant for offline payment.
Conclusion	Test successful.

Table 11: Unit Testing 9

Evidence:

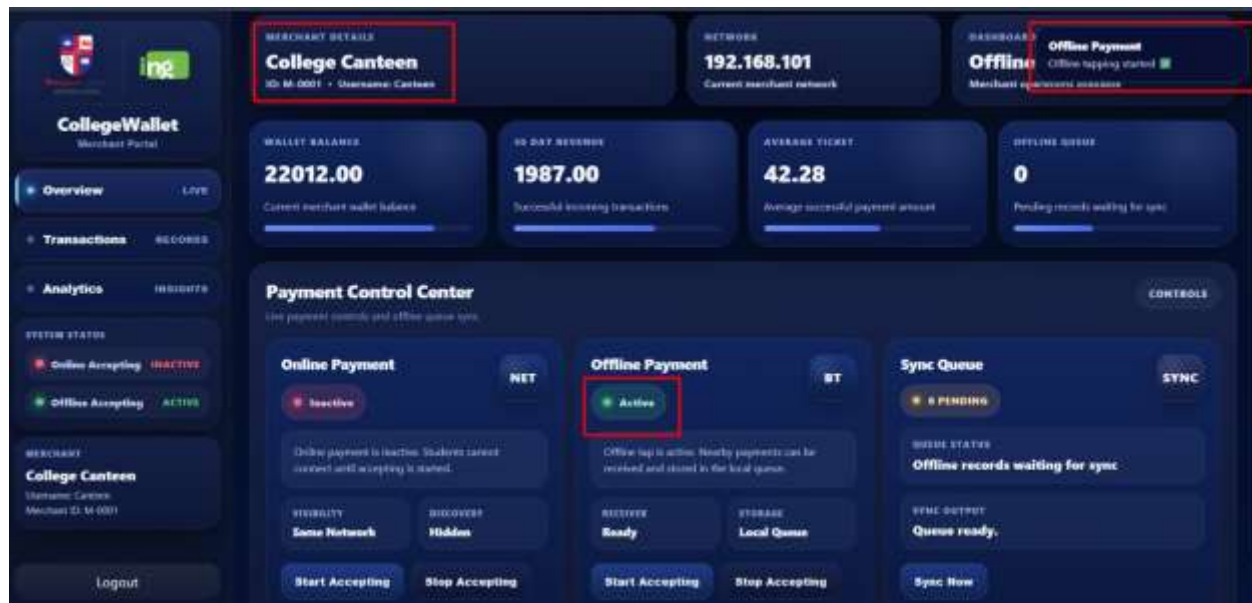


Figure 87: Canteen starts accepting offline payment

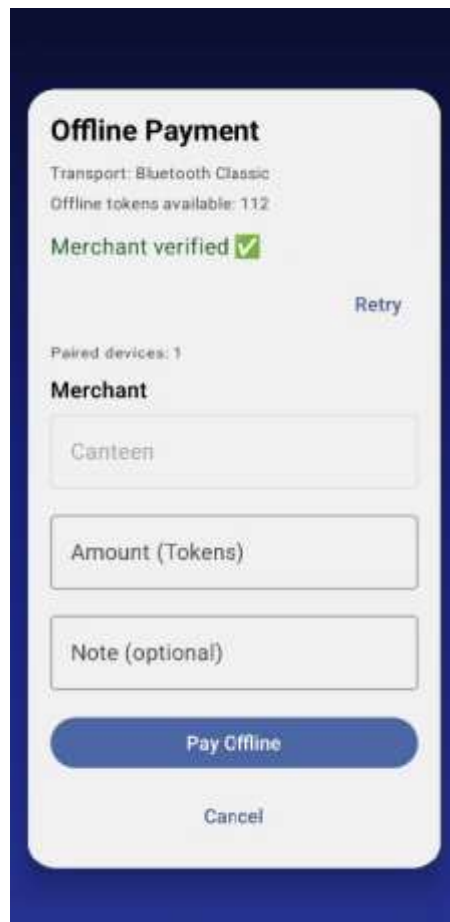


Figure 88: Canteen verified as nearby accepting merchant

4.1.10. Test Case 10

Unit Testing	Objectives
Objective	To test whether an offline payment can be completed successfully with valid payment details.
Action	Valid payment details were entered on the offline payment screen and the payment was submitted.
Expected Result	The system should complete the offline payment successfully and display a success confirmation message.
Actual Result	The offline payment was completed successfully, and the system displayed a success confirmation message.
Conclusion	Test successful.

Table 12: Unit Testing 10

Evidence:

The screenshot shows a mobile application interface for 'Offline Payment'. At the top, it says 'Offline Payment' in bold. Below that, it displays 'Transport: Bluetooth Classic' and 'Offline tokens available: 112'. A green checkmark indicates 'Merchant verified'. There is a 'Retry' button on the right. Below this, it says 'Paired devices: 1'. The 'Merchant' field contains the text 'Canteen'. The 'Amount (Tokens)' field contains '200'. The 'Note (optional)' field contains 'Lunch'. At the bottom, there are two buttons: 'Pay Offline' (blue) and 'Cancel' (grey).

Figure 89: Adding details for offline payment



Figure 90: Offline payment successful

4.1.11. Test Case 11

Unit Testing	Objectives
Objective	To test whether a successful offline payment is correctly reflected in the student transaction history and in the merchant pending queue.
Action	A successful offline payment was completed from the student application, and then the student transaction history and merchant pending queue were checked.
Expected Result	The offline payment should be recorded in the student transaction history and should appear in the merchant pending queue before synchronization.
Actual Result	The successful offline payment was displayed in the student transaction history and was also shown in the merchant pending queue.
Conclusion	Test successful.

Table 13: Unit Testing 11

Evidence:



Figure 91: Offline payment displayed in transaction history

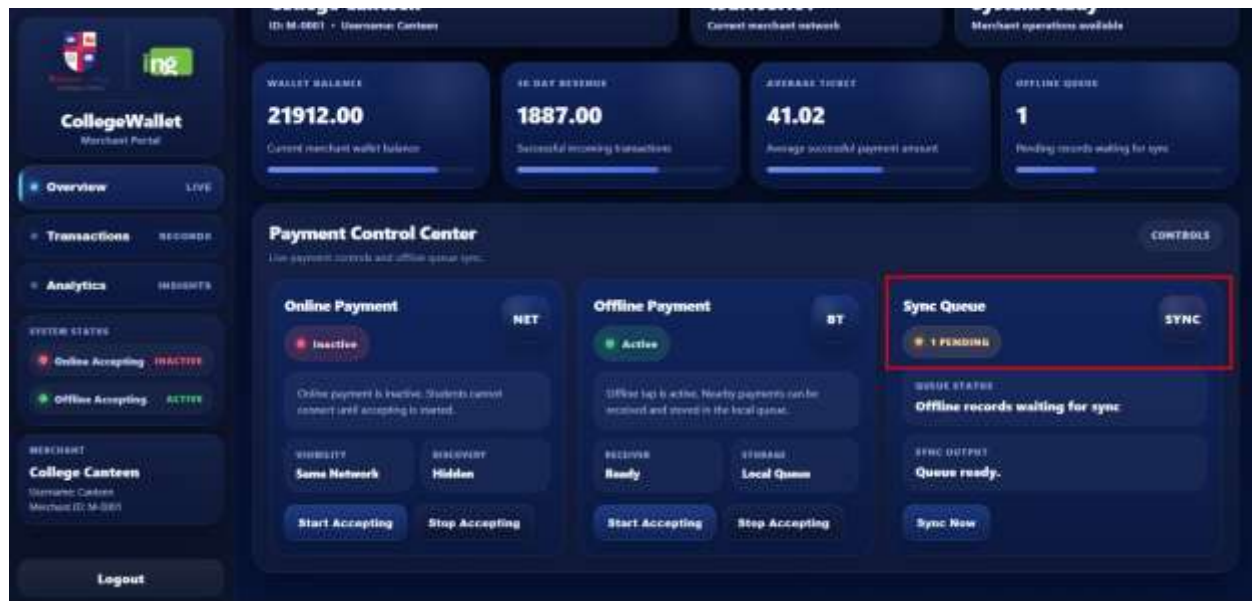


Figure 92: Pending payment in merchant after student transfer

4.1.12. Test Case 12

Unit Testing	Objectives
Objective	To test whether offline queued transactions can be synchronized successfully and then displayed in the merchant transaction history.
Action	The merchant synchronization function was performed after an offline payment had been placed in the pending queue.
Expected Result	The pending offline transaction should be synchronized successfully and then moved to the merchant transaction history.
Actual Result	The offline queued transaction was synchronized successfully and was displayed in the merchant transaction history after synchronization.
Conclusion	Test successful.

Table 14: Unit Testing 12

Evidence:

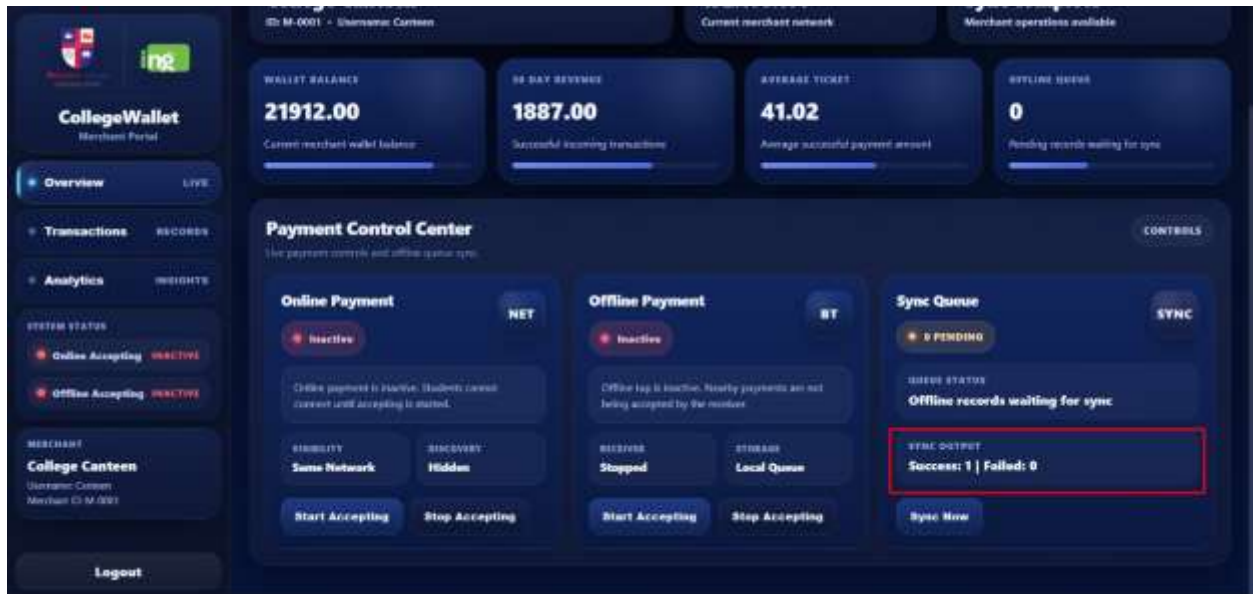


Figure 93: Successful sync after syncing the payment

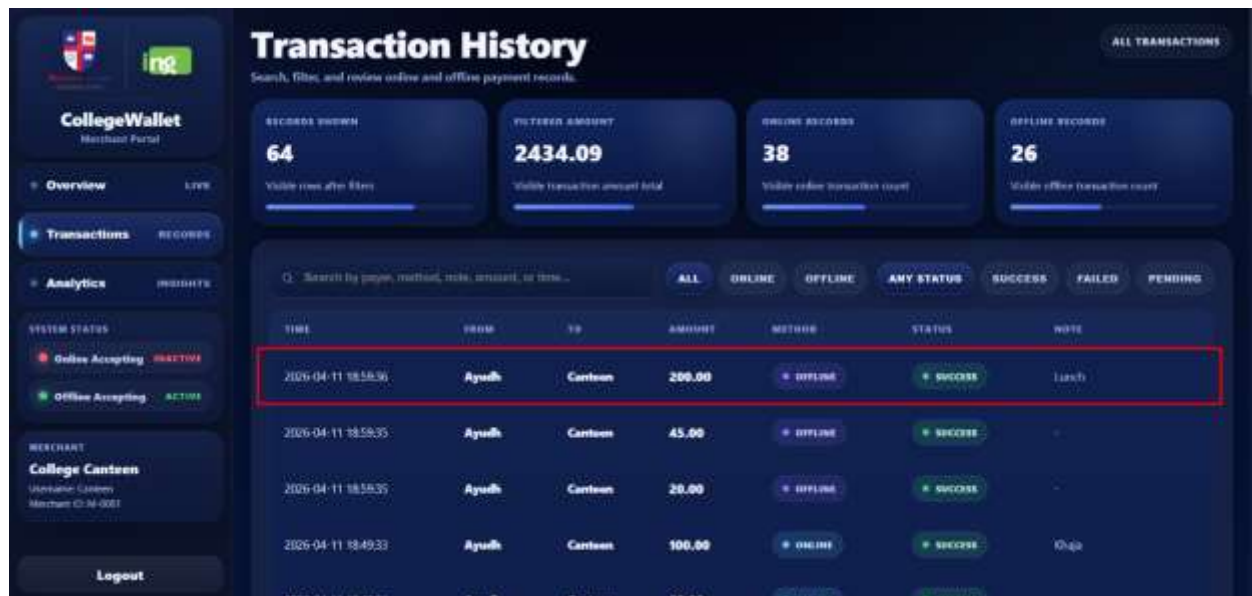


Figure 94: Successfully displayed in transaction history after syncing

4.1.13. Test Case 13

Unit Testing	Objectives
Objective	To test whether the analytics dashboard updates correctly after multiple successful online payments.
Action	Multiple successful online payments were performed, and the analytics dashboard was checked afterwards.
Expected Result	The analytics dashboard should update the revenue, average transaction value, daily transaction average, payment method split, and other related metrics correctly after the successful online payments.
Actual Result	After the successful online payments were completed, the analytics dashboard updated the revenue, average transaction value, daily transaction average, payment method split, and other related metrics correctly.
Conclusion	Test successful.

Table 15: Unit Testing 13

Evidence:



Figure 95: Analytics before

Timestamp	Merchant	Amount	Payment Method	Status	Item
2026-04-14 23:34:04	Ayudh	200.00	ONLINE	SUCCESS	Food
2026-04-14 23:33:52	Ayudh	200.00	ONLINE	SUCCESS	Food
2026-04-14 23:33:07	Ayudh	100.00	ONLINE	SUCCESS	Food
2026-04-14 23:32:41	Ayudh	20.00	ONLINE	SUCCESS	Food
2026-04-14 23:32:22	Ayudh	200.00	ONLINE	SUCCESS	Food
2026-04-14 23:32:13	Ayudh	200.00	ONLINE	SUCCESS	Food
2026-04-14 23:31:56	Ayudh	200.00	ONLINE	SUCCESS	Food
2026-04-14 23:31:44	Ayudh	200.00	ONLINE	SUCCESS	Food
2026-04-14 23:31:37	Ayudh	200.00	ONLINE	SUCCESS	Food
2026-04-14 23:31:28	Ayudh	200.00	ONLINE	SUCCESS	Food
2026-04-14 23:31:09	Ayudh	200.00	ONLINE	SUCCESS	Food

Figure 96: Total 2000 tokens transaction online



Figure 97: Analytics after

Here, successful online payment of total 2000 tokens was made. After those transactions were completed, the analytics dashboard was checked, and the revenue, average transaction value, daily transaction average, payment method split, and other related metrics were updated accordingly.

4.1.14 Test Case 14

Unit Testing	Objectives
Objective	To test whether the analytics dashboard updates correctly after multiple successful offline payments.
Action	Multiple successful offline payments were performed and synchronized, and the analytics dashboard was checked afterwards.
Expected Result	The analytics dashboard should update the revenue, average transaction value, daily transaction average, payment method split, and other related metrics correctly after the successful offline payments are synchronized.
Actual Result	After the successful offline payments were completed and synchronized, the analytics dashboard updated the revenue, average transaction value, daily transaction average, payment method split, and other related metrics correctly.
Conclusion	Test successful.

Table 16: Unit Testing 14

Evidence:



Figure 98: Analytics before offline payment

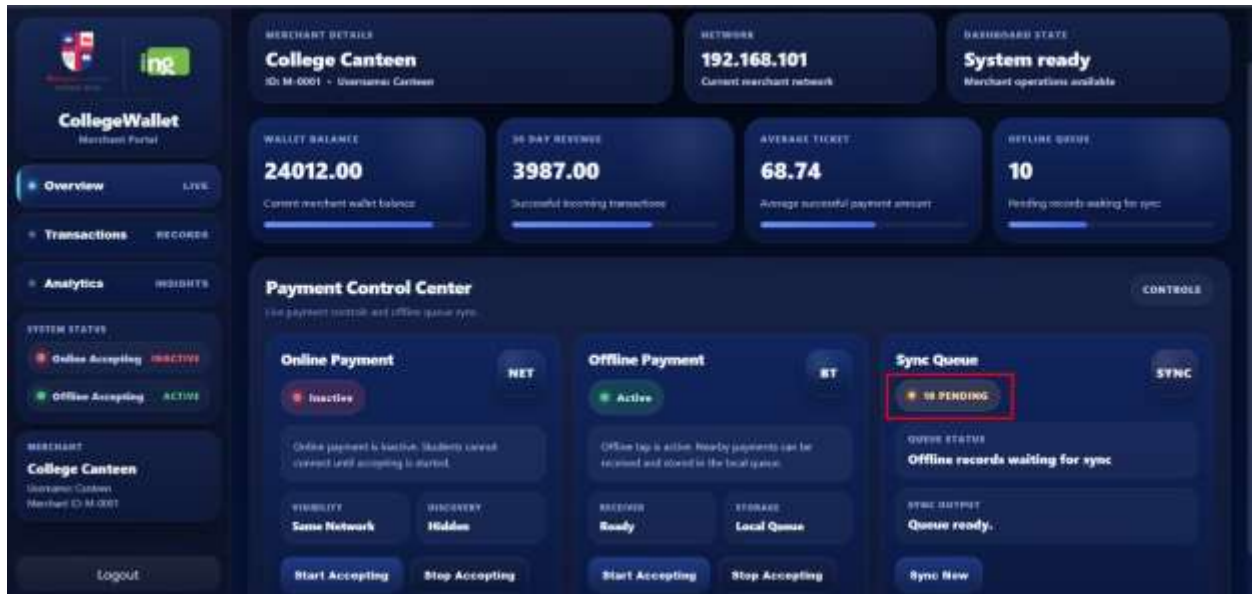


Figure 99: Offline payments pending

The screenshot displays the CollegeWallet Merchant Portal dashboard for 'College Canteen'. The main content area shows a list of transactions. The transactions are listed in a table with columns: TIME, FROM, TO, AMOUNT, METHOD, STATUS, and NOTE. The transactions are all successful offline payments of 200.00, recorded on 2025-04-14 at 23:54:38. The transactions are listed in a table with columns: TIME, FROM, TO, AMOUNT, METHOD, STATUS, and NOTE. The transactions are all successful offline payments of 200.00, recorded on 2025-04-14 at 23:54:38.

TIME	FROM	TO	AMOUNT	METHOD	STATUS	NOTE
2025-04-14 23:54:38	Ayudh	Canteen	200.00	OFFLINE	SUCCESS	Food
2025-04-14 23:54:38	Ayudh	Canteen	200.00	OFFLINE	SUCCESS	Food
2025-04-14 23:54:38	Ayudh	Canteen	200.00	OFFLINE	SUCCESS	Food
2025-04-14 23:54:38	Ayudh	Canteen	200.00	OFFLINE	SUCCESS	Food
2025-04-14 23:54:38	Ayudh	Canteen	200.00	OFFLINE	SUCCESS	Food
2025-04-14 23:54:38	Ayudh	Canteen	200.00	OFFLINE	SUCCESS	Food
2025-04-14 23:54:38	Ayudh	Canteen	200.00	OFFLINE	SUCCESS	Food
2025-04-14 23:54:38	Ayudh	Canteen	200.00	OFFLINE	SUCCESS	Food
2025-04-14 23:54:38	Ayudh	Canteen	200.00	OFFLINE	SUCCESS	Food
2025-04-14 23:54:38	Ayudh	Canteen	200.00	OFFLINE	SUCCESS	Food

Figure 100: Successful offline transactions



Figure 101: Analytics after offline payments

Here, multiple successful offline payments of 200 tokens each (total 2000 tokens) were made. After those transactions were completed and synchronized, the analytics dashboard was checked, and the revenue, average transaction value, daily transaction average, payment method split, and other related metrics were updated accordingly.

4.1.15. Test Case 15

Unit Testing	Objectives
Objective	To test whether the logout function works correctly for both the student and the merchant.
Action	The logout function was selected by both the student and the merchant, and the confirmation prompt was accepted.
Expected Result	The system should display a confirmation prompt and then log the student and merchant out successfully after confirmation.
Actual Result	The system displayed the confirmation prompt and then logged both the student and the merchant out successfully after confirmation.
Conclusion	Test successful.

Table 17: Unit Testing 15

Evidence:

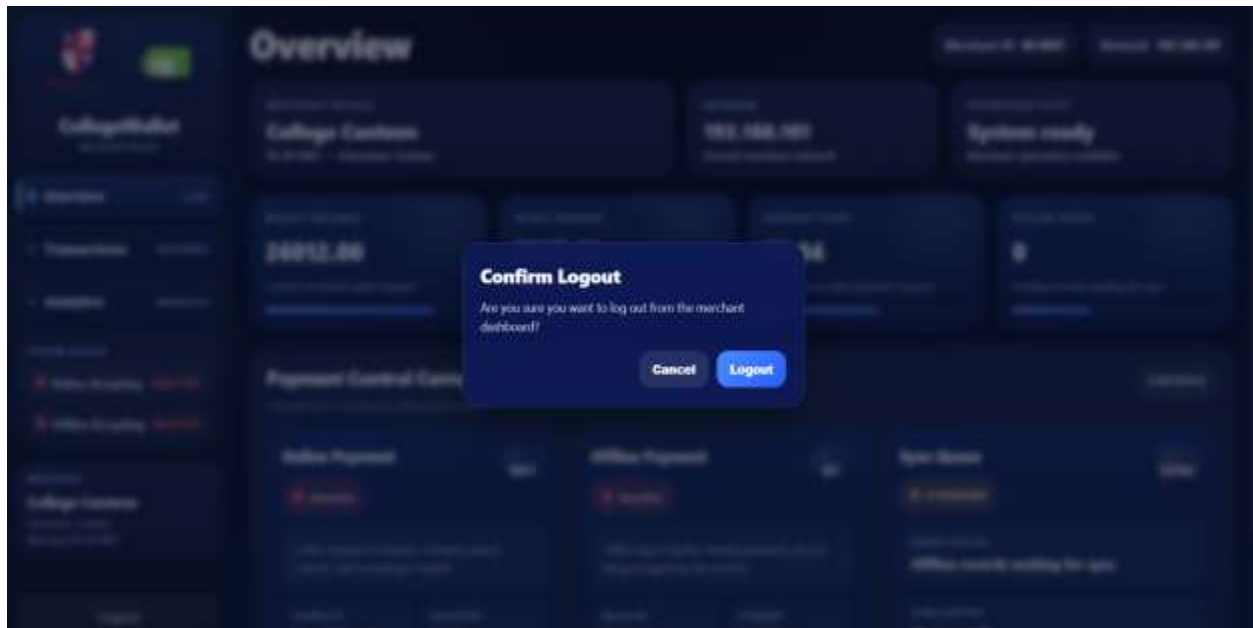


Figure 102: Merchant logout Confirmation

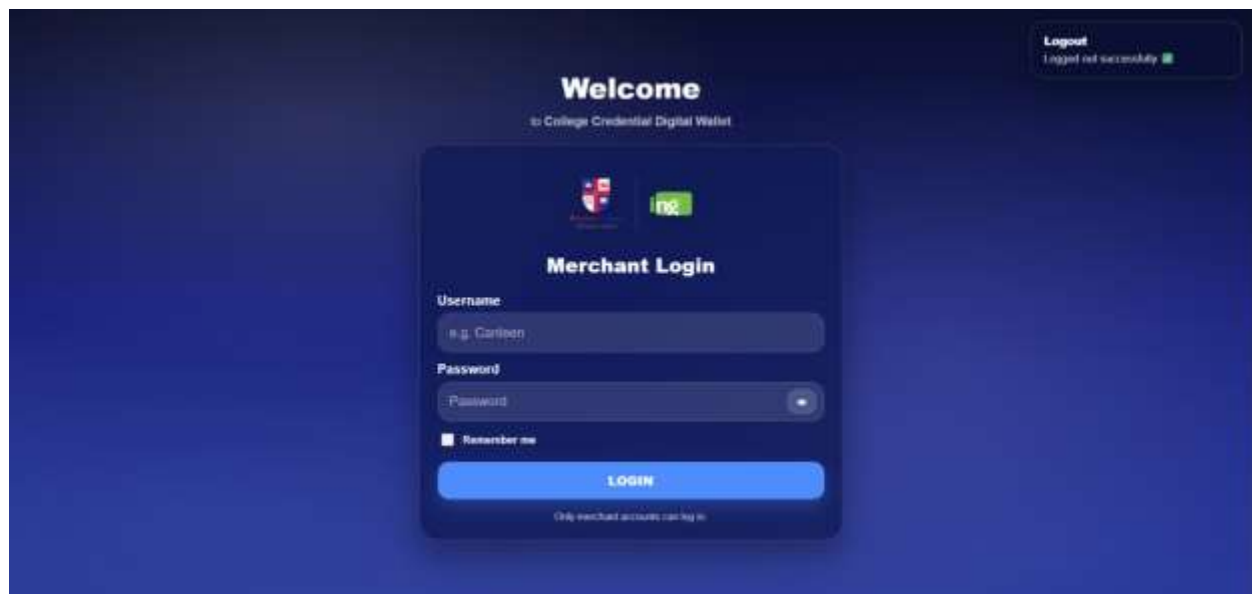


Figure 103: Merchant Logout Successful

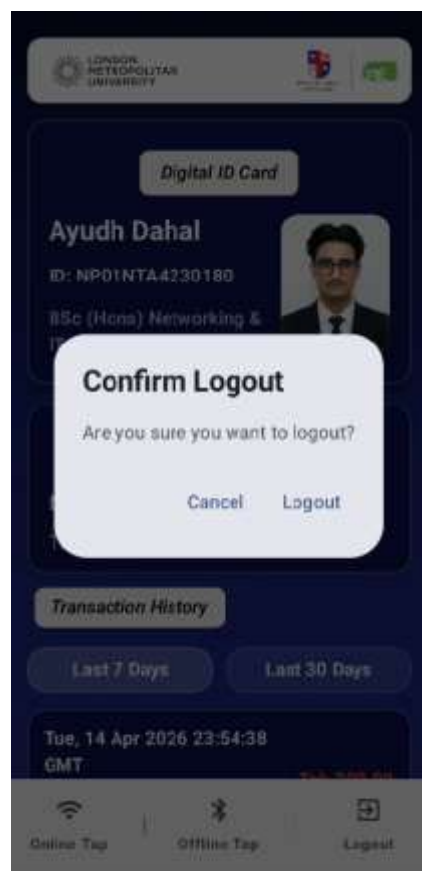


Figure 104: Android Logout Confirmation

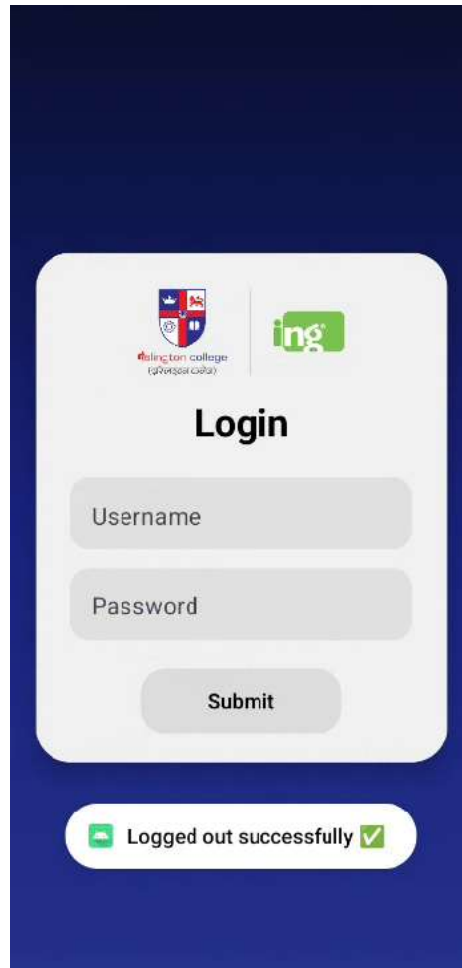


Figure 105: Android Logout Successful

4.2. System Testing

System testing was carried out to evaluate the complete behaviour of the College Credential Digital Wallet system by testing the full workflow for both the application and merchant system. Unlike unit testing, which is mainly concerned with individual functionality, system testing in this case aimed at ensuring that all the parts of the system function as intended when used collectively. System testing is concentrated on the entire process of online payments and offline payments.

The related system components and implementation details are described in [Section 3.7](#).

Test Plan:

Test Case	Objective
1	To test the complete online payment workflow from student login to successful payment completion and transaction display on both the student and merchant sides.
2	To test the complete offline payment workflow from student login to successful offline payment, pending queue display on the merchant side, and final synchronization into merchant transaction history.

Table 18: System Testing Test Plan

4.2.1. Test Case 1

Unit Testing	Objectives
Objective	To test the complete online payment workflow from student login to successful payment completion and transaction display on both the student and merchant sides.
Action	The student logged in and completed the online transaction. After that, transaction history was checked on both sides.
Expected Result	The complete online payment process should be performed successfully, and the completed transaction should be displayed correctly in both the student and the merchant transaction history.
Actual Result	The student logged in successfully, completed the online payment process, and the transaction was displayed correctly in both the student and the merchant transaction history.
Conclusion	Test successful.

Table 19: System Testing 1

Evidence:

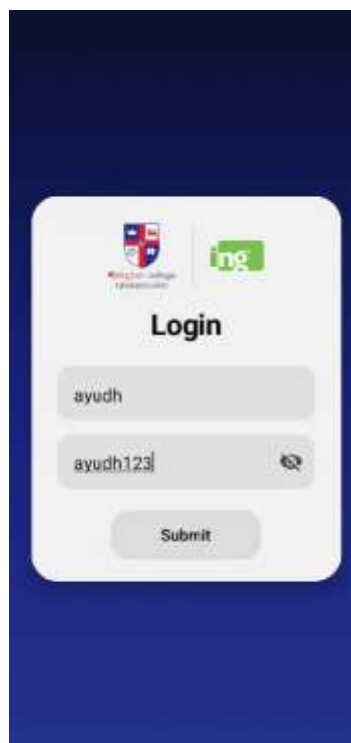


Figure 106: Adding valid student details for login



Figure 107: Login Successful

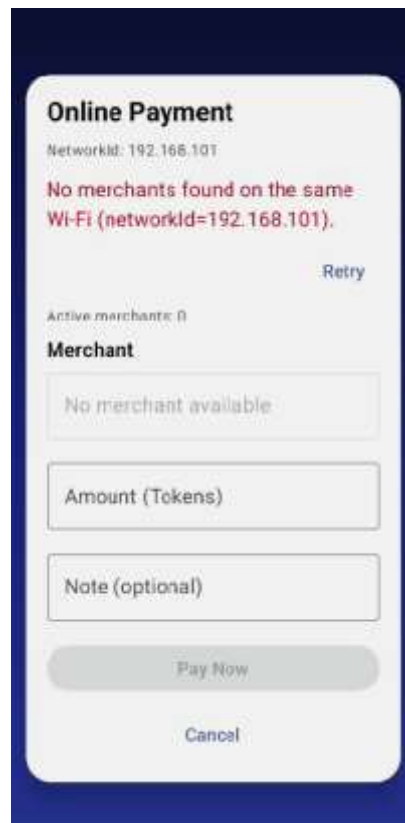


Figure 108: No one accepting online payment

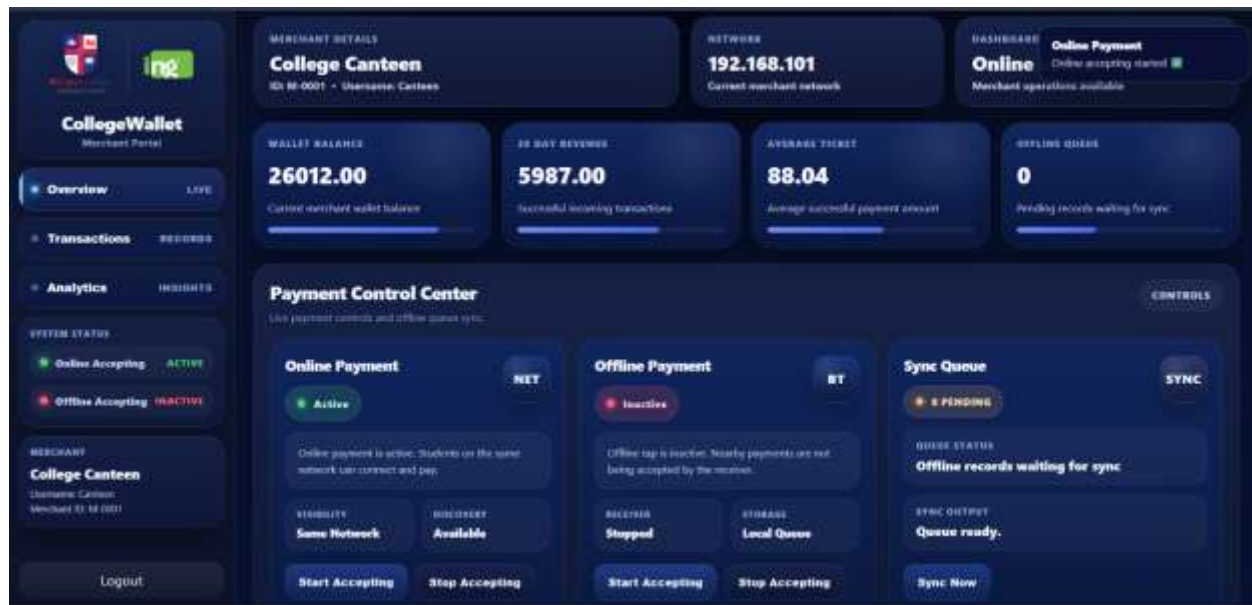


Figure 109: Merchant Online tapping started

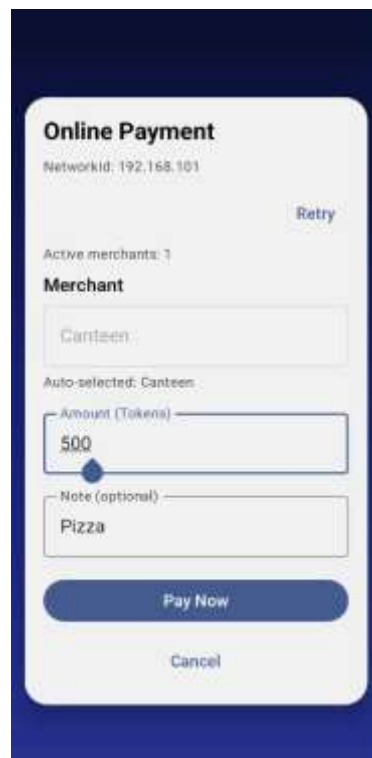


Figure 110: Canteen gets selected automatically



Figure 111: Payment Successful

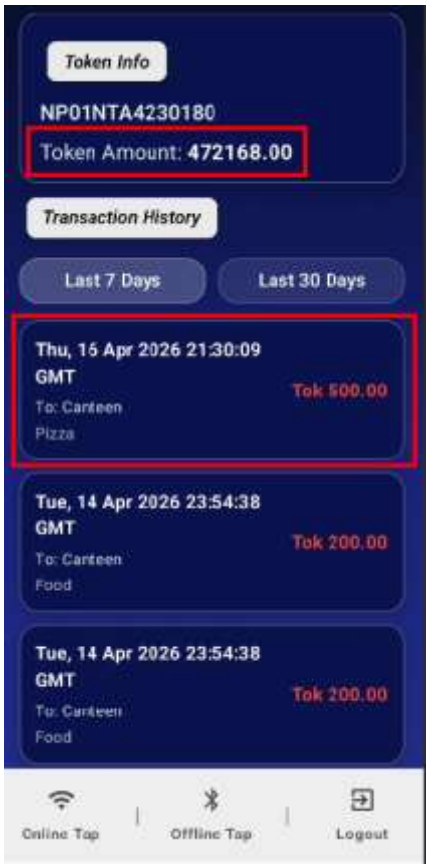


Figure 112: Student Transaction History

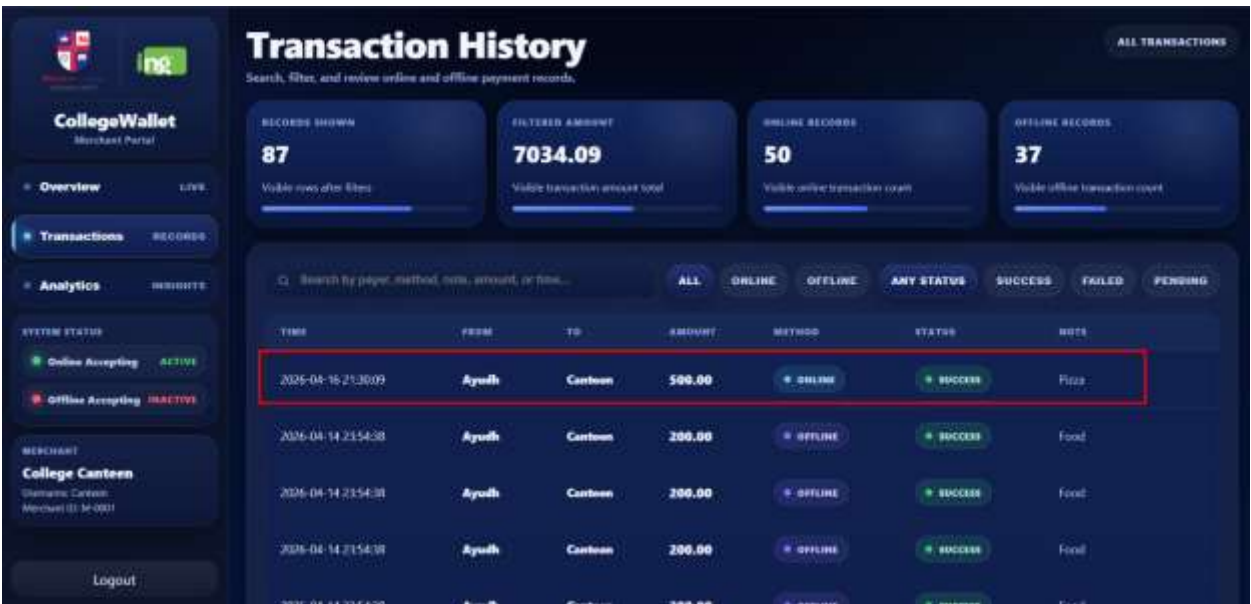


Figure 113: Canteen Transaction History

4.2.2. Test Case 2

Unit Testing	Objectives
Objective	To test the complete offline payment workflow from student login to successful offline payment, pending queue display on the merchant side, and final synchronization into merchant transaction history.
Action	The student logged in and completed the offline transaction. After that, pending queue was confirmed, payment was synced and transaction history was checked on both sides.
Expected Result	The complete online payment process should be performed successfully, and the completed transaction should be displayed correctly in both the student and the merchant transaction history.
Actual Result	The student logged in successfully, completed the online payment process, and the transaction was displayed correctly in both the student and the merchant transaction history.
Conclusion	Test successful.

Table 20: System Testing 2

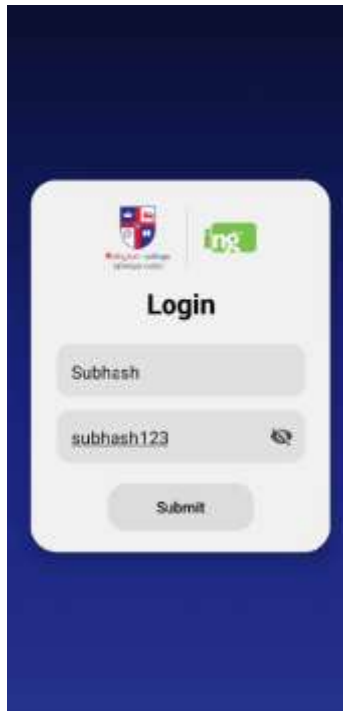
Evidence:

Figure 114: Adding valid login details



Figure 115: Login Successful

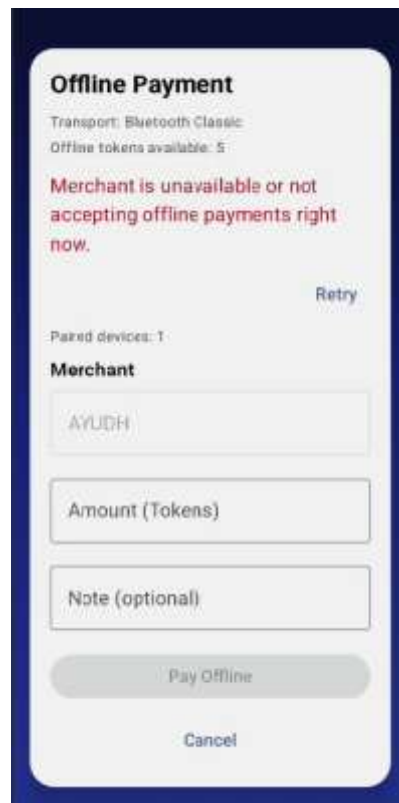


Figure 116: No one nearby accepting offline payment

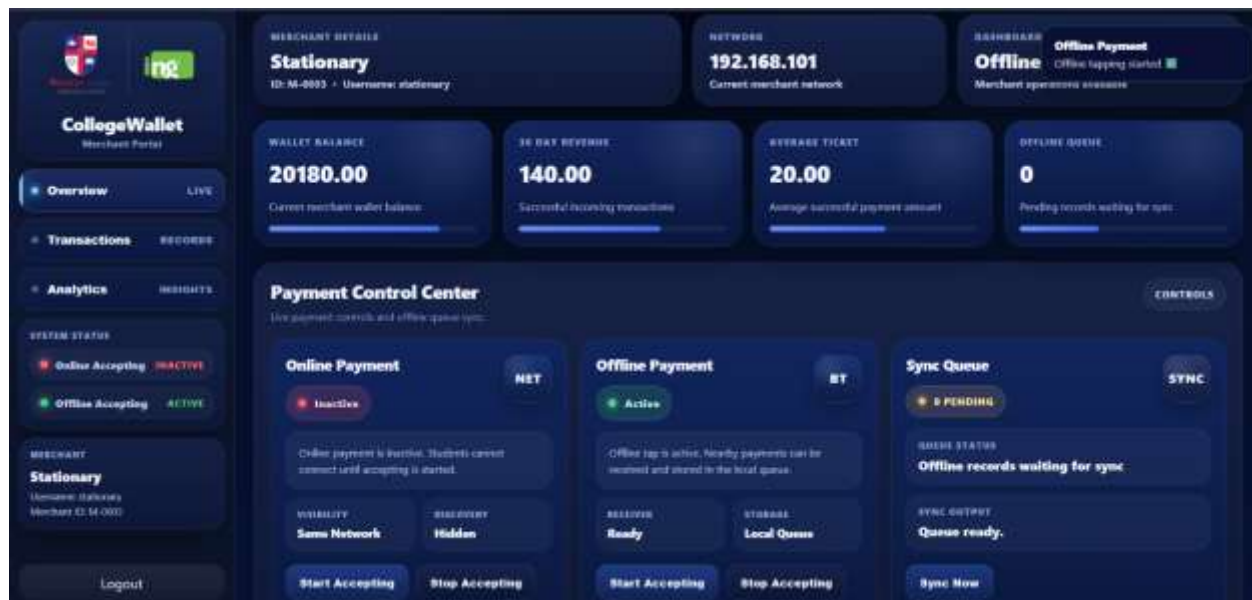


Figure 117: Stationary starts accepting offline payment



Figure 118: Stationary gets verified and selected automatically



Figure 119: Payment Successful



Figure 120: Student Transaction history

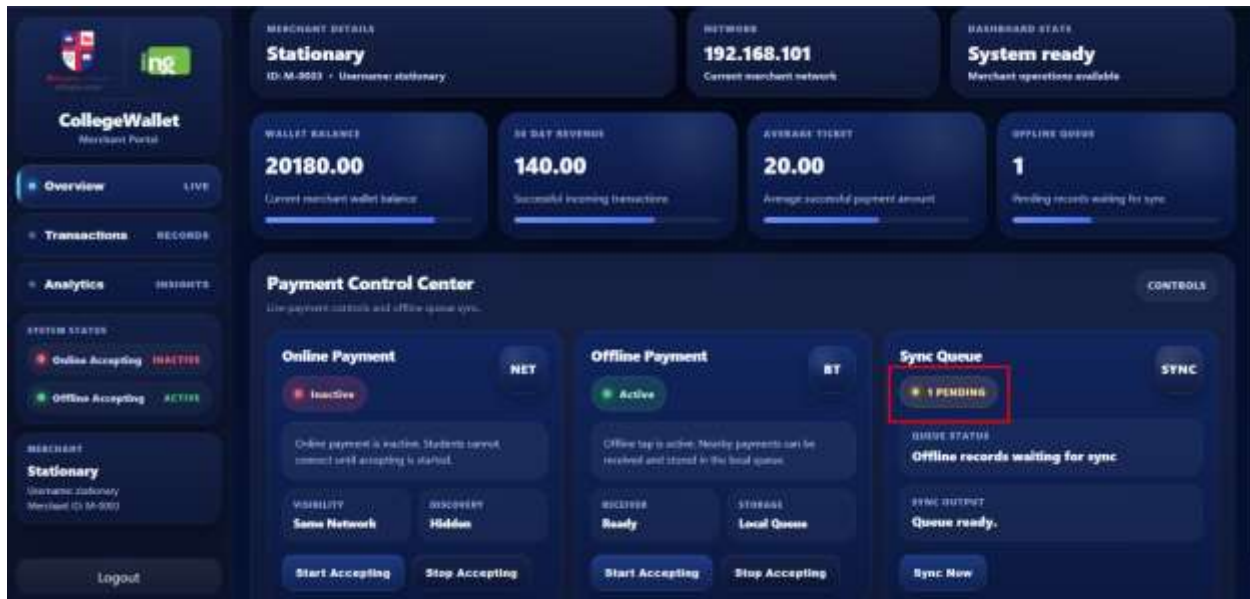


Figure 121: Payment pending in merchant web

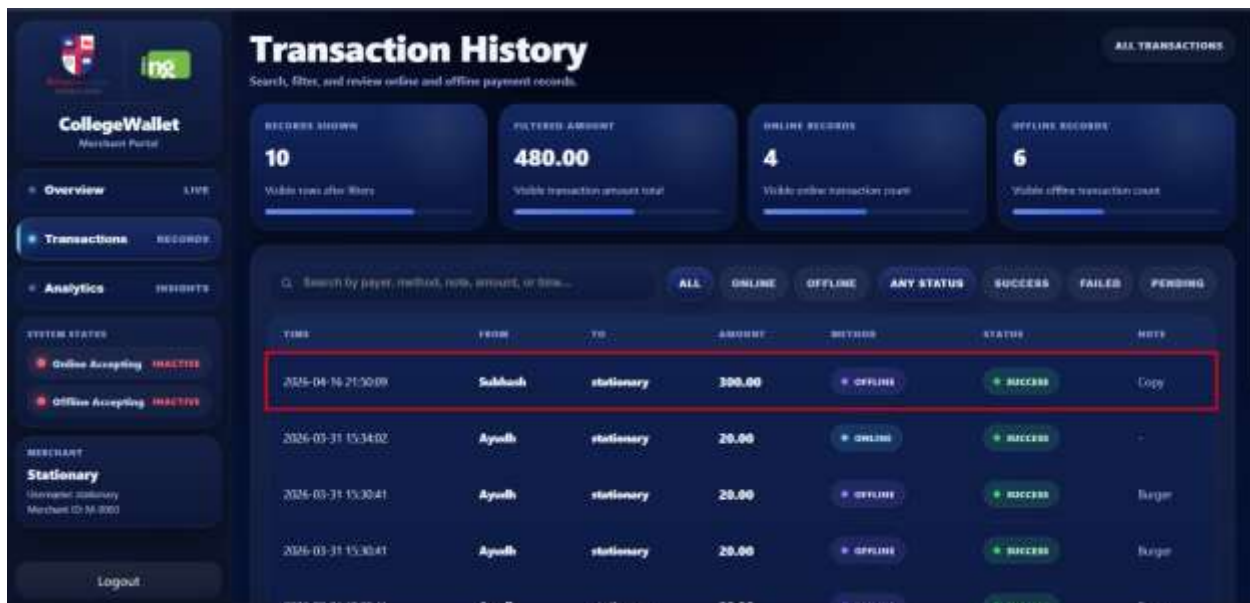


Figure 122: Transaction History after syncing

Note: This test was carried out using a different student account and a different merchant account for clearer demonstration.

4.3. Security Testing

Security testing was carried out to evaluate whether the College Credential Digital Wallet system could safeguard transactions and customer information from misappropriation, replay attacks, failure due to expiration, and poor storage security practices. The objective behind conducting security testing was to ensure that the security mechanisms were effective in ensuring proper transaction procedures both offline and online. Security testing in this project was performed to ensure rejection of replay token, rejection of expired tokens, encryption for offline token storage, and rejection of transactions with amounts above the balance.

The corresponding security mechanisms are described in [Section 3.7.5](#)

Test Plan:

Test Case	Objective
1	To test whether the system rejects the replay of an already used online payment token.
2	To test whether the system rejects the replay of an already used offline payment token during synchronization.
3	To test whether the system rejects an expired online payment token.
4	To test whether the system rejects an expired offline payment token during synchronization.
5	To test whether offline payment tokens are stored securely in encrypted form rather than plain readable form.
6	To test whether the system rejects an online payment attempt when the student tries to transfer an amount greater than the available balance.
7	To test whether the system rejects an offline payment attempt when the student tries to transfer an amount greater than the available balance or available offline token value.

Table 21: Security Testing Test Plan

4.3.1. Test Case 1

Unit Testing	Objectives
Objective	To test whether the system rejects replay of an already used online payment token.
Action	A successful online payment was first completed. After that, the same online token ID and signature were reused through the replay testing page.
Expected Result	The system should reject the reused online token, prevent duplicate processing, and return a failure response.
Actual Result	The replay attempt was rejected with HTTP 409 and the message "Token already used". No duplicate successful transaction was created.
Conclusion	Test successful.

Table 22: Security Testing 1

Evidence:

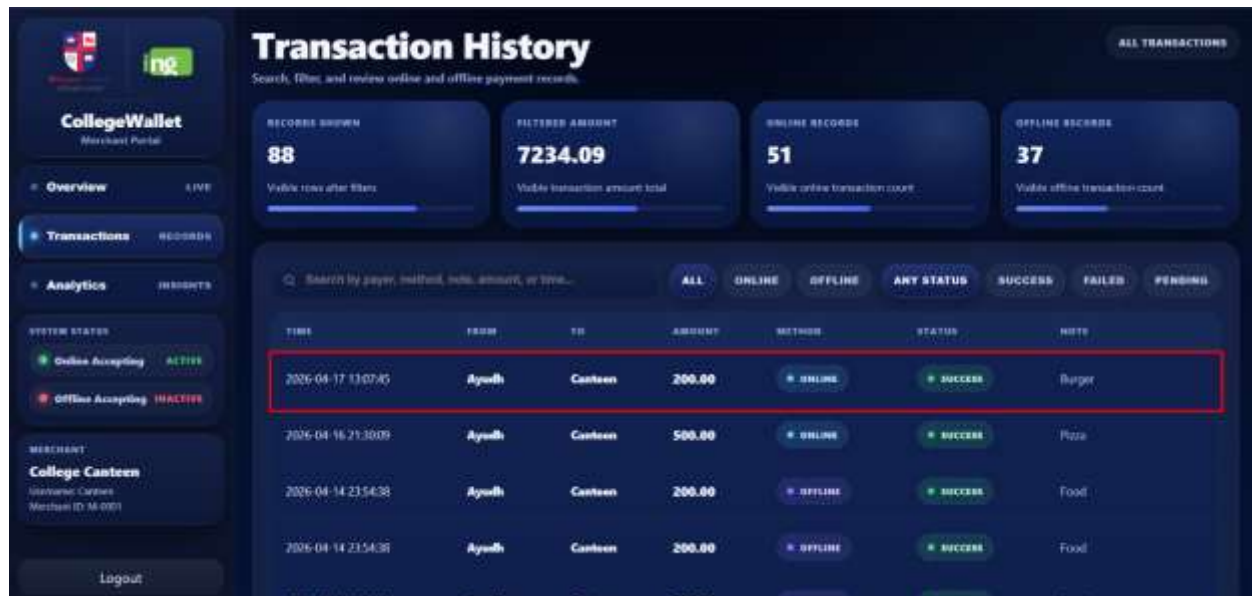


Figure 123: Successful Payment

Evidence Logs

Backend audit logs + offline receiver logs

Backend Audit Events

#	TIME	EVENT	OUTCOME	ACTOR	MERCHANT	SENDER	RECEIVER	AMOUNT	CLIENT ID	DETAILS
746	2020-04-17 13:06:30	merchant_checkin	SUCCESS	Carlson	Carlson	-	-	-	-	["network_id": "172.30.30.1", "ip": "172.30.30.1"]
745	2020-04-17 13:07:45	online_token_test	SUCCESS	-	Carlson	Ayudh	Carlson	200.00	14aa2ba40e4b4e3e8b37ef9d4d10964f	["transaction_id": 104, "note": "Burger", "ip": "172.30.30.1"]
744	2020-04-17 13:07:45	online_token_test	SUCCESS	-	Carlson	Ayudh	Carlson	200.00	14aa2ba40e4b4e3e8b37ef9d4d10964f	["expires_at": "2020-04-17T07:24:15Z", "ttl_seconds": 90, "ip": "172.30.30.1"]
743	2020-04-17 13:07:26	merchant_checkout	SUCCESS	Carlson	Carlson	-	-	-	-	["network_id": "172.30.30.1", "ip": "172.30.30.1"]
742	2020-04-17 13:07:10	merchant_checkout	SUCCESS	Carlson	Carlson	-	-	-	-	["ip": "172.30.30.1"]
741	2020-04-17 13:07:08	student_login	SUCCESS	ayudh	-	-	-	-	-	["ip": "172.30.30.1"]
740	2020-04-17 13:07:05	merchant_checkout	SUCCESS	Carlson	Carlson	-	-	-	-	["network_id": "172.30.30.1", "ip": "172.30.30.1"]
739	2020-04-17 13:06:38	merchant_checkout	SUCCESS	Carlson	Carlson	-	-	-	-	["network_id": "172.30.30.1", "ip": "172.30.30.1"]
738	2020-04-17 13:06:08	merchant_checkout	SUCCESS	Carlson	Carlson	-	-	-	-	["network_id": "172.30.30.1", "ip": "172.30.30.1"]
737	2020-04-17 13:05:35	merchant_checkout	SUCCESS	Carlson	Carlson	-	-	-	-	["network_id": "172.30.30.1", "ip": "172.30.30.1"]

Figure 124: Online Payment log with details

Replay Token Test

This page replays the exact same token payload back to ipayonline tokenized using your browser.

TOKEN ID 14aa2ba40e4b4e3e8b37ef9d4d10964f	CURRENT TOKEN STATUS USED
SENDER Ayudh	MERCHANT Carlson
AMOUNT 200.0	TRANSACTION ID 104

Replay Payload

```
{
  "token_id": "14aa2ba40e4b4e3e8b37ef9d4d10964f",
  "sender": "14aa2ba40e4b4e3e8b37ef9d4d10964f",
  "expires_at": "2020-04-17T07:24:15Z",
  "signature": "3c19b388c77c0b4e9727f1ba4d8c943205819f13c7a1b53fabc9d5198a7ba"
}
```

[Replay Same Token](#)

After replay, refresh: [http://localhost:3000/14aa2ba40e4b4e3e8b37ef9d4d10964f](#)

Figure 125: Same token used for testing

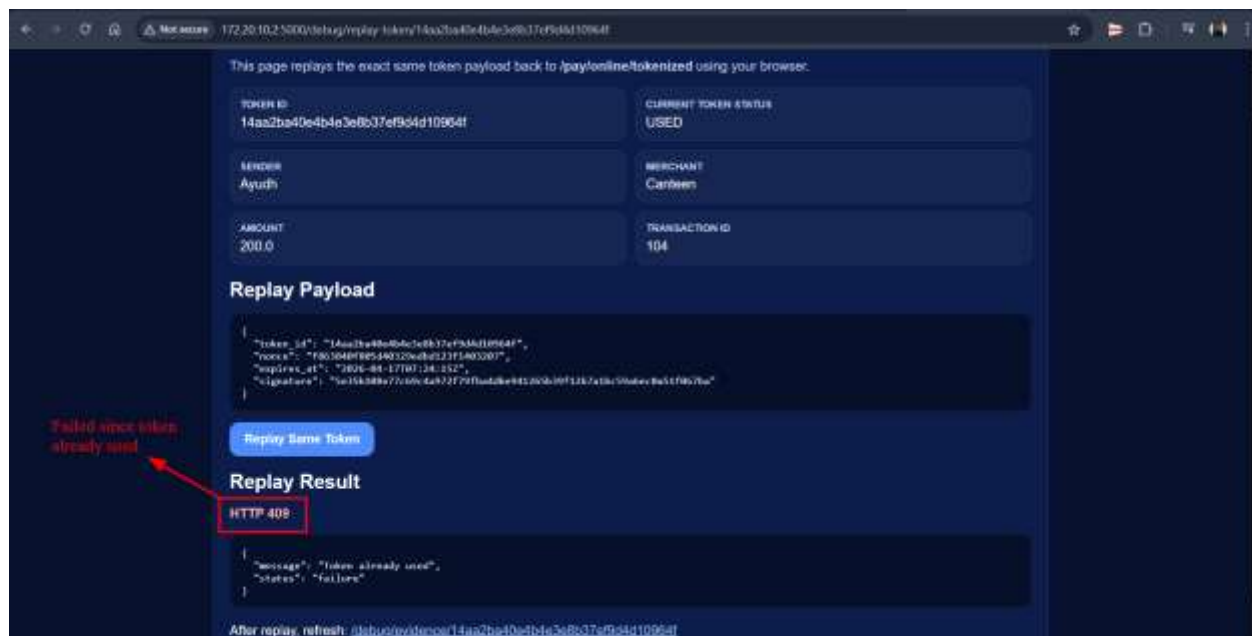


Figure 126: Token replay failed since it was already used

4.3.2. Test Case 2

Unit Testing	Objectives
Objective	To test whether the system rejects replay of an already used offline payment token during synchronization.
Action	A successful offline payment was first completed and synchronized. After that, the same offline token ID and original client transaction details were reused through the offline replay testing page.
Expected Result	The system should reject the reused offline token during synchronization and should not create a duplicate successful transaction.
Actual Result	The replay synchronization process completed, but the payment result was marked as FAILED with the message “Offline token already used”. No duplicate successful offline transaction was added.
Conclusion	Test successful.

Table 23: Security Testing 2

Evidence:

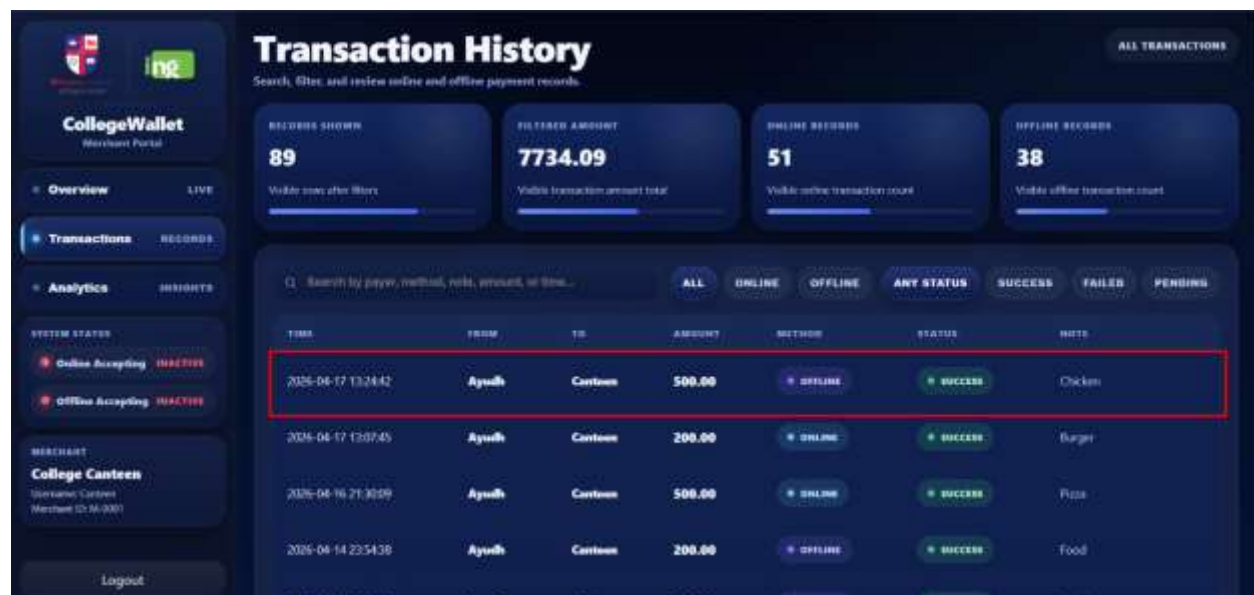
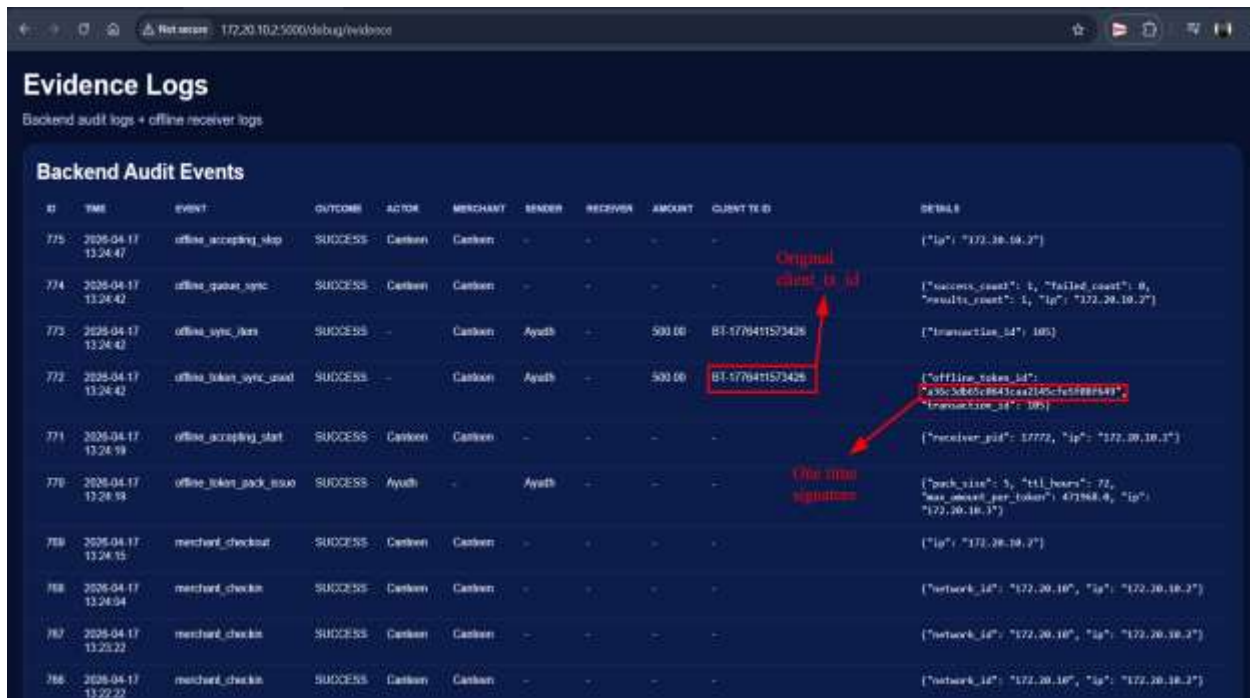


Figure 127: Successful Offline Payment

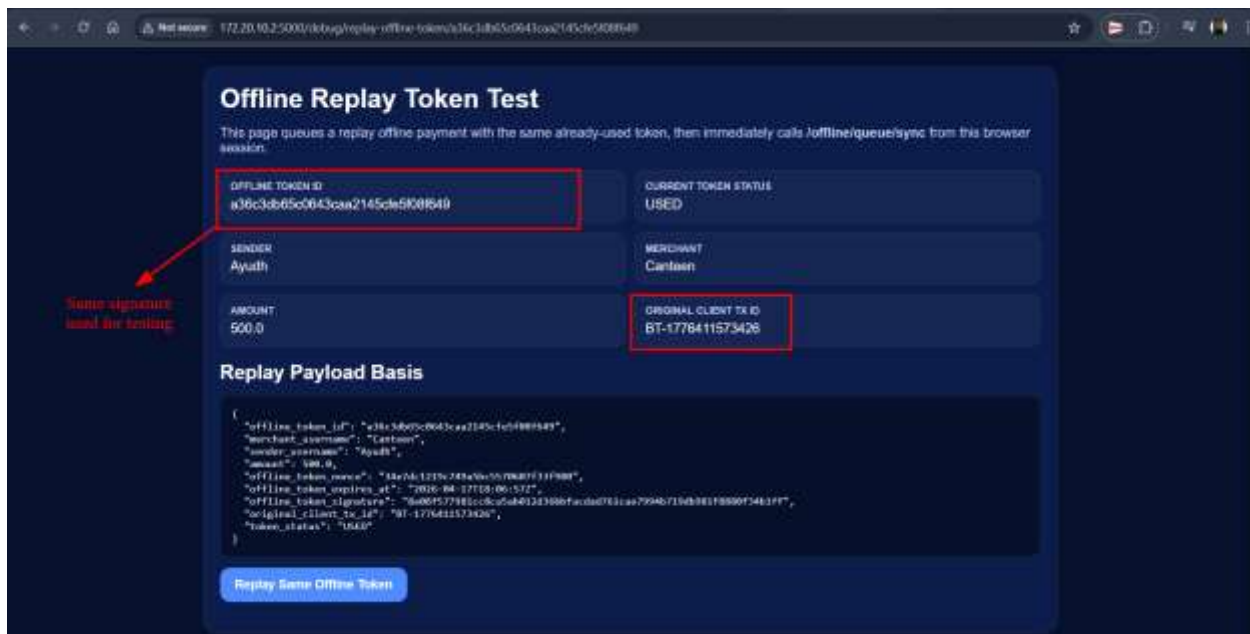


Evidence Logs
Backend audit logs + offline receiver logs

Backend Audit Events

ID	TIME	EVENT	OUTCOME	ACTION	MERCHANT	SENDER	RECEIVER	AMOUNT	CLIENT TX ID	DETAILS
775	2026-04-17 13:24:47	offline_scraping_start	SUCCESS	Cartoon	Cartoon	-	-	-	-	{"ip": "172.20.10.2"}
774	2026-04-17 13:24:42	offline_queue_sync	SUCCESS	Cartoon	Cartoon	-	-	-	-	{"success_count": 1, "failed_count": 0, "results_count": 1, "ip": "172.20.10.2"}
773	2026-04-17 13:24:42	offline_token_sync	SUCCESS	-	Cartoon	Ayudh	-	500.00	BT-1776411573426	["transaction_id": 105]
772	2026-04-17 13:24:42	offline_token_sync_used	SUCCESS	-	Cartoon	Ayudh	-	500.00	BT-1776411573426	{"offline_token_id": "a36c3b65c0643caa2145cfe588f549", "transaction_id": 105}
771	2026-04-17 13:24:39	offline_scraping_start	SUCCESS	Cartoon	Cartoon	-	-	-	-	["receiver_id": 17772, "ip": "172.20.10.2"]
770	2026-04-17 13:24:39	offline_token_sync_used	SUCCESS	Ayudh	-	Ayudh	-	-	-	["back_time": 5, "ttl_hours": 72, "max_amount_per_token": 47184.0, "ip": "172.20.10.2"]
769	2026-04-17 13:24:35	merchant_checkout	SUCCESS	Cartoon	Cartoon	-	-	-	-	["ip": "172.20.10.2"]
768	2026-04-17 13:24:34	merchant_checkout	SUCCESS	Cartoon	Cartoon	-	-	-	-	["network_id": "172.20.10", "ip": "172.20.10.2"]
767	2026-04-17 13:23:22	merchant_checkout	SUCCESS	Cartoon	Cartoon	-	-	-	-	["network_id": "172.20.10", "ip": "172.20.10.2"]
766	2026-04-17 13:22:22	merchant_checkout	SUCCESS	Cartoon	Cartoon	-	-	-	-	["network_id": "172.20.10", "ip": "172.20.10.2"]

Figure 128: Offline Payment logs with details



Offline Replay Token Test
This page queues a replay offline payment with the same already-used token, then immediately calls /offline/queue/sync from this browser session.

Offline Token ID: a36c3b65c0643caa2145cfe588f549

Current Token Status: USED

Sender: Ayudh

Merchant: Cartoon

Amount: 500.0

Original Client TX ID: BT-1776411573426

Replay Payload Basis

```
{
  "offline_token_id": "a36c3b65c0643caa2145cfe588f549",
  "merchant_username": "Cartoon",
  "sender_username": "Ayudh",
  "amount": 500.0,
  "offline_token_nonce": "34e3d121e248a9e5570a07f33f9a0",
  "offline_token_expiry_id": "7036-04-17T02:00:57Z",
  "offline_token_signature": "5d0d577981c0c0c0b412d30b6fanda07e31ca7994b719d3e1f800974817f7",
  "original_client_tx_id": "BT-1776411573426",
  "token_status": "USED"
}
```

Replay Same Offline Token

Figure 129: Same token used for testing

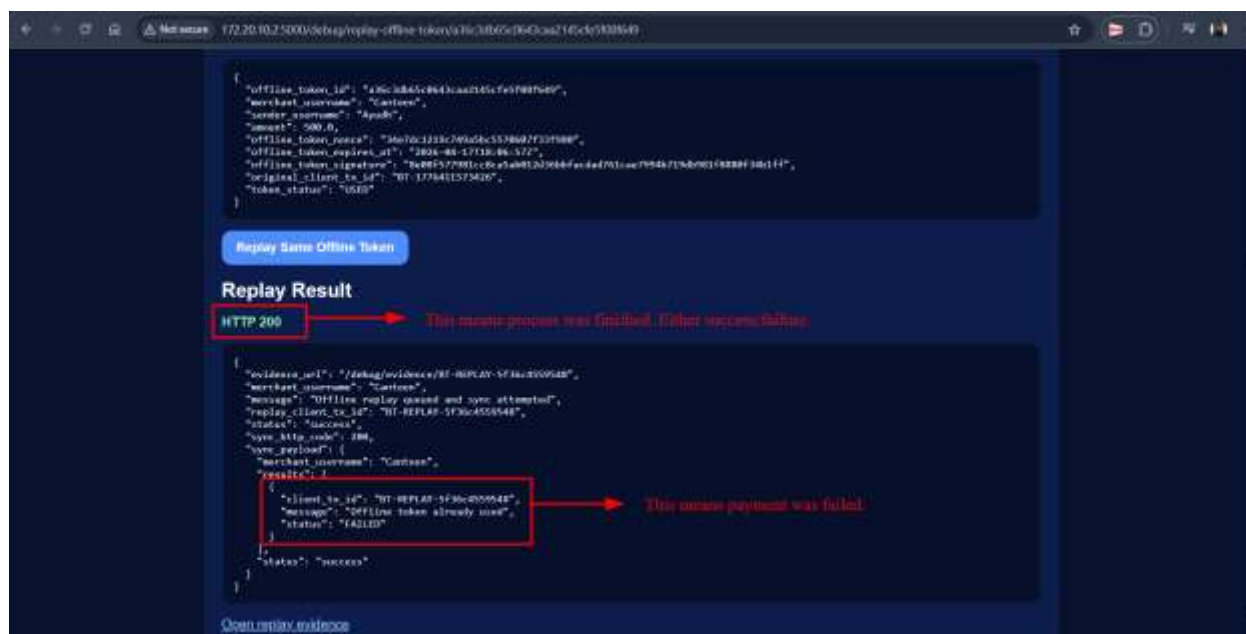


Figure 130: Token replay failed since it was already used

4.3.3. Test Case 3

Unit Testing	Objectives
Objective	To test whether the system rejects an expired online payment token.
Action	The online expired-token testing page was opened using an online token, and the token was prepared for forced expiry and retesting.
Expected Result	The system should reject the expired online token and return a failure response when the same expired token is submitted.
Actual Result	The system returned HTTP 400 and rejected the payment attempt with the message “Token already expired. Current Status: USED.”
Conclusion	Test successful.

Table 24: Security Testing 3

Evidence:

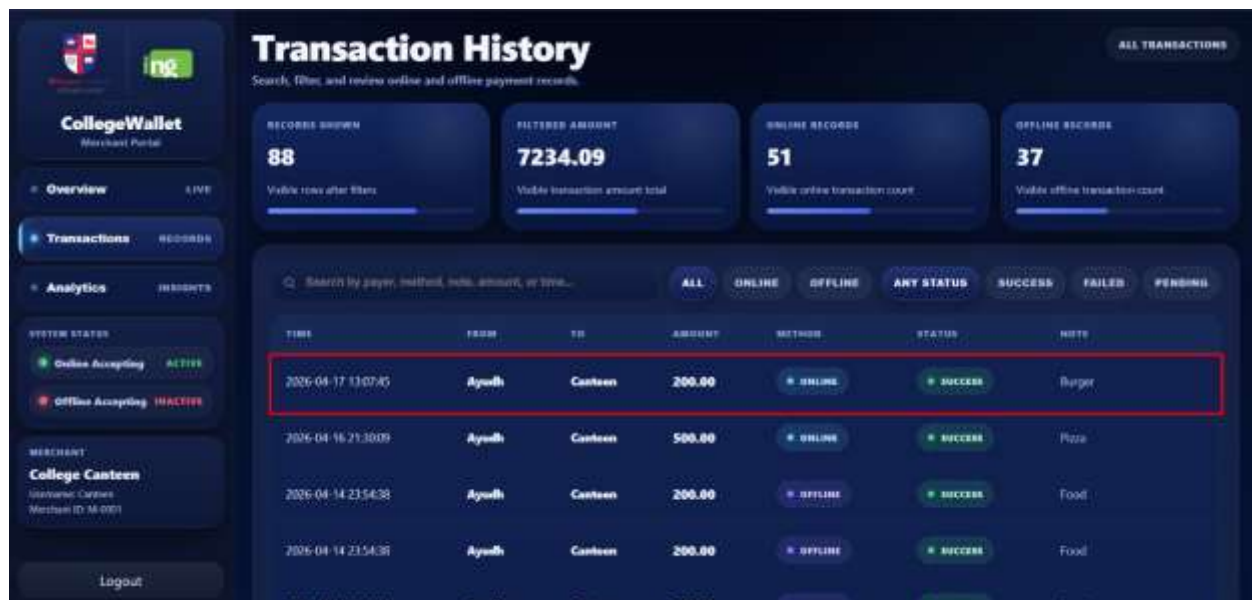


Figure 131: Successful Online Payment

Evidence Logs

Backend audit logs + offline receiver logs

Backend Audit Events

ID	TIME	EVENT	OUTCOME	ACTOR	MERCHANT	SENDER	RECEIVER	AMOUNT	CLIENT ID	DETAILS
746	2020-04-17 13:06:20	merchant_checkin	SUCCESS	Carlsen	Carlsen	-	-	-	-	[{"network_id": "172.30.30", "ip": "172.30.30.2"}]
745	2020-04-17 13:07:45	online_token_used	SUCCESS	-	Carlsen	Ayudh	Carlsen	200.00	14aa2ba40e4b4e3e8537ef854d10964f	[{"transaction_id": 184, "note": "Burger", "ip": "172.30.30.2"}]
744	2020-04-17 13:07:45	online_token_receive	SUCCESS	-	Carlsen	Ayudh	Carlsen	200.00	14aa2ba40e4b4e3e8537ef854d10964f	[{"expires_at": "2020-04-17T07:24:15Z", "ttl_seconds": 90, "ip": "172.30.30.2"}]
743	2020-04-17 13:07:26	merchant_checkin	SUCCESS	Carlsen	Carlsen	-	-	-	-	[{"network_id": "172.30.30", "ip": "172.30.30.2"}]
742	2020-04-17 13:07:10	merchant_checkout	SUCCESS	Carlsen	Carlsen	-	-	-	-	[{"ip": "172.30.30.2"}]
741	2020-04-17 13:07:08	student_login	SUCCESS	ayudh	-	-	-	-	-	[{"ip": "172.30.30.1"}]
740	2020-04-17 13:07:06	merchant_checkin	SUCCESS	Carlsen	Carlsen	-	-	-	-	[{"network_id": "172.30.30", "ip": "172.30.30.2"}]
739	2020-04-17 13:06:30	merchant_checkin	SUCCESS	Carlsen	Carlsen	-	-	-	-	[{"network_id": "172.30.30", "ip": "172.30.30.2"}]
738	2020-04-17 13:06:00	merchant_checkin	SUCCESS	Carlsen	Carlsen	-	-	-	-	[{"network_id": "172.30.30", "ip": "172.30.30.2"}]
737	2020-04-17 13:05:30	merchant_checkin	SUCCESS	Carlsen	Carlsen	-	-	-	-	[{"network_id": "172.30.30", "ip": "172.30.30.2"}]

Note: A red box highlights the token ID '14aa2ba40e4b4e3e8537ef854d10964f' in the 'online_token_used' event, with an arrow pointing to the 'Online Expired Token Test' page in Figure 133.

Figure 132: Payment logs with details

Online Expired Token Test

This page forces the selected online token to expire in the backend, then sends the same token payload to `/payonline/tokenized`.

Use this only on a fresh test token that has not been used yet.

TOKEN ID 14aa2ba40e4b4e3e8537ef854d10964f	CURRENT STATUS USED
SENDER Ayudh	MERCHANT Carlsen
AMOUNT 200.0	CURRENT EXPIRES AT 2020-04-17T07:24:15Z

Payload That Will Be Sent

```
{
  "token_id": "14aa2ba40e4b4e3e8537ef854d10964f",
  "nonce": "f803d48f865480329e8d123f1403287",
  "expires_at": "2020-04-17T07:24:15Z",
  "signature": "5e3b385e7f0b54e7727f0f8a8b8c113b59f12b7a3b0b453f8a7ba8"
}
```

Force Expire + Test Consume

Note: A red box highlights the token ID '14aa2ba40e4b4e3e8537ef854d10964f' in the 'TOKEN ID' field, with an arrow pointing to the 'Evidence Logs' table in Figure 132.

Figure 133: Using same token id for expiration test

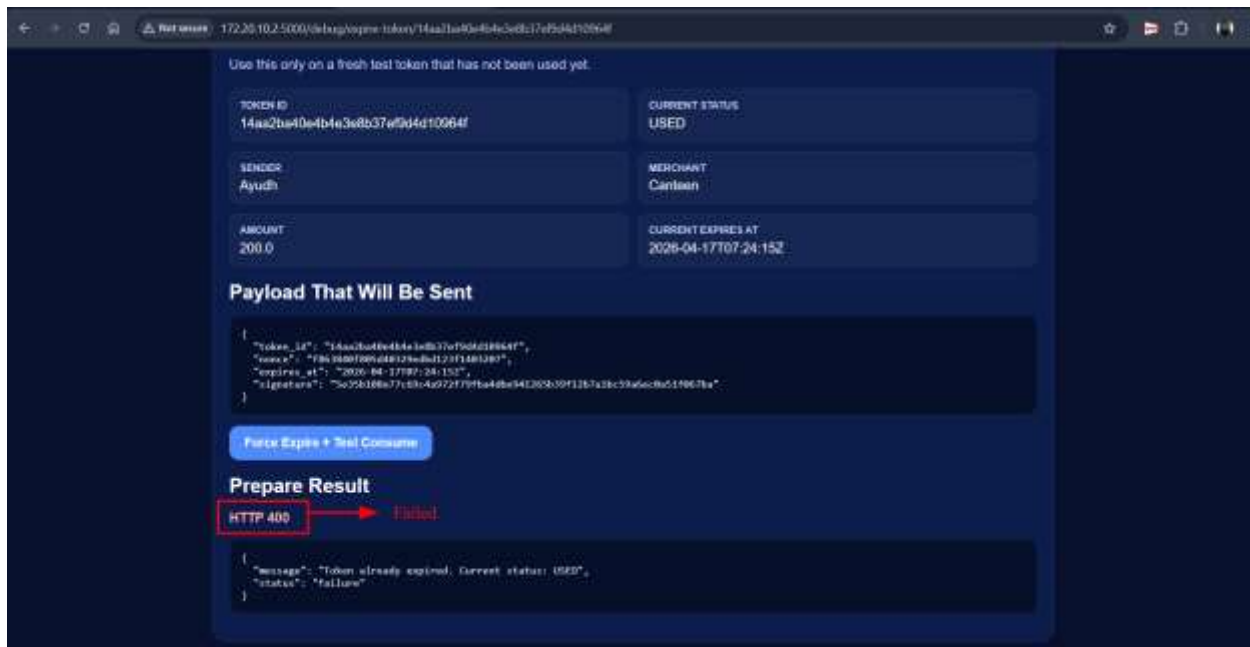


Figure 134: Failed as token got expired

4.3.4. Test Case 4

Unit Testing	Objectives
Objective	To test whether the system rejects an expired offline payment token during synchronization.
Action	The offline expired-token testing page was opened using an offline token, and the token was prepared for forced expiry, queueing, and synchronization.
Expected Result	The system should reject the expired offline token during synchronization and should not allow the payment to be processed successfully.
Actual Result	The system returned HTTP 400 and rejected the offline synchronization attempt with the message “Token already expired. Current status: USED.”
Conclusion	Test successful.

Table 25: Security Testing 4

Evidence:

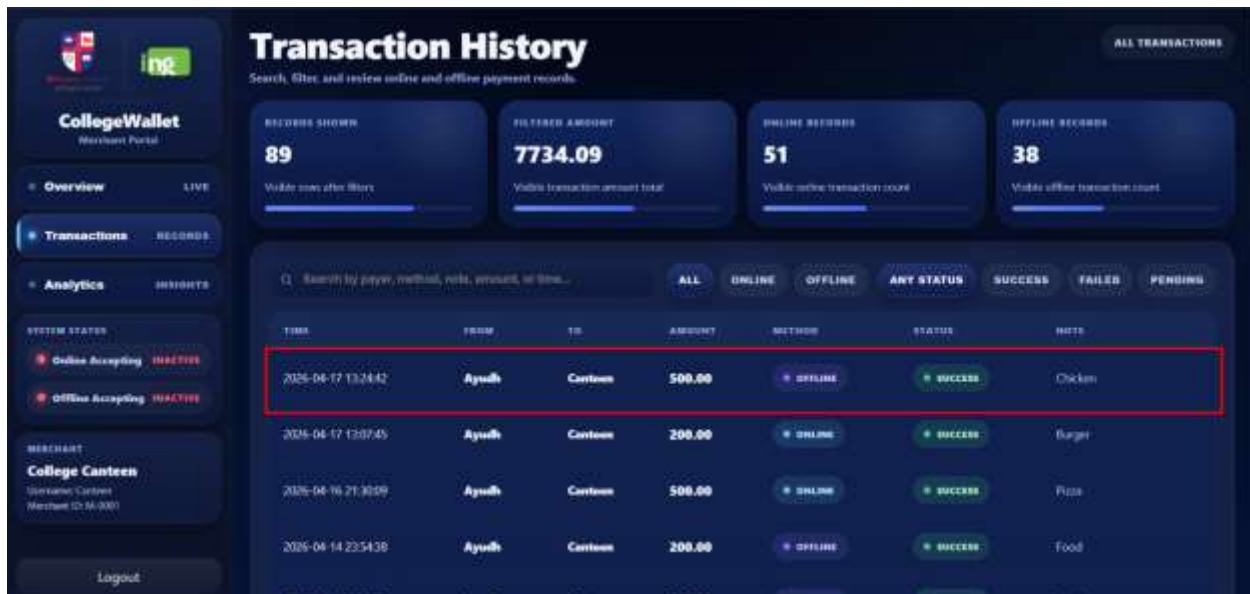


Figure 135: Successful Offline Payment

Figure 136: Payment logs with details

Figure 137: Using same token id for expiration test

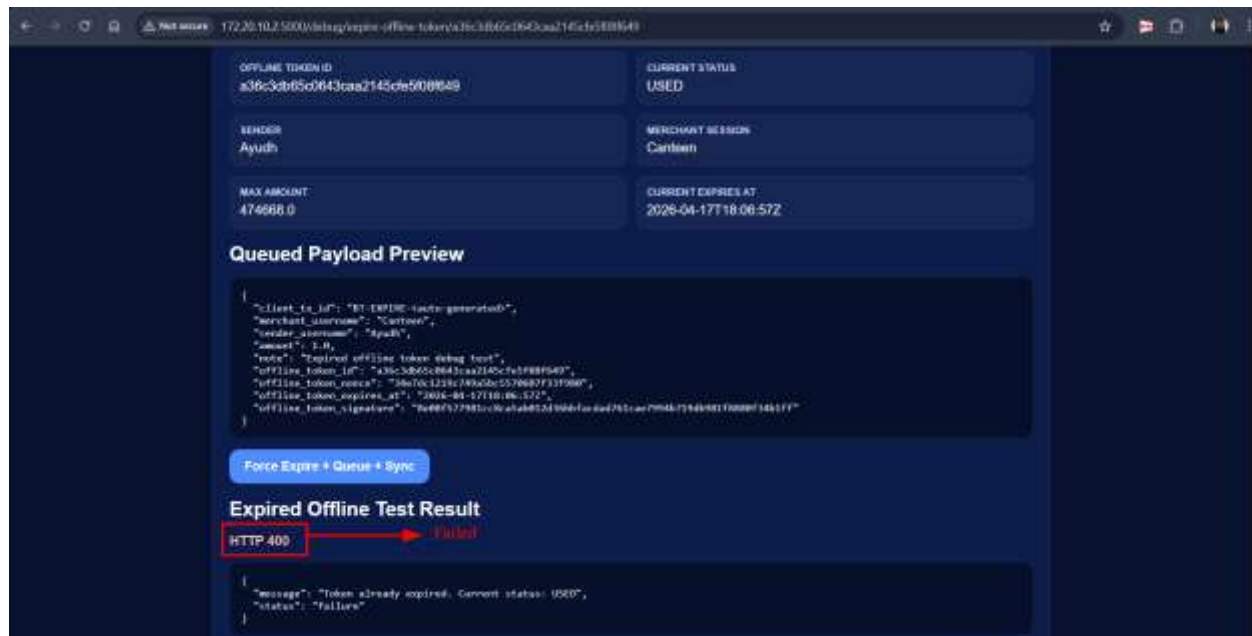


Figure 138: Offline payment failed since token was expired

4.3.5. Test Case 5

Unit Testing	Objectives
Objective	To test whether offline tokens are stored securely in encrypted form instead of plain raw data.
Action	The normal offline token storage file and the secure offline token storage file were inspected and compared.
Expected Result	Sensitive offline token data should not be stored in readable plain text in secure storage. Instead, it should appear in encrypted form.
Actual Result	The normal storage file showed readable raw token data, whereas the secure storage file showed encrypted values, confirming that secure token storage was applied.
Conclusion	Test successful.

Table 26: Security Testing 5

Evidence:

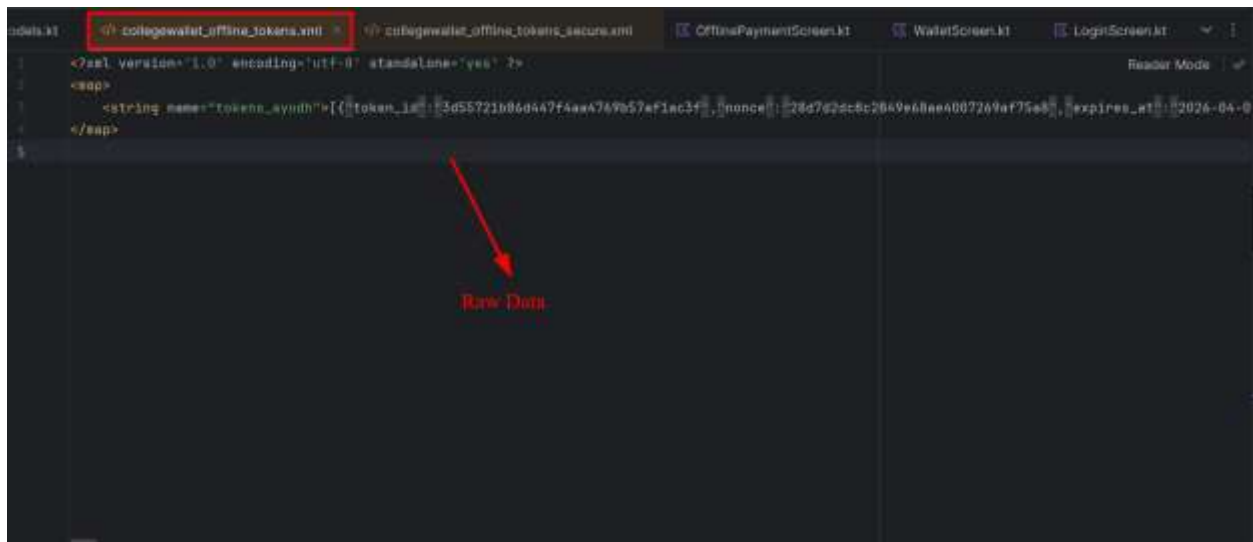


Figure 139: Android package normal file showing raw data

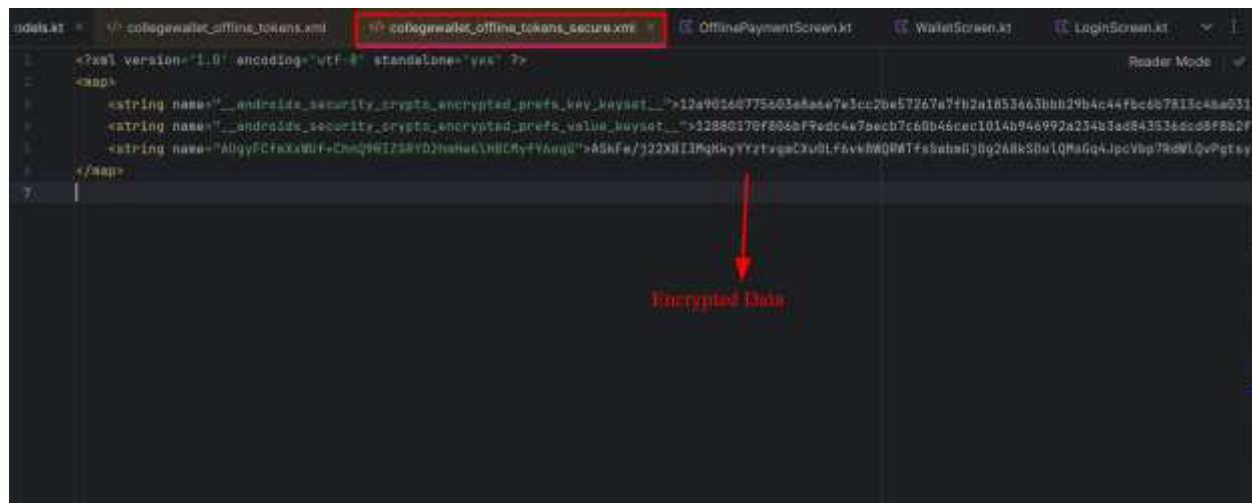


Figure 140: Android package secure offline token file showing encrypted token

Note: The evidence shows that offline token data stored without security is readable in plain form, whereas the secured version stores the same data in encrypted form, confirming proper protection of sensitive token information.

4.3.6. Test Case 6

Unit Testing	Objectives
Objective	To test whether the system rejects an online payment attempt when the student tries to transfer an amount greater than the available balance.
Action	The student wallet balance was first checked, and then an online payment was attempted using an amount greater than the available balance.
Expected Result	The system should reject the payment and display an insufficient balance message.
Actual Result	The system rejected the online payment attempt and displayed an insufficient balance message.
Conclusion	Test successful.

Table 27: Security Testing 6

Evidence:



Figure 141: Initial balance of the user

The screenshot displays a mobile application interface for online payments. At the top, the title 'Online Payment' is shown in bold. Below it, the 'NetworkId' is listed as '172.20.10'. A 'Retry' button is positioned to the right. The status 'Active merchants: 1' is displayed. Under the 'Merchant' section, a text box contains 'Canteen', and below it, 'Auto-selected: Canteen' is shown. The 'Amount (Tokens)' field is set to '5000000000'. The 'Note (optional)' field contains the word 'Food'. A prominent blue 'Pay Now' button is centered. At the bottom, a white pill-shaped button with a green icon and the text 'Insufficient balance' indicates the payment failure.

Figure 142: Insufficient balance for online payment

4.3.7. Test Case 7

Unit Testing	Objectives
Objective	To test whether the system rejects an offline payment attempt when the student tries to transfer an amount greater than the available balance.
Action	The student wallet balance was first checked, and then an offline payment was attempted using an amount greater than the available balance.
Expected Result	The system should reject the payment and display an insufficient balance message.
Actual Result	The system rejected the offline payment attempt and displayed an insufficient balance message.
Conclusion	Test successful.

Table 28: Security Testing 7

Evidence:



Figure 143: Initial User balance

The image shows a mobile application interface for an offline payment system. The screen has a dark blue background with a white rounded rectangle in the center. At the top, the title "Offline Payment" is displayed in bold. Below it, the transport method is "Bluetooth Classic" and the number of offline tokens available is 64. A green checkmark indicates the merchant is verified. A "Retry" button is located to the right. Below this, it shows "Paired devices: 1". The "Merchant" section has a text input field containing "Canteen". The "Amount (Tokens)" input field contains "80000000". The "Note (optional)" input field contains "Food". A blue "Pay Offline" button is at the bottom. A white error message bubble with a green icon and the text "Insufficient balance" is overlaid on the bottom of the screen.

Offline Payment

Transport: Bluetooth Classic
Offline tokens available: 64

Merchant verified ✓

Retry

Paired devices: 1

Merchant

Canteen

Amount (Tokens)
80000000

Note (optional)
Food

Pay Offline

Insufficient balance

Figure 144: Insufficient balance for offline payment

4.4. Critical Analysis

4.4.1. Project Deliverables Evaluation

The project was able to deliver the key elements of the final system. The digital identity, wallet balance, transaction history, and online and offline payment support were added to a student Android application. Moreover, a backend server, MySQL database integration, merchant web interface, offline synchronization mechanism, and testing evidence were also realized as a part of the project. Security features like replay denial, expiry verification, password hashing, and encrypted local storage added additional technical value to the final prototype.

In addition, some design decisions evolved between the interim and final stages, meaning that several diagrams and planned flows had to be revised to match the actual implementation. Overall, the project fulfilled its primary objective of integrating digital credential access and secure campus payment functionalities in a single system.

Supporting technical terms and extended implementation details are provided in [Appendix 9.1](#) and [Appendix 9.3](#).

4.4.2. Limitations and Challenges

Despite the successful implementation, the project still has some limitations. The system was designed and was experimented in a small academic setting and tested with a limited number of devices and primarily manual testing. Consequently, it was not tested in large-scale real-world conditions of deployment.

The other difficulty was the integration of various modules like android development, backend APIs, database management, Bluetooth communication, offline queuing, synchronization and security managing in a single project. The offline payment was particularly difficult considering that it meant that there had to be good storage of the transactions and subsequent synchronization as well as discouraging abuse. Thus, the project is successful as a final year prototype but still more work would be required before actual institutional implementation.

5. Conclusion and Future Works

5.1. Legal, Social and Ethical Issues

5.1.1. Legal Issues

The College Digital Credential Wallet system being proposed requires dealing with the personal identity information of students, wallet balances, and transaction history, so the legal compliance is a significant factor to consider. As the system is the storage of personal and financial-related information, the implementation should be accompanied by an adequate access control, a secure authenticated system, and sound data management. Any actual deployment would have to be in a manner that information of students is gathered regarding valid institutional reasons and not unlawfully revealed. Moreover, the records of transactions and audit logs are to be kept neatly to foster accountability and dispute resolution. Thus, the current project being a prototype, its real implementation would have to be conducted in line with corresponding data protection and institutional policies on ICT.

5.1.2. Social Issues

Socially, the system can lead to increased convenience and efficiency among students and merchants in the college setup. It integrates the features of digital identity and cashless payments into one platform, eliminating the necessity to have many items, including physical ID cards and cash. This will help to save time when verifying the attendance, making payments in the canteen and other campus activities. However, availability, usability and confidence in digital systems are also important to social acceptance. Not all users will be willing to use a digital wallet at first because they are not familiar with it, are used to relying on mobile devices, or they fear technical malfunctions. Thus, awareness, usability and user confidence play a significant role in the effective adoption of the system.

5.1.3. Ethical Issues

Ethically, the project should guarantee the privacy of the user, fairness, and security at any given time. The system should not disclose any sensitive data and must not be used to abuse the information in transactions because it handles student credentials and payment activity. It is also necessary to be transparent in terms of what data is kept, how it is utilized, and who is allowed to access it. The system should not discriminate users because of technical constraints or lack of digital literacy or temporary connectivity problems. This is tried to be addressed by the project which supports online, and offline payments flows and provides security measures like token validation, expiry control, replay protection and encrypted local storage. These characteristics contribute towards a less abusive system design and minimize the possibility of abuse.

5.2. Advantages:

The created system offers several feasible benefits in a college set-up as listed below:

- It integrates digital identity and wallet-based payment into one application, enhancing convenience among the students.
- It allows online and offline payments, which increases the flexibility of the system in the cases when the internet connection is weak or unstable.
- Merchant web portal enables vendors to control the activity of payment, records of transactions, and synchronization through a simple interface based on the browser.
- The security features like signed tokens, validation of expiry, rejection of replay, audit logging, and encrypted storage of offline tokens enhance trust and reliability of the system.
- Lastly, the project shows that a campus-based digital wallet can be introduced with the help of practical and available technologies and retain a structured testing and security control.

5.3. Future Work

To improve and expand the system, various additions can be done in the future. Some of the future works are listed below:

- Biometric Authentication Integration
- Real-Time Notification System
- Institution-Wide Multi-Merchant Expansion
- Advanced Administrative Dashboard
- Backup and recovery based on clouds.
- Cross-Platform Application Support

These areas of work in the future can pinpoint the potential directions in which the prototype can be extended into a more comprehensive and production-ready campus solution.

A brief explanation of each proposed future enhancement is provided in [Appendix 9.4](#).

5.4. Conclusion

This project was able to design and develop a prototype College Digital Credential Wallet that incorporates digital student identity and token-based payment services into one system. The implementation that will be completed is student Android application, Flask back-end, merchant web portal, central database support, offline transaction queue, and various security to assist in safeguarding the transactions. Among the most important works of the project is that they have both online and Bluetooth Classic-based offline payment methods that make the system more realistic in a real-college setting, where the connection is not always guaranteed.

Another lesson learnt in the project was the need to balance usability, security, and implementation feasibility. By the method of structured development, repeated testing, and incremental refinements, the final prototype could meet its significant functional goals. The results of the testing revealed that the important features like login, access to digital ID, online payment, offline payment, synchronization, analytics, replay rejection, expiry validation, and encrypted token processing were all established.

To sum up, the project fulfilled its primary objective of creating a viable and secure college-centred digital wallet prototype. Even though some more implementation can be done to expand the deployment, the system offers a robust premise on future expansion and illustrates a significant implementation of secure digital payment and identity technologies within an educational setting.

8. References

- Drumond, C., 2018. *What is Scrum? Breaking down the Agile framework*. [Online] Available at: <https://www.atlassian.com/agile/scrum> [Accessed 28 11 2025].
- Frank, D., 2025. This is How to Conquer Software Projects Using the Waterfall Model. *Level Up Coding*, Issue 2025, p. 12.
- Geeksforgeeks, 2023. *Waterfall Model - Software Engineering*. [Online] Available at: <https://www.geeksforgeeks.org/software-engineering/waterfall-model/> [Accessed 29 11 2025].
- IEEE, 2004. *Digital Identity*. [Online] Available at: <https://ieeexplore.ieee.org/abstract/document/1337889/> [Accessed 28 11 2025].
- IEEE, 2025. The Rise and Evolution of Agile Software Development. *The Rise and Evolution of Agile Software Development*, Issue 2018.
- Jakobsson, M., 2001. *Security Weaknesses in Bluetooth*. s.l., RSA Conference 2001.
- Kieseberg, P., 2014. *QR Code Security: A Survey of Attacks and Challenges for Usable Security*. Austria, ResearchGate.
- Lipapayments, 2021. *Lipapayments*. [Online] Available at: <https://www.lipapayments.com/>
- Market Research Future, 2025. *Digital Identity Solution Market*. [Online] Available at: <https://www.marketresearchfuture.com/reports/digital-identity-solution-market-30294> [Accessed 28 11 2025].
- More, J., 2023. Prototyping Model in Software Development Life Cycle (SDLC): Phases, Advantages & Disadvantages. *Software Development Life Cycle*, Issue 2023.
- nvisia, 2020. *nvisia*. [Online] Available at: <https://www.nvisia.com/insights/agile-methodology>
- Relia, 2023. *Prototype Model in Software Engineering: Types, Phases, Use Cases*. [Online] Available at: <https://reliasoftware.com/blog/prototype-model-in-software-engineering> [Accessed 29 11 2025].

Tappit, 2021. *Tappit*. [Online]
Available at: <https://tappit.com/>
[Accessed 2025].

TouchNet, 2023. *OneCard Campus ID*. [Online]
Available at: <https://www.touchnet.com/campus-id/onecard-campus-id>
[Accessed 2025].

Transact, 2023. *Transact Mobile Credential for higher education*. [Online]
Available at: <https://www.transactcampus.com/solutions/campus-id/transact-mobile-credential>
[Accessed 2025].

Truong, P. D., 2025. The three pillars of empiricism. *The three pillars of empiricism*, Issue 2023.

Zhang, S., 11. *The Importance of Computer Network Applications and Security*. [Online]
Available at:
https://www.researchgate.net/publication/381525461_The_Importance_of_Computer_Network_Applications_and_Security
[Accessed 28 11 2025].

9. Appendix:

9.1. Technical Project Terms

9.1.1. Tokenization

In this project tokenization is the process of creating signed payment tokens rather than trusting raw payment requests. Instead of using payment information on its own to approve a transaction, the backend issues-controlled tokens, which include the data like token ID, amount, expiry time, nonce, and signature. Such tokens are subsequently employed in the online and offline payment flows.

This will enhance security since a token can be checked before a transaction is approved. It is also able to easily implement rules like expiry checking, replay rejection and amount restriction. One of the security measures employed to secure the payment process in this project was tokenization.

9.1.2. HMAC-SHA256

In this project, HMAC-SHA256 is a cryptographic technique that is employed to produce and check the signature of the tokens. HMAC abbreviation means Hash-based Message Authentication Code and SHA-256 is a hashing algorithm to generate the signature. It assists in the verification of payment tokens, which were not changed after being issued by the backend.

In this system, important token fields are combined and signed using a secret key known only to the backend. The backend recomputes the signature when a token is subsequently accessed and verifies the signature with the original signature. In case the values are not equal, then the token is invalid. This can help guard against hacking and illegitimate alteration of tokens.

9.1.3. Encrypted Local Storage

Encrypted local storage is the secure storage of sensitive data in the Android device in an encrypted form. In this project, offline tokens are made locally such that even in cases where the internet is not available, offline payments can be made. As such tokens are security sensitive, they should not be stored in plain readable form.

To overcome this, the project encrypts local storage on the Android side. This guarantees that data that involve tokens is secure even when a person attempts to investigate local application storage. An encrypted local storage enhances the offline payment capability and minimizes the chances of exposing sensitive tokens to the device.

9.1.4. Bluetooth Classic RFCOMM

The communication method employed in this project to enable the student Android device to make payments offline to the merchant receiver is Bluetooth Classic RFCOMM. RFCOMM is a Bluetooth protocol, which offers serial-like data communication, so it is an appropriate protocol to exchange structured offline payment payloads between devices.

Under this system, in case of the impossibility of internet-based processing, the payment details are transmitted via Bluetooth Classic to the receiver on the merchant side. This enables the local merchant to receive the request of payment and store it in a pending queue to be synchronized later. Bluetooth Classic was chosen since it was feasible, stable and conformed to the real implementation of the project.

9.1.5. Synchronization and Offline Queue.

Offline queue- The offline queue is a local short term storage device that is used to store the offline payment transactions pending synchronization with the central backend. In this project, when a payment is made offline via Bluetooth Classic, the payment is not confirmed in the main server database immediately. Rather, it is inserted into a local SQLite queue containing associated token fields and transaction metadata.

Synchronization- The later process of transferring these queued records to the backend later when the merchant comes online again is called synchronization. In the synchronization process, the backend verifies token data, expiration status, replaying of transactions and balances and then approves the transaction to the main system. This design permits offline payments to be convenient and still control ultimate validation by the server.

9.1.6. Audit Logging

Audit logging is a process of capturing key system activities to track, review, and analyse activities to ensure security. Audit records are utilized in this project to monitor key happenings like attempts to log in, token generation, payment, replay failure, expiry rejection, and synchronization outcomes.

Audit logging is aimed at offering accountability and traceability. It assists developers and evaluators to have an idea of what occurred in the system in the normal and testing mode. Audit logging in this project proved to be particularly helpful as it assisted in providing evidence of security testing and aiding in the verification of the behaviour of replay-token rejection, expired-token rejection and other controlled test cases.

[Back to 2.2. Understanding the project](#)

9.2. Actual System Architecture Overview

The real system architecture of the College Digital Credential Wallet project is separated into two major sides: the client side, and the server side. Client side consists of the student Android application, the secure local token storage, the offline transaction queue, and the Bluetooth Classic communication when making an offline payment. The server side consists of the Flask API backend and MySQL database and merchant web portal where the transactions, merchant presence, analytics, and synchronization will be managed.

The Android student application in the online workflow is an interface that talks to Flask backend using REST API. The backend then communicates with the MySQL database to authenticate users, issue online payment tokens, validate transaction, update balances and transaction history. The server side also tracks merchant activity to enable the application to know merchants that are accepting payment.

The Android app in the offline workflow utilizes offline tokens that were previously issued and stored safely on the device and transmits the payment payload via Bluetooth Classic RFCOMM to a merchant-side receiver. The central database is not immediately updated with the received transaction. Rather it is held in a local offline queue as a pending transaction. The transaction queue is synchronized with the Flask backend, and upon connectivity, token validity, expiry, and replay status are verified and the final acceptance of the transaction is determined.

The purpose of this architecture was to ensure the separation of online and offline flows, which are related to each other by a shared backend validation model. Consequently, the system can still be operational even in case of lack of internet connection and still maintain central control of security and transactions.

[Back to 3.6.2. System Architecture Diagram](#)

9.3. Extended Implementation

9.3.1. Android Application Details (Continued)

For online payment, the Android application verifies the merchants who are currently accepting payment and presents it to the student. When there is only a single merchant, then this merchant is automatically selected; when there are many merchants, the user can also choose a merchant manually. Once the student finishes entering the amount and optional payment information, the application forwards the request to the backend to process.

For offline payment, it does not need an immediate server payment, but transfers payment information via Bluetooth Classic to a merchant device and is powered by a pre-issued offline token. Data containing local secure tokens also gets stored in the application to facilitate offline behaviour. This online and offline separability was a significant aspect of the ultimate implementation since it enabled the project to be operable even in the absence of the student device being connected to the internet.

[Back to 3.7.1. Android Application Implementation](#)

9.3.2. Backend Service Implementation (Continued)

In the case of online payments, the backend is used to verify the sender and receiver, user role, payment amount, invalid self-payment and ensure that sufficient balance exists before balances are updated and transaction logged. Besides direct online transfer processing, the backend provides short-lived online payment tokens and stores their status, signature, expiry time and any other transaction information. The token lifetime of online payment is set to a short intentionally, and this attribute facilitates replay prevention and expiry testing. In the case of offline payments, the backend sends offline token packs and stores the token metadata that will be used to authenticate them later, including token ID, nonce, expiry time, signature, and amount allowed. The delayed validation model, which is applied during the offline workflow, is supported by this structure, in which the payment is initially buffered locally and subsequently reconciled with the server. The debug routes (replay-token and expired-token) are also located in the backend and were deliberately left there since they help test the evidence needed in the final report.

[Back to 3.7.2. Backend Implementation](#)

9.3.3. Merchant Web Portal Implementation (Continued)

One of the implementation features, on the merchant side, is the control of the Bluetooth Classic offline receiver. The project implements a special controller script to start, stop, and monitor the receiver process instead of all this being embedded within the logic of the web UI. This controller holds process state, logs to a log file, and enables the merchant portal to understand whether the receiver is running. The receiver itself waits to receive Bluetooth Classic RFCOMM payment payloads, checks the validity of the received structure and inserts accepted offline payments into the local SQLite queue as pending transactions. This architecture maintains the offline flow as a module: the web portal manages the service, the receiver deals with Bluetooth communication, and synchronization in the future sends validated results to the backend. It was an expedient implementation decision since it minimized the risk and did not disrupt the already operating offline Bluetooth Classic flow.

[Back to 3.7.3. Merchant Web Portal Implementation](#)

9.3.4. Database and Local Storage Implementation (Continued)

The offline queue holds pending payments along with the token related fields including offline token ID, nonce, expiry time, and signature. The additional fields are necessary since they enable the backend to confirm later that the payment that has been queued is valid and has not been replayed or tampered with during synchronization. Status and error information are also logged in the queue to enable failed sync attempts to be diagnosed. This makes the offline model more realistic and gives definite indications to the report particularly in the security and synchronization tests that have already been done in later chapters. Since the storage design is already an extension of the ERD and DFD figures, the description here must be concise and only based on one or two focused code or schema screenshots.

[Back to 3.7.4. Database and Local Storage Implementation](#)

9.3.5. Security Mechanisms Implementation (Continued)

In the case of the offline route, storage and synchronization security are also available. The Bluetooth receiver does not complete the payment in the central database. Rather, it puts the transaction into a local queue awaiting, and the transaction metadata is a token, and the server-side validation is done later. This helps in avoiding the offline route bypassing the primary token checks. Service state is also monitored with the assistance of receiver controller and receiver logs and offline readiness is provided in the Android side via secure local token storage. The security features were also not only implemented, but also tested by replay-token rejection, expired-token rejection, encrypted local token verification as well as insufficient-balance rejection. Hence, this part must remain implementation-oriented and refer to the testing chapter and appendix to provide evidence in detail.

[Back to 3.7.5. Security Mechanisms Implementation](#)

9.4. Future Works Explained

9.4.1. Biometric Authentication Integration

One way the system could be improved in the future would be to include biometric identification like fingerprint or facial recognition. In the present prototype, user authentication can be done by use of username and password. The biometric authentication may provide an additional security step, particularly when it comes to the sensitive tasks like opening the wallet, verifying payment, or accessing secured information in the student identity. This would enhance security and convenience of users.

9.4.2. Real-Time Notification System

The system currently is centred on core identity and payment capabilities, but new versions might have a real-time notification platform. This feature would inform students of successful payments, failed transactions, low wallet balance, token expiry events, and synchronization status. Real-time alerts might also be used to track the impending offline transactions and activity within the system on the merchant side. This would enhance the level of transparency and user awareness in normal use.

9.4.3. Institution-Wide Multi-Merchant Expansion

The prototype already shows the system within a small merchant setup. The platform can be expanded to the entire institution in the future to accommodate more merchants like canteens, stationeries, libraries, transport counters, and event ticketing points. This would render the system more helpful as an institution-wide online payment and identity system and would be more indicative of actual institutional usage.

9.4.4. Advanced Administrative Dashboard

The project may be enhanced in the future with a special administrative dashboard to the management of the institution. This dashboard may offer general transaction data, merchant activity data, student wallet data, suspicious transaction data, and audit data. This would enhance the monitoring of the system and assist administrators in handling mass deployment better.

9.4.5. Backup and Recovery Based on Clouds

The next enhancement that would be important in the future would be the inclusion of cloud-based backup and recovery support. This would assist in saving valuable information like user data, transaction logs, token metadata and audit information to prevent losses in case of local failure or unintentional loss of data. The recovery mechanisms might be enhanced by cloud-based mechanisms that would enhance reliability, continuity and maintainability of the system in the long term.

9.4.6. Cross-Platform Application Support

The existing student side implementation is built on Android. The system might be enhanced in the future to serve various platforms including iOS and web-based interfaces. The ability to cross-platform would enhance accessibility and make the solution more feasible to a broader spectrum of users and institutions. It would also enhance flexibility in case the project was later evolved into a production ready system.

[Back to 5.3. Future Works](#)

9.5. Gantt Chart

teamgantt
Created with Free Edition

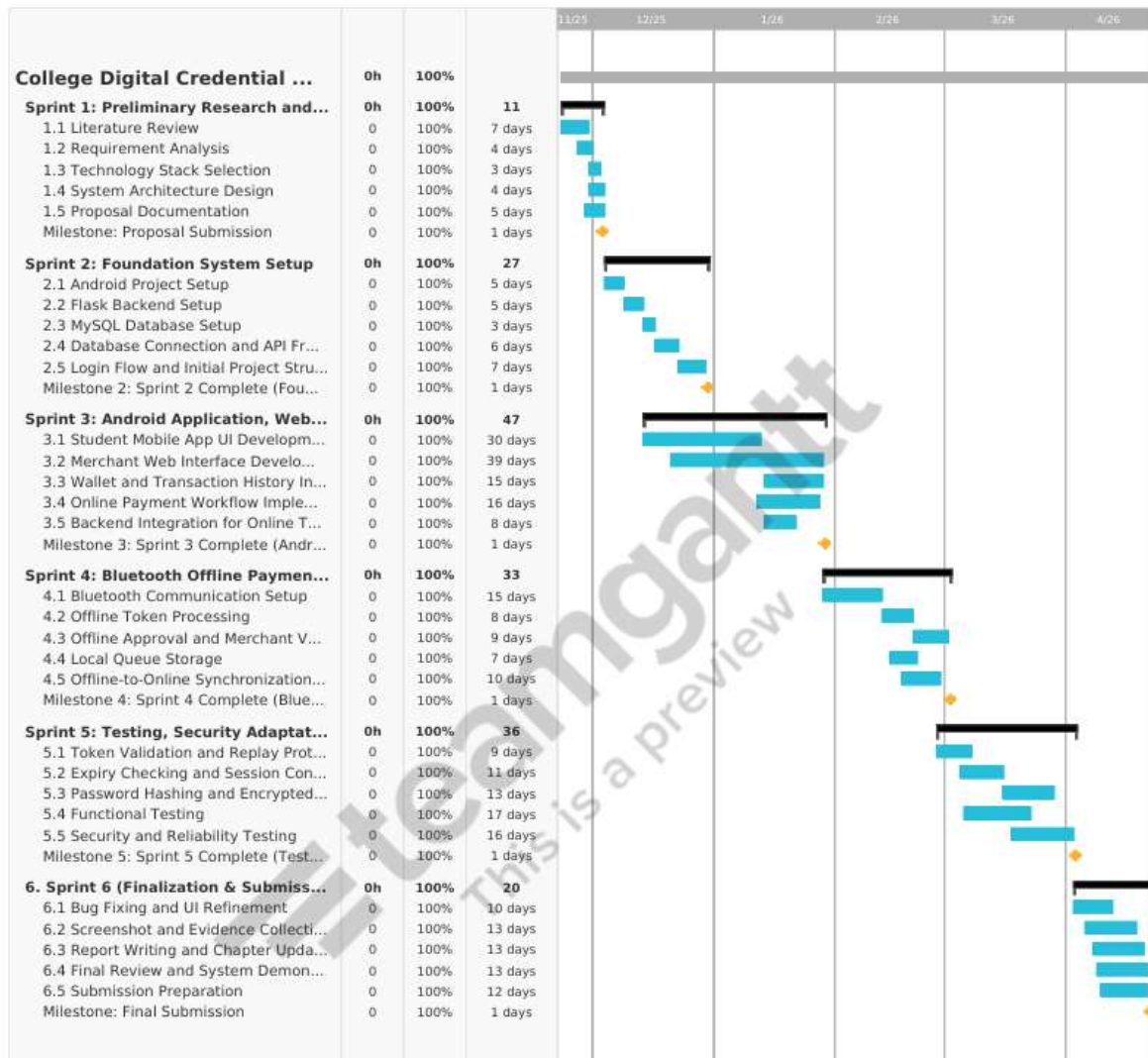


Figure 145: Gantt Chart

[Back to 3.3. Phases of Methodology](#)

4/20/26, 10:48 PM

College Digital Credential Wallet - List | TeamGantt

 Unlock powerful features with Pro Manager. [Try free for 14 days](#)
College Digital Credential Wallet

▼ Sprint 1: Preliminary Research and Planning (100%)		11/23/25	12/3/25
100%	1.1 Literature Review	11/23/25	11/29/25
100%	1.2 Requirement Analysis	11/27/25	11/30/25
100%	1.3 Technology Stack Selection	11/30/25	12/2/25
100%	1.4 System Architecture Design	11/30/25	12/3/25
100%	1.5 Proposal Documentation	11/29/25	12/3/25
	Milestone: Proposal Submission		12/3/25
▼ Sprint 2: Foundation System Setup (100%)		12/4/25	12/30/25
100%	2.1 Android Project Setup	12/4/25	12/8/25
100%	2.2 Flask Backend Setup	12/8/25	12/13/25
100%	2.3 MySQL Database Setup	12/14/25	12/16/25
100%	2.4 Database Connection and API Framework	12/17/25	12/22/25
100%	2.5 Login Flow and Initial Project Structure	12/23/25	12/29/25
	Milestone 2: Sprint 2 Complete (Foundation System Setup)		12/30/25

▼ Sprint 3: Android Application, Web-based Merchant and Online Payment (100%)		12/14/25	1/29/26
100%	3.1 Student Mobile App UI Development	12/14/25	1/12/26
100%	3.2 Merchant Web Interface Development	12/21/25	1/28/26
100%	3.3 Wallet and Transaction History Integration	1/4/26	1/28/26
100%	3.4 Online Payment Workflow Implementation	1/12/26	1/27/26
100%	3.5 Backend Integration for Online Transactions	1/14/26	1/21/26
📦	Milestone 3: Sprint 3 Complete (Android Application, Web-based Merchant and Online Payment)		1/29/26
▼ Sprint 4: Bluetooth Offline Payment Development (100%)		1/29/26	3/2/26
100%	4.1 Bluetooth Communication Setup	1/29/26	2/12/26
100%	4.2 Offline Token Processing	2/13/26	2/20/26
100%	4.3 Offline Approval and Merchant Verification	2/21/26	3/1/26
100%	4.4 Local Queue Storage	2/15/26	2/21/26
100%	4.5 Offline-to-Online Synchronization Logic	2/18/26	2/27/26
📦	Milestone 4: Sprint 4 Complete (Bluetooth Offline Payment Development)		3/2/26
▼ Sprint 5: Testing, Security Adaptations and Improvements (100%)		2/27/26	4/3/26
100%	5.1 Token Validation and Replay Protection	2/27/26	3/7/26
100%	5.2 Expiry Checking and Session Controls	3/5/26	3/15/26
100%	5.3 Password Hashing and Encrypted Storage	3/16/26	3/28/26
100%	5.4 Functional Testing	3/6/26	3/22/26
100%	5.5 Security and Reliability Testing	3/18/26	4/2/26
📦	Milestone 5: Sprint 5 Complete (Testing, Security Adaptations and Improvements)		4/3/26
▼ 6. Sprint 6 (Finalization & Submission) (100%)		4/3/26	4/22/26
100%	6.1 Bug Fixing and UI Refinement	4/3/26	4/12/26
100%	6.2 Screenshot and Evidence Collection	4/6/26	4/18/26
100%	6.3 Report Writing and Chapter Updates	4/8/26	Today
100%	6.4 Final Review and System Demonstration	4/9/26	Tomorrow
100%	6.5 Submission Preparation	4/10/26	Tomorrow
📦	Milestone: Final Submission		Wednesday

Figure 146: Gantt Chart (List View)

9.6. Survey:

9.6.1. Pre-Survey Form

Pre Development User Survey for College Credential Digital Wallet (Secure Digital ID & Campus Payment System)

This survey is part of a Final Year Project on developing a **College Credential Digital Wallet**. It aims to gather user opinions on current college ID systems, payment methods, security concerns, and the acceptance of a secure digital alternative. All responses are confidential. By completing this survey, you confirm that your participation is voluntary and that your responses will be used solely for academic research purposes.

 [Switch account](#) 

 Not shared

* Indicates required question

Full Name *

Your answer

Email *

Your answer

College / University Name *

Your answer

Figure 147: Pre-Survey Form 1

Q1. Do you currently use a physical ID card for college identification? *

☐ Yes

☐ No

Q2. What do you mainly use your college ID for? *

☐ Identity verification

☐ Attendance

☐ Canteen / cafeteria payments

☐ Other: _____

Q3. How do you currently make payments inside the college? *

☐ Cash

☐ QR-based payment

☐ College-issued cards

Q4. Have you ever faced issues making payments due to poor or no internet connectivity inside campus? *

☐ Yes, frequently

☐ Sometimes

☐ Rarely

☐ Never

Figure 148: Pre-Survey Form 2

Q5. What problems have you faced with physical ID cards and QR-based systems? *

- ☐ ID card can be lost, stolen and duplicated
- ☐ Carrying multiple cards is inconvenient
- ☐ QR Codes require internet connection and has slow verification
- ☐ All of the above

Q6. How important is offline payment capability (no internet required) for campus transactions? *

- ☐ Extremely important
- ☐ Important
- ☐ Neutral
- ☐ Not Important

Q7. Would you prefer a single mobile app that replaces: *

i. Physical ID card

ii. Multiple payment methods

iii. Separate access systems

- ☐ Yes
- ☐ No

Figure 149: Pre-Survey Form 3

Q8. How secure do you feel current college ID and payment systems are? *

- ☐ Very Secure
- ☐ Somewhat Secure
- ☐ Neutral
- ☐ Not Secure

Q9. How important are strong security features in a college digital wallet? *

- ☐ Extremely important
- ☐ Important
- ☐ Neutral
- ☐ Not Important

Q10. Which features would you find most useful in a College Digital Credential Wallet? *

- ☐ Digital student ID with secure login & authentication
- ☐ Token-based dynamic and secure payments
- ☐ Offline payments using Bluetooth
- ☐ All of the above

Figure 150: Pre-Survey Form 4

Q11. Would a secure digital credential wallet improve your overall campus experience? *

- ☐ Yes, significantly
- ☐ Yes, Somewhat
- ☐ Not sure
- ☐ No change

Q12. Would you trust a college-issued digital wallet if it used strong encryption and security standards? *

- ☐ Yes
- ☐ Maybe
- ☐ No

Q13. How likely are you to use a College Digital Credential Wallet if implemented by your institution? *

- ☐ Very Likely
- ☐ Likely
- ☐ Unlikely

Figure 151: Pre-Survey Form 5

Q14. Would you recommend such a system to other students or staff if it proved secure and reliable? *

☐ Yes

☐ No

☐ Maybe

Q15. Any additional suggestions or expectations for a College Digital Credential Wallet?

Your answer

Submit Page 1 of 1 [Clear form](#)

Never submit passwords through Google Forms.

This content is neither created nor endorsed by Google. - [Contact form owner](#) - [Terms of Service](#) - [Privacy Policy](#)

Does this form look suspicious? [Report](#)

Google Forms

Figure 152: Pre-Survey Form 6

[Back to 3.4. Survey Analysis](#)

9.6.2. Pre-Survey Form Filled Sample

Pre Development User Survey for College Credential Digital Wallet (Secure Digital ID & Campus Payment System)

This survey is part of a Final Year Project on developing a **College Credential Digital Wallet**. It aims to gather user opinions on current college ID systems, payment methods, security concerns, and the acceptance of a secure digital alternative. All responses are confidential. By completing this survey, you confirm that your participation is voluntary and that your responses will be used solely for academic research purposes.

 [Switch account](#)

 Draft saved

 Not shared

* Indicates required question

Full Name *



Email *



College / University Name *

Islington College

Figure 153: Pre-Survey Form Filled Sample 1

Q1. Do you currently use a physical ID card for college identification? *

☒ Yes

☐ No

Q2. What do you mainly use your college ID for? *

☒ Identity verification

☐ Attendance

☐ Canteen / cafeteria payments

☐ Other: _____

Q3. How do you currently make payments inside the college? *

☐ Cash

☒ QR-based payment

☐ College-issued cards

Q4. Have you ever faced issues making payments due to poor or no internet connectivity inside campus? *

☒ Yes, frequently

☐ Sometimes

☐ Rarely

☐ Never

Figure 154: Pre-Survey Form Filled Sample 2

Q5. What problems have you faced with physical ID cards and QR-based systems? *

- ☐ ID card can be lost, stolen and duplicated
- ☐ Carrying multiple cards is inconvenient
- ☐ QR Codes require internet connection and has slow verification
- ☒ All of the above

Q6. How important is offline payment capability (no internet required) for campus transactions? *

- ☒ Extremely important
- ☐ Important
- ☐ Neutral
- ☐ Not Important

Q7. Would you prefer a single mobile app that replaces: *

i. Physical ID card

ii. Multiple payment methods

iii. Separate access systems

- ☒ Yes
- ☐ No

Figure 155: Pre-Survey Form Filled Sample 3

Q8. How secure do you feel current college ID and payment systems are? *

- ☒ Very Secure
- ☐ Somewhat Secure
- ☐ Neutral
- ☐ Not Secure

Q9. How important are strong security features in a college digital wallet? *

- ☒ Extremely important
- ☐ Important
- ☐ Neutral
- ☐ Not Important

Q10. Which features would you find most useful in a College Digital Credential Wallet? *

- ☐ Digital student ID with secure login & authentication
- ☐ Token-based dynamic and secure payments
- ☐ Offline payments using Bluetooth
- ☒ All of the above

Figure 156: Pre-Survey Form Filled Sample 4

Q11. Would a secure digital credential wallet improve your overall campus experience? *

- ☒ Yes, significantly
- ☐ Yes, Somewhat
- ☐ Not sure
- ☐ No change

Q12. Would you trust a college-issued digital wallet if it used strong encryption and security standards? *

- ☒ Yes
- ☐ Maybe
- ☐ No

Q13. How likely are you to use a College Digital Credential Wallet if implemented by your institution? *

- ☒ Very Likely
- ☐ Likely
- ☐ Unlikely

Figure 157: Pre-Survey Form Filled Sample 5

Q13. How likely are you to use a College Digital Credential Wallet if implemented by your institution? *

☒ Very Likely

☐ Likely

☐ Unlikely

Q14. Would you recommend such a system to other students or staff if it proved secure and reliable? *

☒ Yes

☐ No

☐ Maybe

Q15. Any additional suggestions or expectations for a College Digital Credential Wallet?

Great Project. Would definitely try it.

Submit Page 1 of 1 [Clear form](#)

Never submit passwords through Google Forms.

This content is neither created nor endorsed by Google. - [Contact form owner](#) - [Terms of Service](#) - [Privacy Policy](#)

Does this form look suspicious? [Report](#)

Google Forms

Figure 158: Pre-Survey Form Filled Sample 6

[Back to 3.4. Survey Analysis](#)

9.6.3. Pre-Survey Response

Q1. Do you currently use a physical ID card for college identification?

14 responses

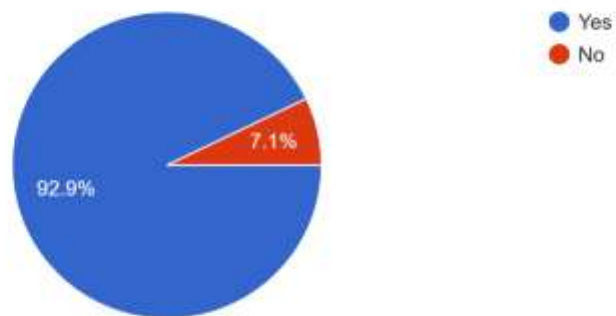


Figure 159: Pre-Survey Response 1

Q2. What do you mainly use your college ID for?

14 responses

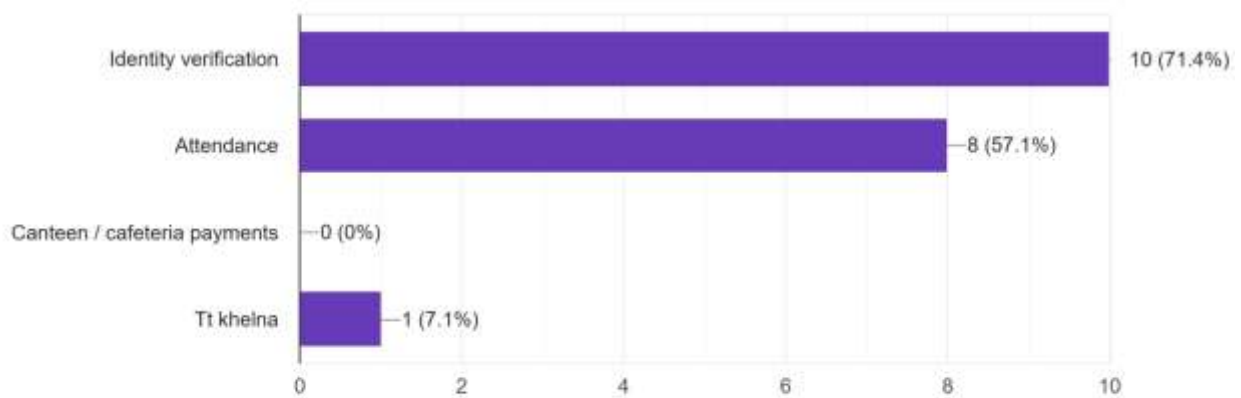


Figure 160: Pre-Survey Response 2

Q3. How do you currently make payments inside the college?

14 responses

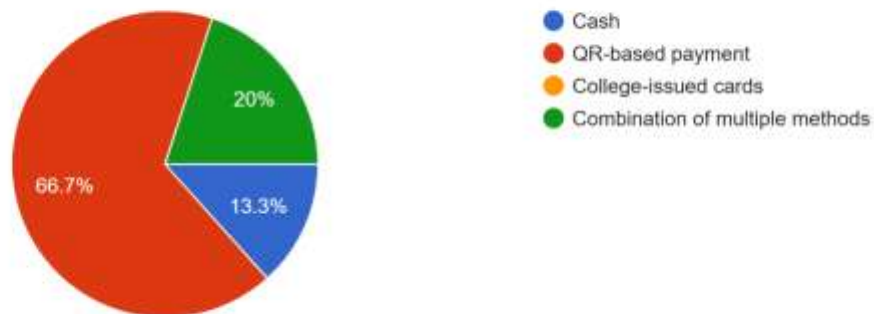


Figure 161: Pre-Survey Response 3

Q4. Have you ever faced issues making payments due to poor or no internet connectivity inside campus?

14 responses

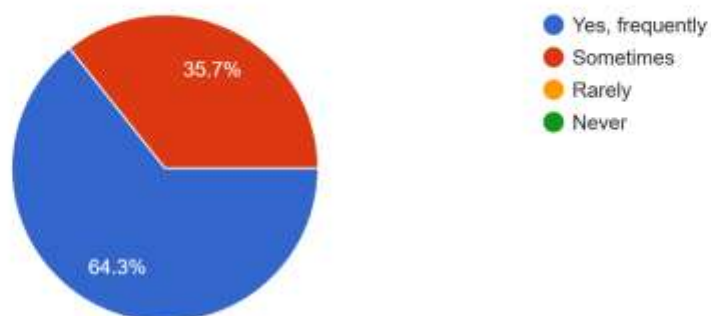


Figure 162: Pre-Survey Response 4

Q5. What problems have you faced with physical ID cards and QR-based systems?

14 responses

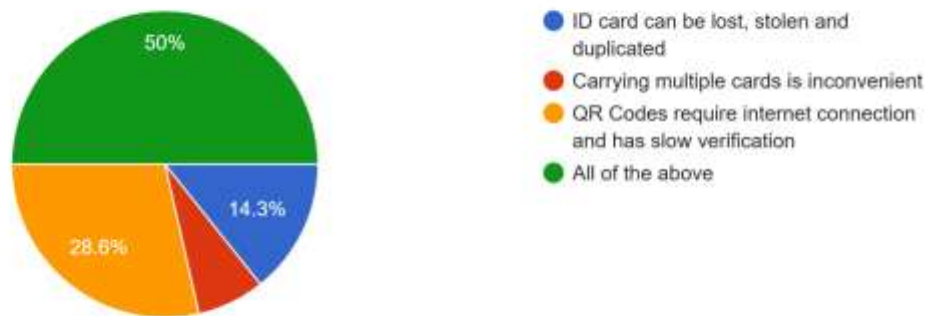


Figure 163: Pre-Survey Response 5

Q6. How important is offline payment capability (no internet required) for campus transactions?

14 responses

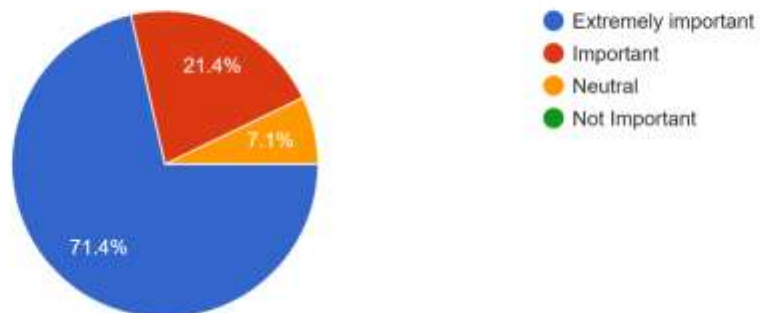


Figure 164: Pre-Survey Response 6

Q7. Would you prefer a single mobile app that replaces: i. Physical ID card ii. Multiple payment methods iii. Separate access systems

14 responses



Figure 165: Pre-Survey Response 7

Q8. How secure do you feel current college ID and payment systems are?

14 responses

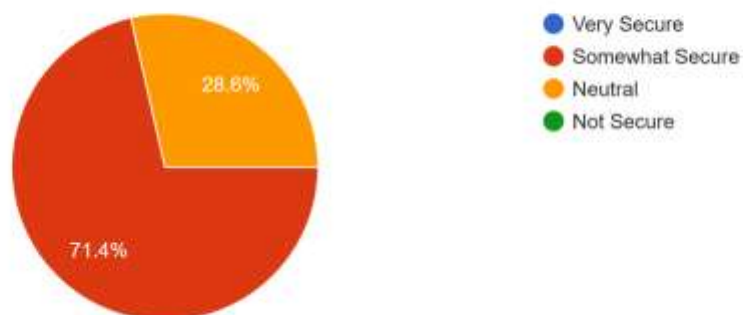


Figure 166: Pre-Survey Response 8

Q9. How important are strong security features in a college digital wallet?

14 responses

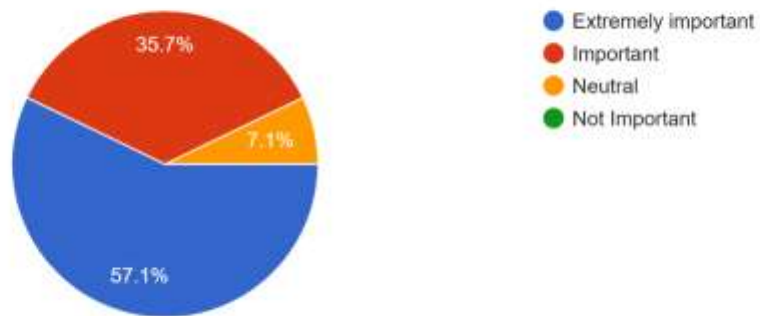


Figure 167: Pre-Survey Response 9

Q10. Which features would you find most useful in a College Digital Credential Wallet?

14 responses



Figure 168: Pre-Survey Response 10

Q11. Would a secure digital credential wallet improve your overall campus experience?

14 responses

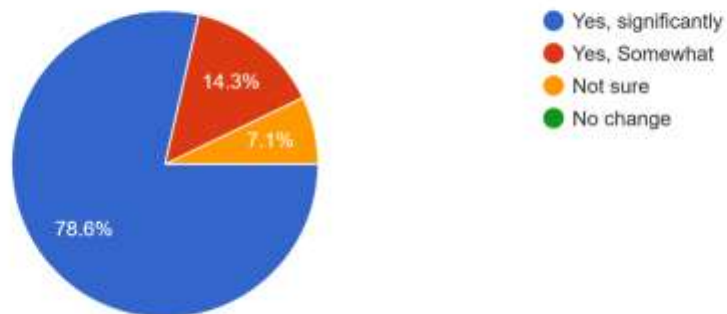


Figure 169: Pre-Survey Response 11

Q12. Would you trust a college-issued digital wallet if it used strong encryption and security standards?

14 responses

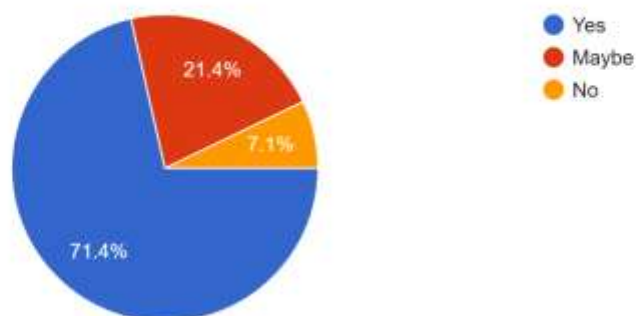


Figure 170: Pre-Survey Response 12

Q13. How likely are you to use a College Digital Credential Wallet if implemented by your institution?

14 responses

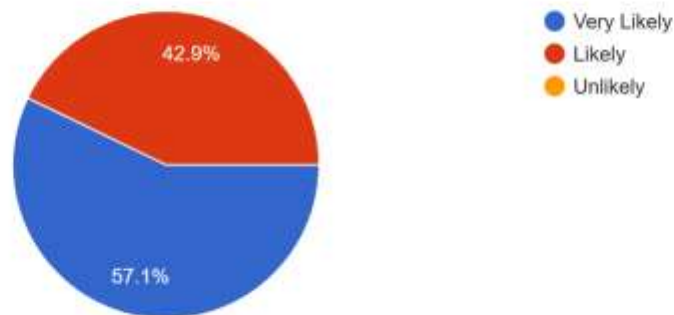


Figure 171: Pre-Survey Response 13

Q14. Would you recommend such a system to other students or staff if it proved secure and reliable?

14 responses

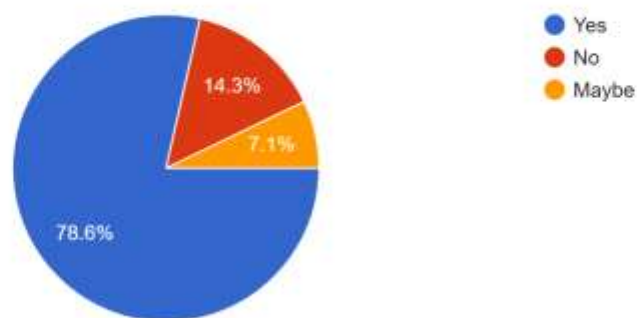


Figure 172: Pre-Survey Response 14

[Back to 3.4. Survey Analysis](#)

9.6.4. Post-Survey Form

Post-Evaluation Survey for College Digital Credential Wallet System

This survey is intended to collect feedback after using the College Digital Credential Wallet prototype. Your responses will help evaluate the usability, functionality, reliability, and overall effectiveness of the system.



Not shared

* Indicates required question

Q1. How easy was it to log in and access the system? *

- ☐ A. Very easy
- ☐ B. Easy
- ☐ C. Difficult
- ☐ D. Very Difficult

Q2. How clear and understandable was the user interface of the mobile application?



- ☐ A. Very clear
- ☐ B. Clear
- ☐ C. Somewhat confusing
- ☐ D. Very confusing

Figure 173: Post-Survey Form 1

Q3. How easy was it to view your digital student ID in the application? *

- ☐ A. Very easy
- ☐ B. Easy
- ☐ C. Difficult
- ☐ D. Very difficult

Q4. How easy was it to check wallet balance and transaction history? *

- ☐ A. Very easy
- ☐ B. Easy
- ☐ C. Difficult
- ☐ D. Very difficult

Q5. How satisfied were you with the online payment process? *

- ☐ A. Very satisfied
- ☐ B. Satisfied
- ☐ C. Dissatisfied
- ☐ D. Very dissatisfied

Figure 174: Post-Survey Form 2

Q6. How effective was the offline Bluetooth payment feature when internet access was unavailable? *

- ☐ A. Very effective
 - ☐ B. Effective
 - ☐ C. Slightly effective
 - ☐ D. Not effective
-

Q7. How quickly did the system respond during payment operations? *

- ☐ A. Very fast
 - ☐ B. Fast
 - ☐ C. Slow
 - ☐ D. Very slow
-

Q8. How reliable did the system feel during your testing? *

- ☐ A. Very reliable
 - ☐ B. Reliable
 - ☐ C. Slightly unreliable
 - ☐ D. Unreliable
-

Figure 175: Post-Survey Form 3

Q9. How secure did you feel while using the digital wallet system? *

- ☐ A. Very secure
- ☐ B. Secure
- ☐ C. Slightly secure
- ☐ D. Not secure

Q10. How useful was the combination of digital ID and payment features in one application? *

- ☐ A. Very useful
- ☐ B. Useful
- ☐ C. Slightly useful
- ☐ D. Not useful

Q11. How helpful was the offline transaction sync feature after reconnecting to the server? *

- ☐ A. Very helpful
- ☐ B. Helpful
- ☐ C. Slightly helpful
- ☐ D. Not helpful

Figure 176: Post-Survey Form 4

Q12. Compared to physical ID cards and existing payment methods, how convenient was this system? *

- ☐ A. Much more convenient
- ☐ B. More convenient
- ☐ C. Less convenient
- ☐ D. Much less convenient

Q13. How satisfied were you with the overall performance of the system prototype? *

- ☐ A. Very satisfied
- ☐ B. Satisfied
- ☐ C. Dissatisfied
- ☐ D. Very dissatisfied

Q14. Would you be willing to use this system regularly if implemented by your college? *

- ☐ A. Definitely yes
- ☐ B. Probably yes
- ☐ C. Probably no
- ☐ D. Definitely no

Figure 177: Post-Survey Form 5

Q15. Would you recommend this system to other students and staff? *

☐ A. Definitely yes

☐ B. Probably yes

☐ C. Probably no

☐ D. Definitely no

Submit **Clear form**

Figure 178: Post-Survey Form 6

[Back to 3.4. Survey Analysis](#)

9.6.5. Post-Survey Form Filled Sample

Post-Evaluation Survey for College Digital Credential Wallet System

This survey is intended to collect feedback after using the College Digital Credential Wallet prototype. Your responses will help evaluate the usability, functionality, reliability, and overall effectiveness of the system.

[Switch account](#)

 Not shared

 Draft saved

* Indicates required question

Q1. How easy was it to log in and access the system? *

☒ A. Very easy

☐ B. Easy

☐ C. Difficult

☐ D. Very Difficult

Q2. How clear and understandable was the user interface of the mobile application? *

☒ A. Very clear

☐ B. Clear

☐ C. Somewhat confusing

☐ D. Very confusing

Figure 179: Post-Survey Form Filled Sample 1

Q3. How easy was it to view your digital student ID in the application? *

- ☒ A. Very easy
- ☐ B. Easy
- ☐ C. Difficult
- ☐ D. Very difficult

Q4. How easy was it to check wallet balance and transaction history? *

- ☒ A. Very easy
- ☐ B. Easy
- ☐ C. Difficult
- ☐ D. Very difficult

Q5. How satisfied were you with the online payment process? *

- ☒ A. Very satisfied
- ☐ B. Satisfied
- ☐ C. Dissatisfied
- ☐ D. Very dissatisfied

Figure 180: Post-Survey Form Filled Sample 2

Q6. How effective was the offline Bluetooth payment feature when internet access was unavailable? *

- ☒ A. Very effective
 - ☐ B. Effective
 - ☐ C. Slightly effective
 - ☐ D. Not effective
-

Q7. How quickly did the system respond during payment operations? *

- ☒ A. Very fast
 - ☐ B. Fast
 - ☐ C. Slow
 - ☐ D. Very slow
-

Q8. How reliable did the system feel during your testing? *

- ☒ A. Very reliable
 - ☐ B. Reliable
 - ☐ C. Slightly unreliable
 - ☐ D. Unreliable
-

Figure 181: Post-Survey Form Filled Sample 3

Q9. How secure did you feel while using the digital wallet system? *

- ☒ A. Very secure
- ☐ B. Secure
- ☐ C. Slightly secure
- ☐ D. Not secure

Q10. How useful was the combination of digital ID and payment features in one application? *

- ☒ A. Very useful
- ☐ B. Useful
- ☐ C. Slightly useful
- ☐ D. Not useful

Q11. How helpful was the offline transaction sync feature after reconnecting to the server? *

- ☒ A. Very helpful
- ☐ B. Helpful
- ☐ C. Slightly helpful
- ☐ D. Not helpful

Figure 182: Post-Survey Form Filled Sample 4

Q12. Compared to physical ID cards and existing payment methods, how convenient was this system? *

- ☒ A. Much more convenient
- ☐ B. More convenient
- ☐ C. Less convenient
- ☐ D. Much less convenient

Q13. How satisfied were you with the overall performance of the system prototype? *

- ☒ A. Very satisfied
- ☐ B. Satisfied
- ☐ C. Dissatisfied
- ☐ D. Very dissatisfied

Q14. Would you be willing to use this system regularly if implemented by your college? *

- ☒ A. Definitely yes
- ☐ B. Probably yes
- ☐ C. Probably no
- ☐ D. Definitely no

Figure 183: Post-Survey Form Filled Sample 5

Q15. Would you recommend this system to other students and staff? *

☒ A. Definitely yes

☐ B. Probably yes

☐ C. Probably no

☐ D. Definitely no

Submit Clear form

Figure 184: Post-Survey Form Filled Sample 6

[Back to 3.4. Survey Analysis](#)

9.6.6. Post-Survey Response

Q1. How easy was it to log in and access the system?

20 responses

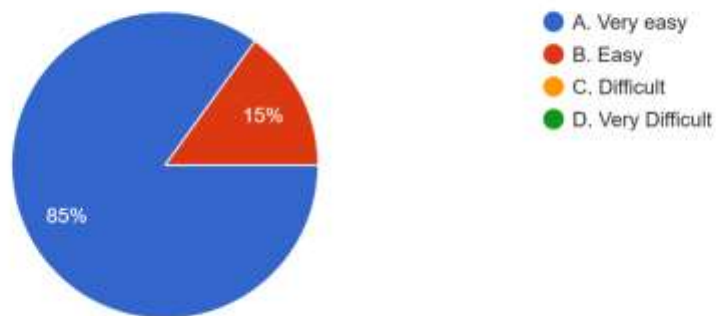


Figure 185: Post-Survey Response 1

Q2. How clear and understandable was the user interface of the mobile application?

20 responses

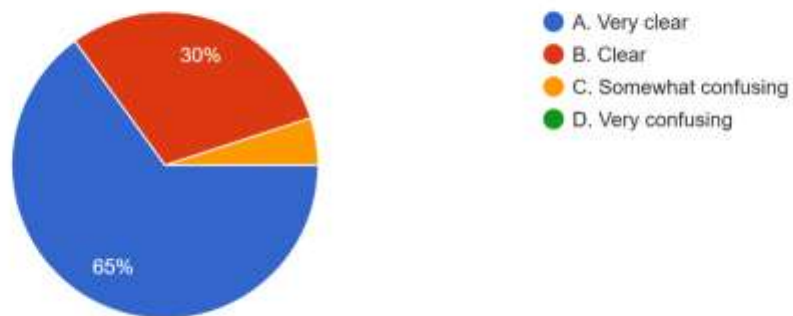


Figure 186: Post-Survey Response 2

Q3. How easy was it to view your digital student ID in the application?

20 responses

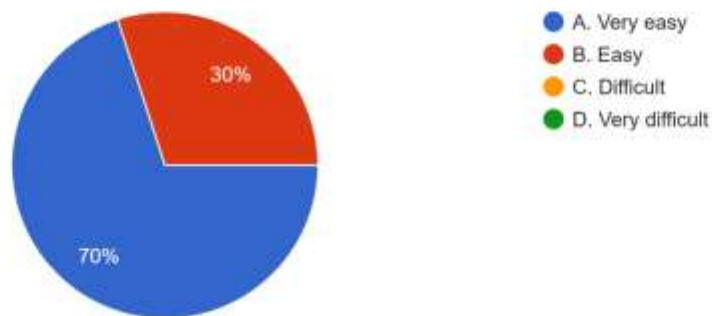


Figure 187: Post-Survey Response 3

Q4. How easy was it to check wallet balance and transaction history?

20 responses

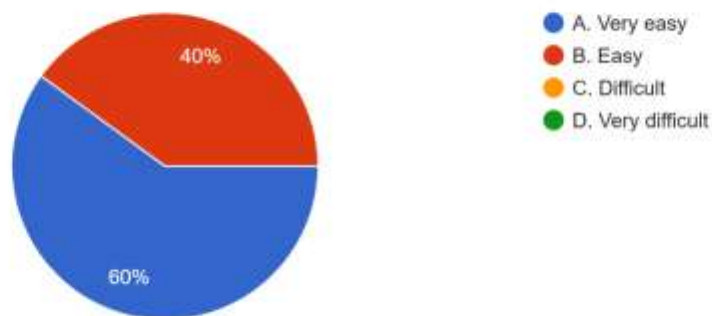


Figure 188: Post-Survey Response 4

Q5. How satisfied were you with the online payment process?

20 responses

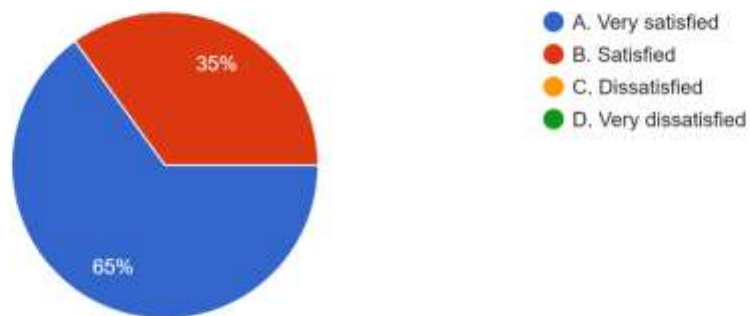


Figure 189: Post-Survey Response 5

Q6. How effective was the offline Bluetooth payment feature when internet access was unavailable?

20 responses

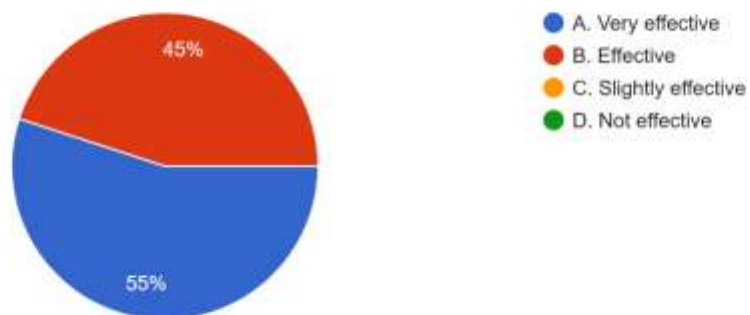


Figure 190: Post-Survey Response 6

Q7. How quickly did the system respond during payment operations?

20 responses

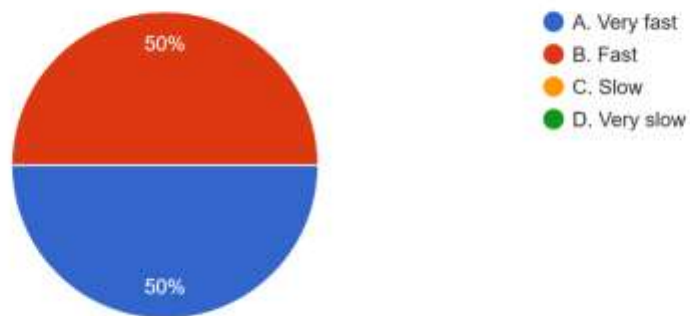


Figure 191: Post-Survey Response 7

Q8. How reliable did the system feel during your testing?

20 responses

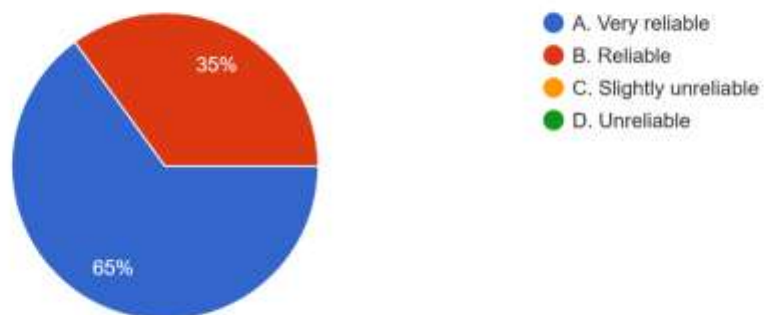


Figure 192: Post-Survey Response 8

Q9. How secure did you feel while using the digital wallet system?

20 responses

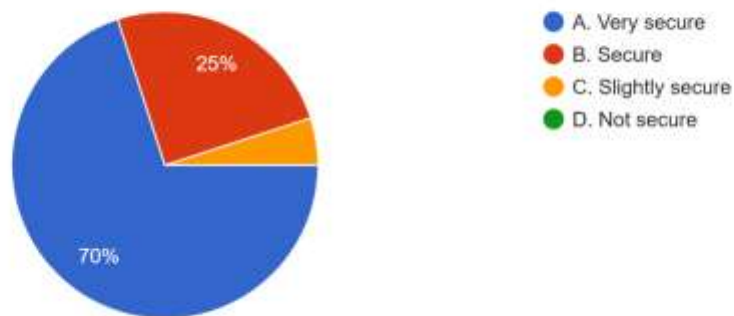


Figure 193: Post-Survey Response 9

Q10. How useful was the combination of digital ID and payment features in one application?

20 responses

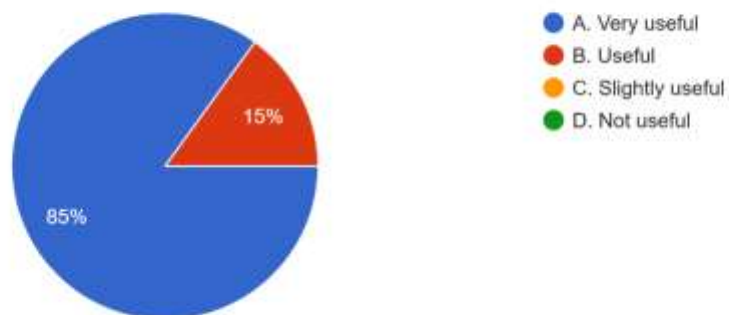


Figure 194: Post-Survey Response 10

Q11. How helpful was the offline transaction sync feature after reconnecting to the server?

20 responses

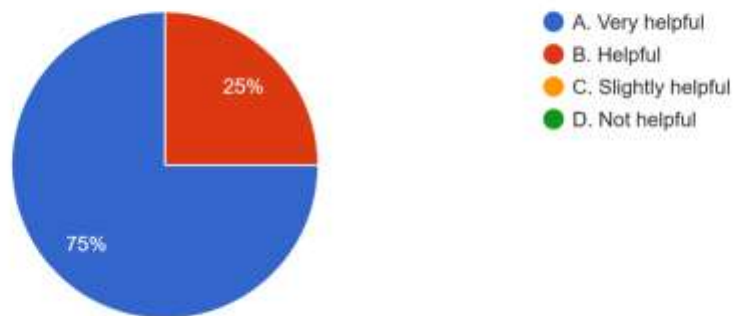


Figure 195: Post-Survey Response 11

Q12. Compared to physical ID cards and existing payment methods, how convenient was this system?

20 responses

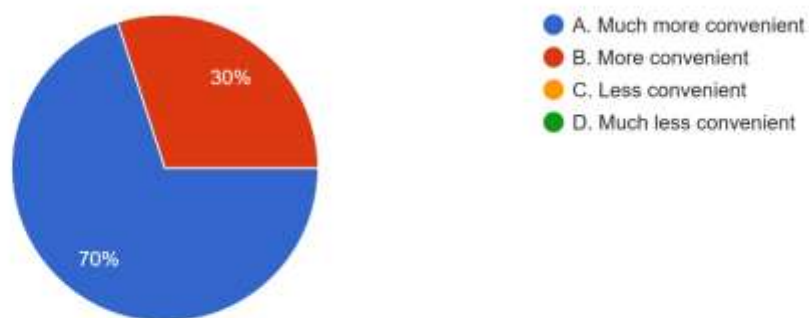


Figure 196: Post-Survey Response 12

Q13. How satisfied were you with the overall performance of the system prototype?

20 responses

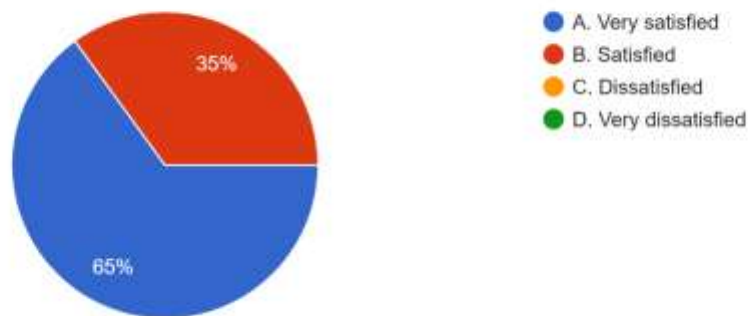


Figure 197: Post-Survey Response 13

Q14. Would you be willing to use this system regularly if implemented by your college?

20 responses

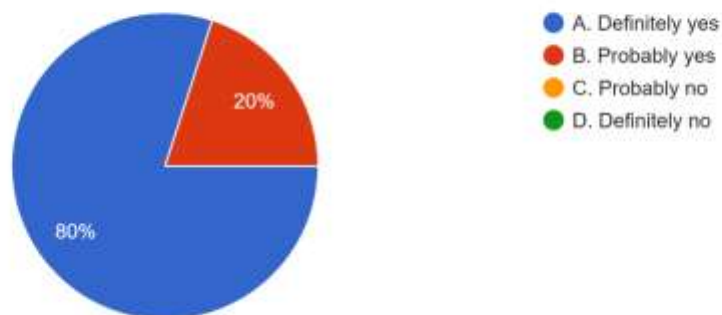


Figure 198: Post-Survey Response 14

Q15. Would you recommend this system to other students and staff?

20 responses

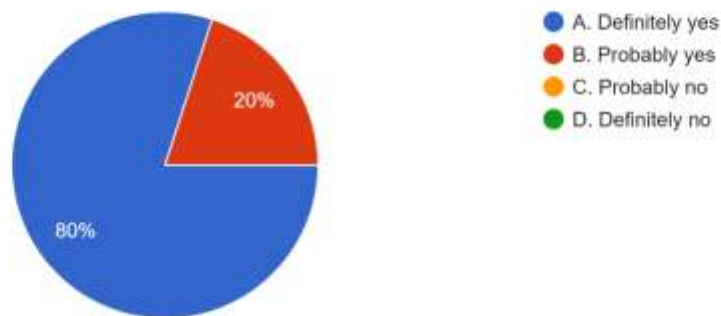


Figure 199: Post-Survey Response 15

[Back to 3.4. Survey Analysis](#)

Area	Lesson Learned from TalkTalk Case
Legacy systems	Old or inherited webpages can remain vulnerable if they are not reviewed and maintained.
Monitoring	Earlier attack attempts must be investigated properly, otherwise the same weakness can be exploited again.
Input handling	User input must not be directly inserted into SQL queries.
Data protection	Customer/account data must be protected from unnecessary exposure.
Remediation	Known vulnerabilities require timely fixes using secure coding controls.